STREAMING VIDEOS

TEAM GUIDE, IN PLAIN WORDS

Group D · Kacem · Daniel · Marouane · Dhia

Advanced Problem Solving

HOW TO READ THIS GUIDE

One page per click of the talk. Each page shows the screen, what appears on it, and what to say in simple words. Lines in a bar are suggested wording, not a script to learn by heart. Numbers on screen always come from our files, never typed by hand.

ON THE DAY

Open the site, press F for full screen and S for the speaker view (notes, timers, next step). Clicker or → moves one step. B blacks the screen. If the laptop fails, the slides PDF has every step.

RUN OF SHOW

17 minutes of talk in the professor's four parts, then 10 minutes of questions.

#	SCENE	PART	SPEAKER	TIME	CLICKS
01	Streaming Videos	Algorithm	Daniel	0.3 min	2
02	The problem in one minute	Algorithm	Daniel	0.7 min	4
03	Why it is hard, and our plan	Algorithm	Kacem	0.8 min	4
04	Running it on the example, pop by pop	Algorithm	Kacem	1.7 min	8
05	When "best per MB" is fooled	Algorithm	Kacem	1.2 min	5
06	Two caches at once, then exact	Algorithm	Kacem	1.3 min	5
07	Experimental protocol	Experiments	Dhia	1 min	4
08	The official result: local contest	Experiments	Dhia	1.5 min	4
09	The class benchmark: from 6th to 2nd	Experiments	Dhia	1.5 min	4
10	Every choice was measured	Experiments	Marouane	1 min	3
11	SWOT	Experiments	Marouane	1 min	4
12	How the final submission is built	Code	Marouane	1.5 min	3
13	Implementation choices	Code	Marouane	1.5 min	4
14	Since our last presentation	Conclusion	Daniel	1 min	5
15	Conclusion	Conclusion	Daniel	1 min	3
16	Backup · Playground	Questions	Everyone	backup	1
17	Backup · Why each step is safe	Questions	Everyone	backup	1
18	Backup · Cost of each step	Questions	Everyone	backup	1

THE BIG PICTURE, AS A STORY

Picture a huge warehouse far out of town that stocks every product, and a few small corner shops close to where people live. Each shop has only a few shelves. If a shop has what you want, you get it quickly. If not, you wait for the warehouse.

In our problem the warehouse is the data center, the corner shops are the cache servers, the shelves are megabytes, the products are videos, and the neighbourhoods are the endpoints (groups of users). We know in advance how many times each neighbourhood will ask for each video.

We choose which videos to put on which shelves. The score is the average time saved per request. Two things make it hard: shelves are small (a knapsack problem), and shops share customers, so a second copy of a video nearby is often worth nothing.

Our answer is a stack of methods. A fast greedy fills the shelves with the best saving per MB first. Then we re-pack one shop at a time exactly. Then we empty two neighbouring shops and rebuild them together (LNS). For small problems, an exact solver finds the proven best answer, and for big ones it tells us how far from perfect we can be at most.

WORDS WE USE (1/2)

Data center
The far place that holds every video. Always available, always slow.

Cache (cache server)
A small, fast storage close to some users, with a fixed size in MB.

Endpoint
A group of users in one place. Each endpoint is linked to some caches (maybe none).

Latency
Waiting time between a request and the video arriving, in milliseconds (ms).

Request description
A line of the input: "this endpoint will ask for this video this many times".

Score
Average time saved per request, in microseconds (µs): the number Google ranks by.

Greedy
Build the answer one best-looking choice at a time, without going back.

Saving per MB
How much time a copy saves, divided by its size. The greedy sorts by it.

Heap (priority queue)
A structure that always gives the item with the highest value first.

Lazy heap
We only re-check the top item. Values only go down, so the top is safe if unchanged.

Evict
Remove a copy that saves nothing any more.

Swap
Replace the weakest copy in a cache by a better video when it strictly helps.

Knapsack
Choose items with sizes and values to fit a bag and get the highest total value.

WORDS WE USE (2/2)

Dynamic programming (DP)
Solve a big problem by filling a table of small answers. Gives the exact knapsack answer.

Re-pack
Freeze every other cache and fill one cache exactly with a knapsack.

LNS
Large neighbourhood search: empty a few related caches, rebuild them, keep it if not worse.

Simulated annealing
A search that sometimes accepts a worse answer to escape a dead end. We tested it; LNS won.

Integer program (MILP)
The whole problem written as equations with 0/1 variables, given to a solver (HiGHS).

Branch and bound
How the solver searches: split the problem, and skip parts that cannot beat the best so far.

Proven optimum
An answer the exact solver proves nobody can beat.

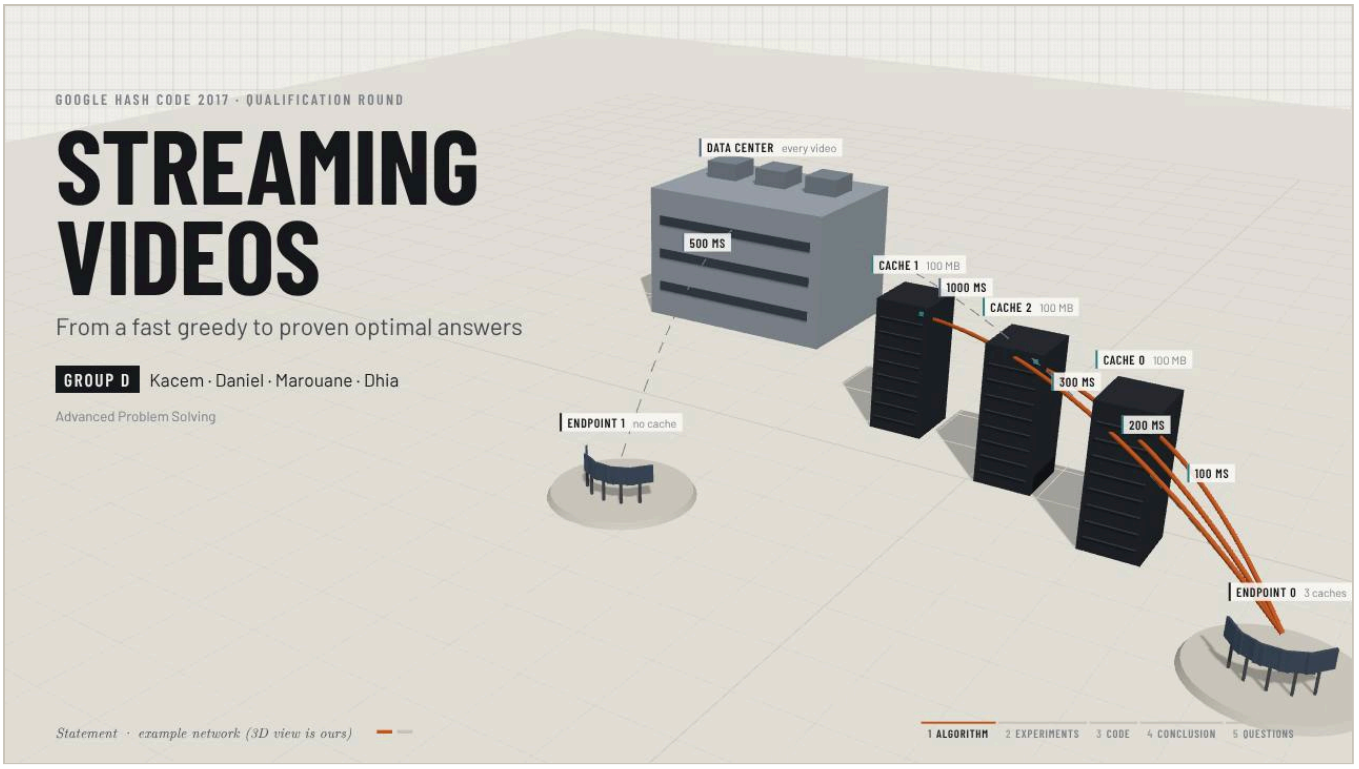
Upper bound (ceiling)
A number no answer can pass. On videos_worth_spreading: 620,009.

VBS / best in class
The best score any group reached on a data set.

NP-hard
No known method solves every case both exactly and fast.

gcd
Greatest common divisor. Dividing all sizes by it keeps the same answer with a smaller table.

Trap instance
A data set built so that the greedy gets about half of the best possible score.



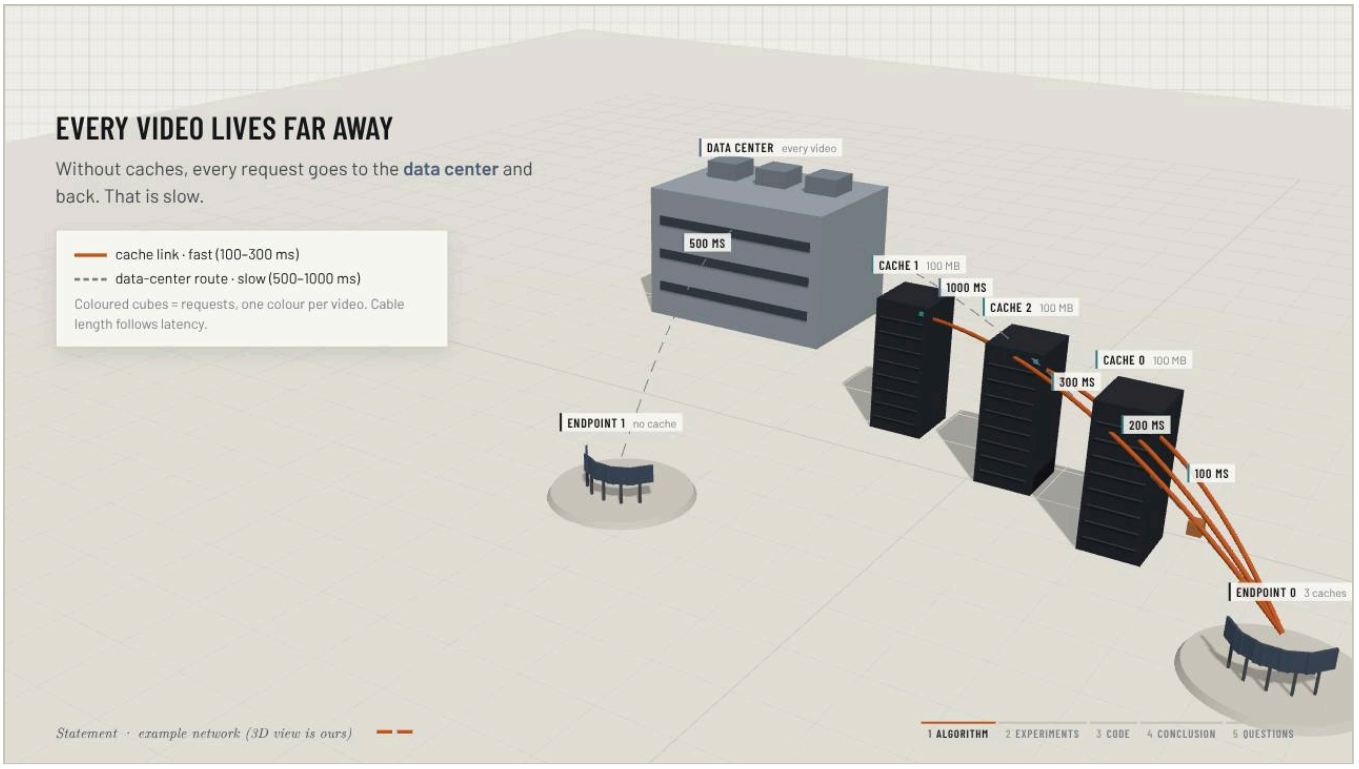
WHAT YOU SEE

Title. The 3D network turns slowly behind: data center at the back, three caches, two groups of users.

IN PLAIN WORDS

> Hello, we are Group D. Our problem is Hash Code 2017, Streaming Videos.

Word to know · Data center The far place that holds every video. Always available, always slow.



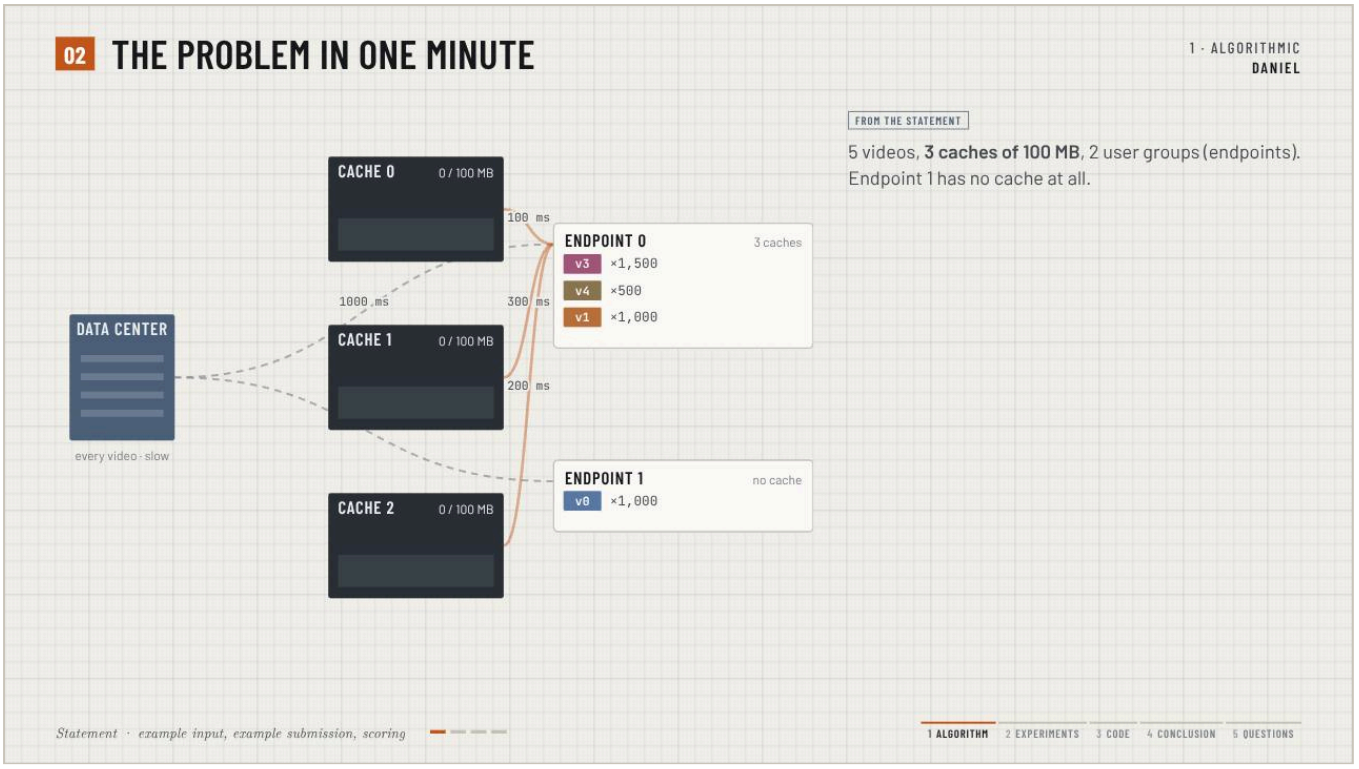
WHAT YOU SEE

Requests start flowing: every cube goes all the way to the data center and back.

IN PLAIN WORDS

> A video service keeps every video in one big data center, far from the users. Every request travels there and back, which is slow.

> Near the users there are a few small cache servers. Our job is to decide which videos to copy into them.



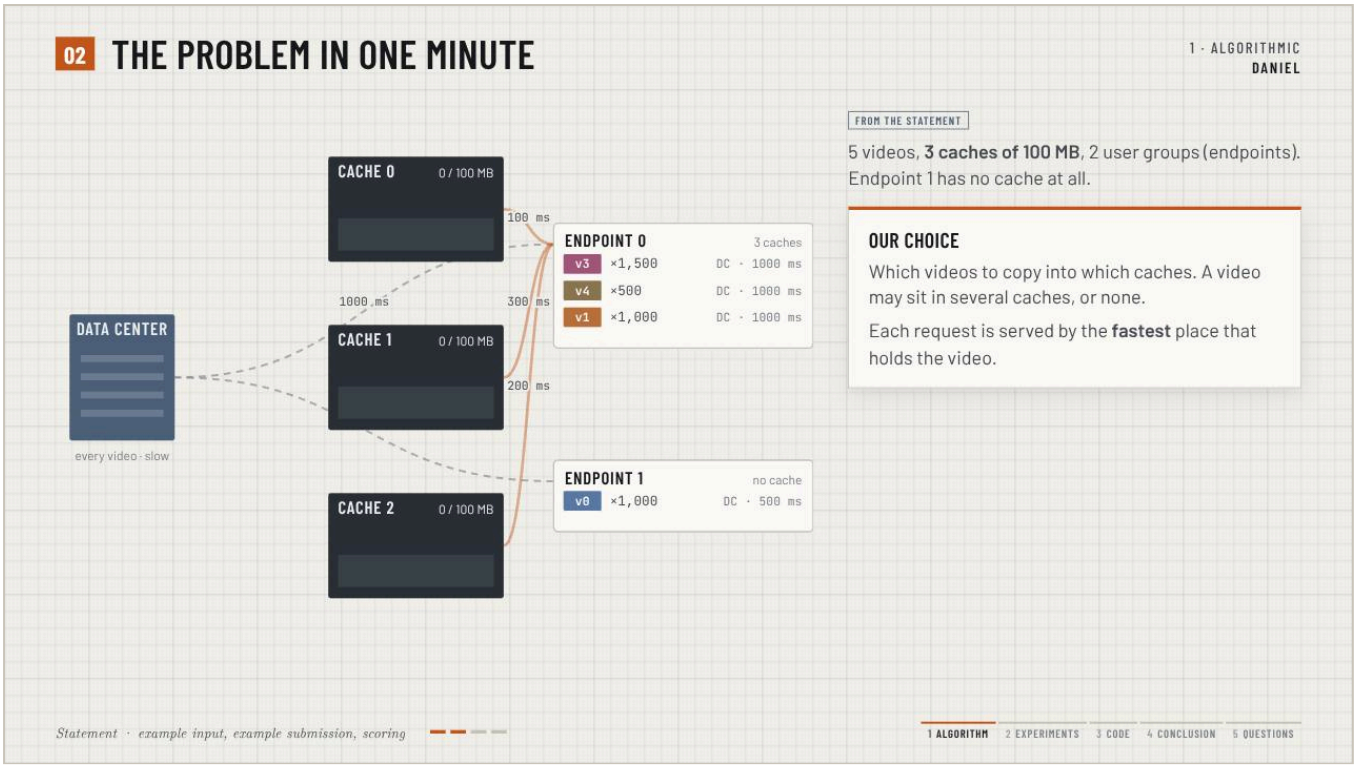
WHAT YOU SEE

The statement's own example: 5 videos, 3 caches of 100 MB, 2 endpoints.

IN PLAIN WORDS

> This is the example from the statement. Endpoint 0 can reach three caches. Endpoint 1 has no cache at all.

Word to know · Score Average time saved per request, in microseconds (µs): the number Google ranks by.

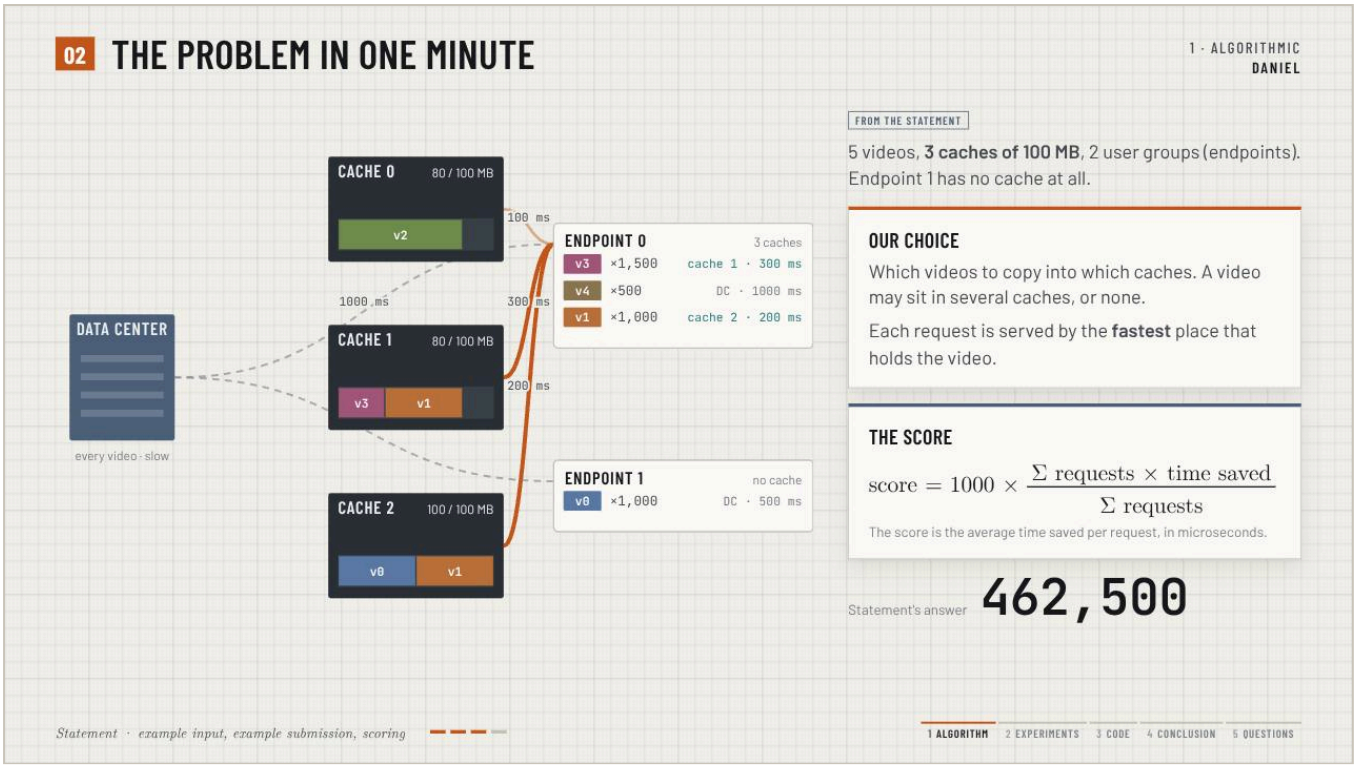


WHAT YOU SEE

Our choice + the rule. The request cards now show where each request is served.

IN PLAIN WORDS

> We choose which videos go into which caches. A request is served by the fastest place that holds the video. Right now, that is always the data center.

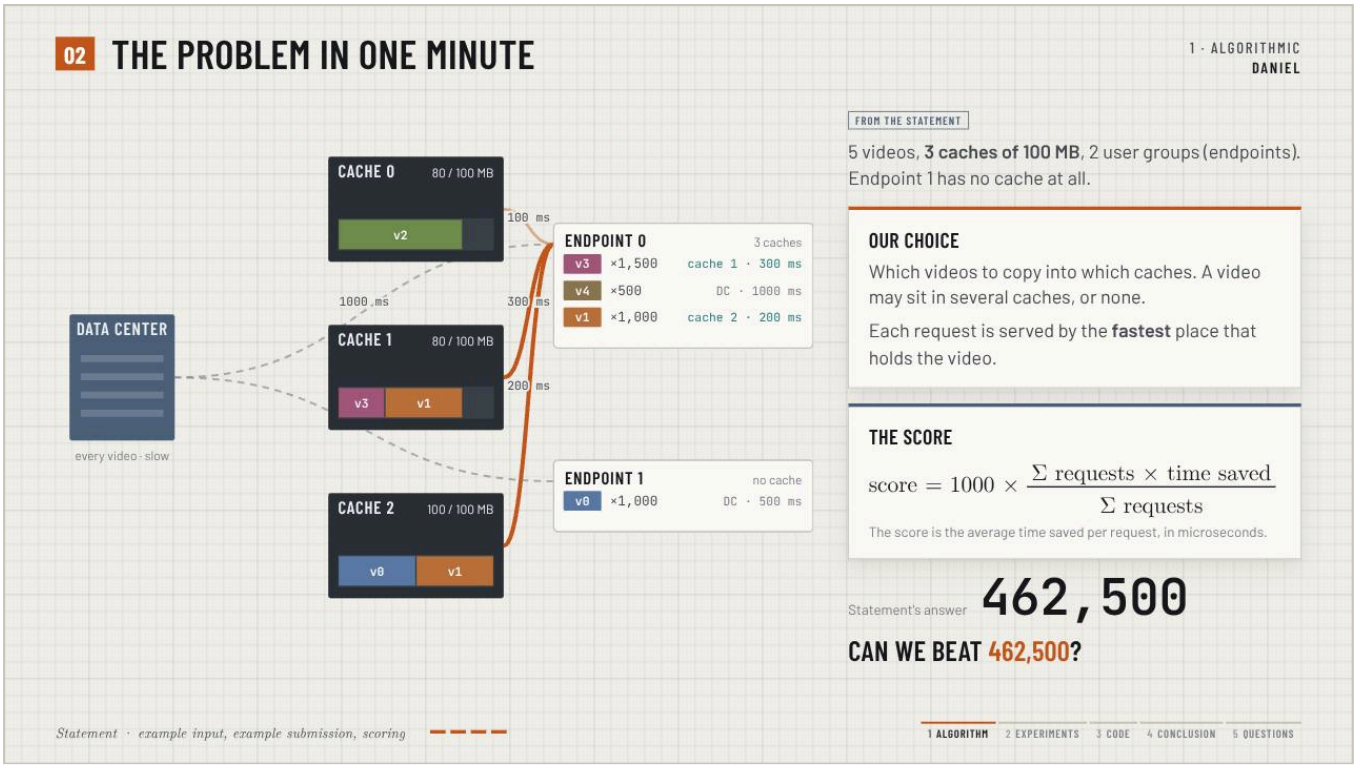


WHAT YOU SEE

The statement's answer is placed; the score counts up to 462,500.

IN PLAIN WORDS

> The score is the average time saved per request, in microseconds. The statement's own answer gets 462,500.



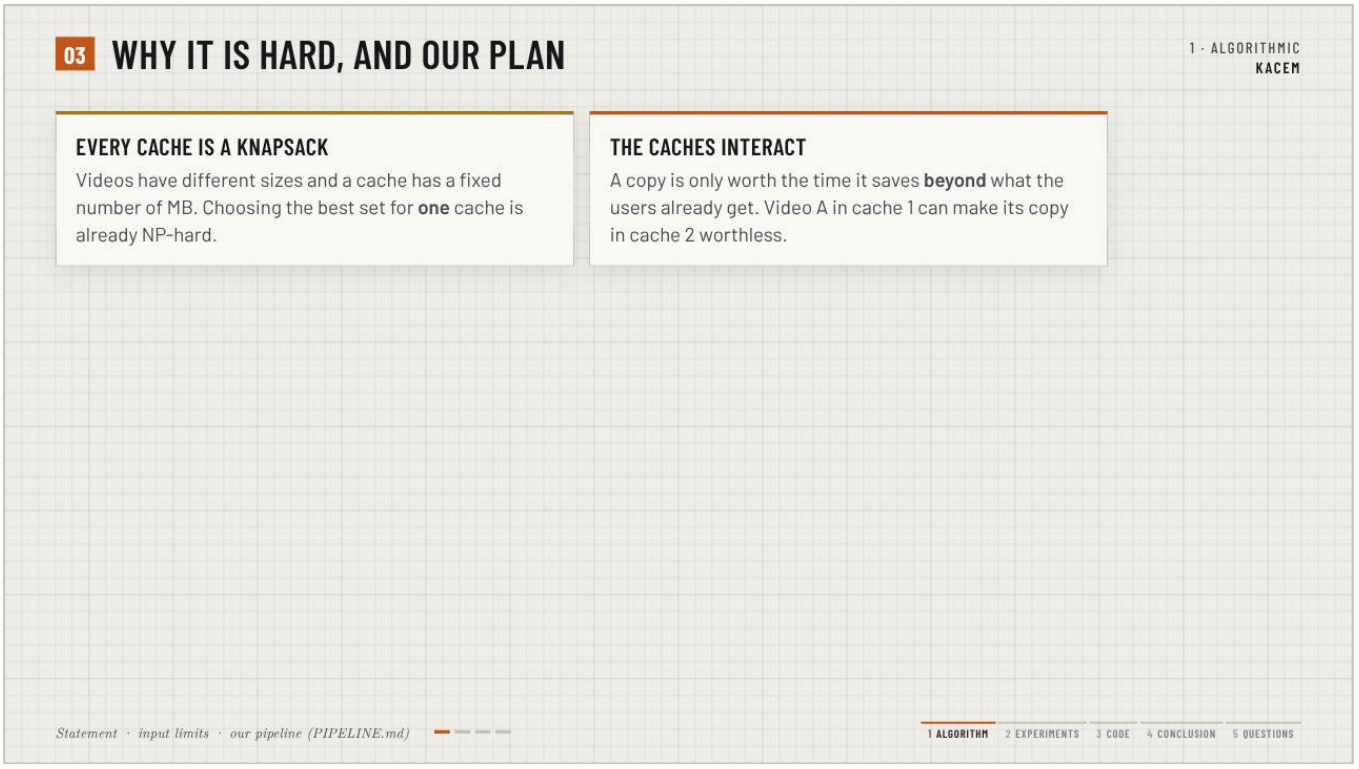
WHAT YOU SEE

"Can we beat 462,500?"

IN PLAIN WORDS

> Can we do better? Kacem will show how our algorithm works.

HANDOVER to Kacem.



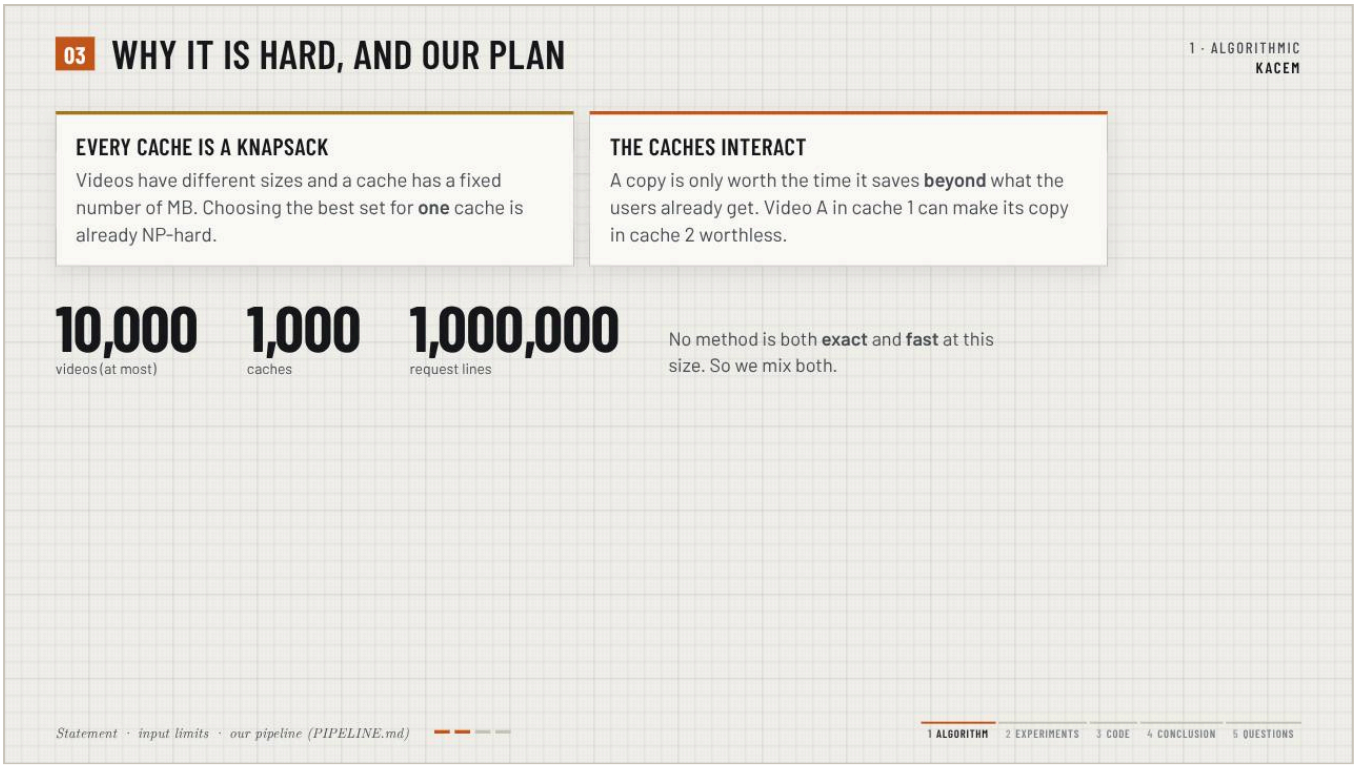
WHAT YOU SEE

Two reasons it is hard.

IN PLAIN WORDS

> Two things make this hard. One: every cache is a knapsack, videos have sizes and the cache has a limit. Two: the caches interact. A copy is only worth the time it saves beyond what the users already get.

Word to know · Knapsack Choose items with sizes and values to fit a bag and get the highest total value.

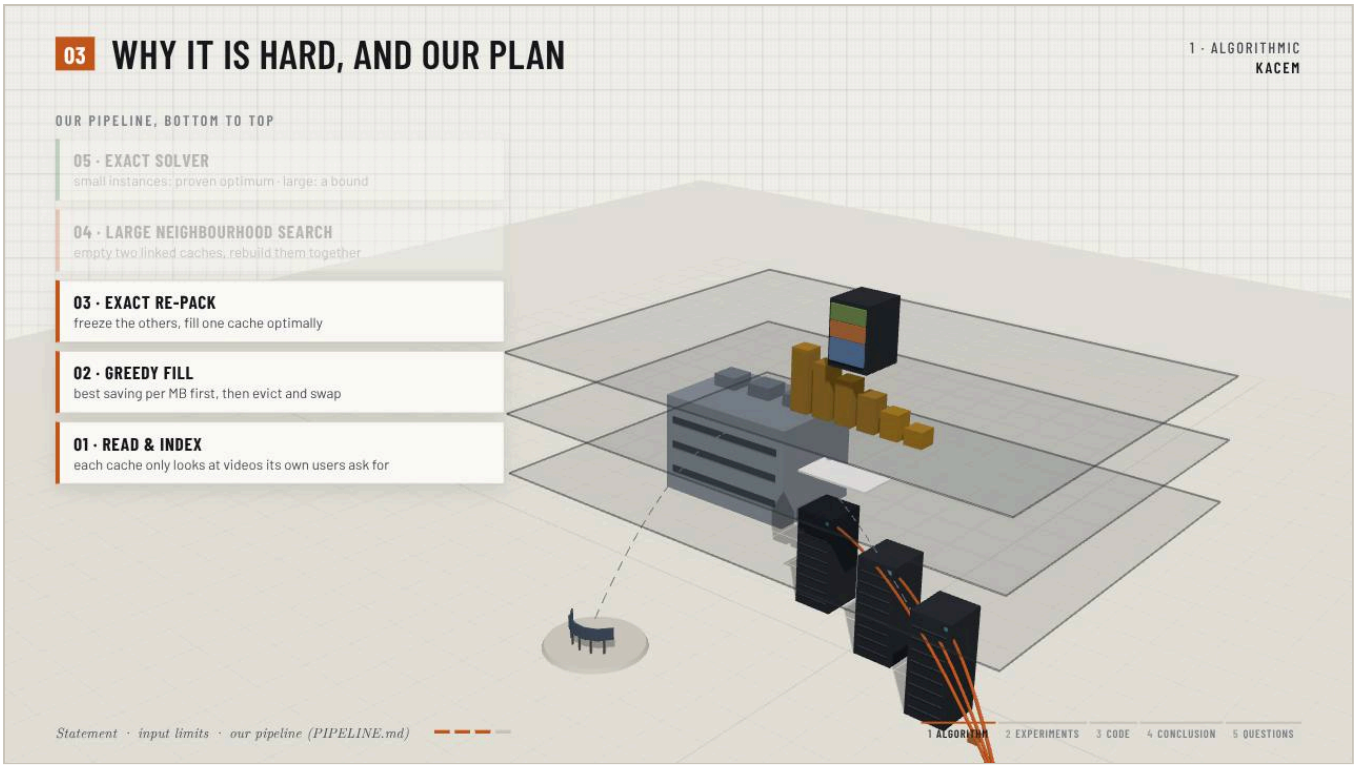


WHAT YOU SEE

The scale from the statement.

IN PLAIN WORDS

> And the data sets go up to ten thousand videos, a thousand caches, a million request lines. Nothing is both exact and fast at this size.

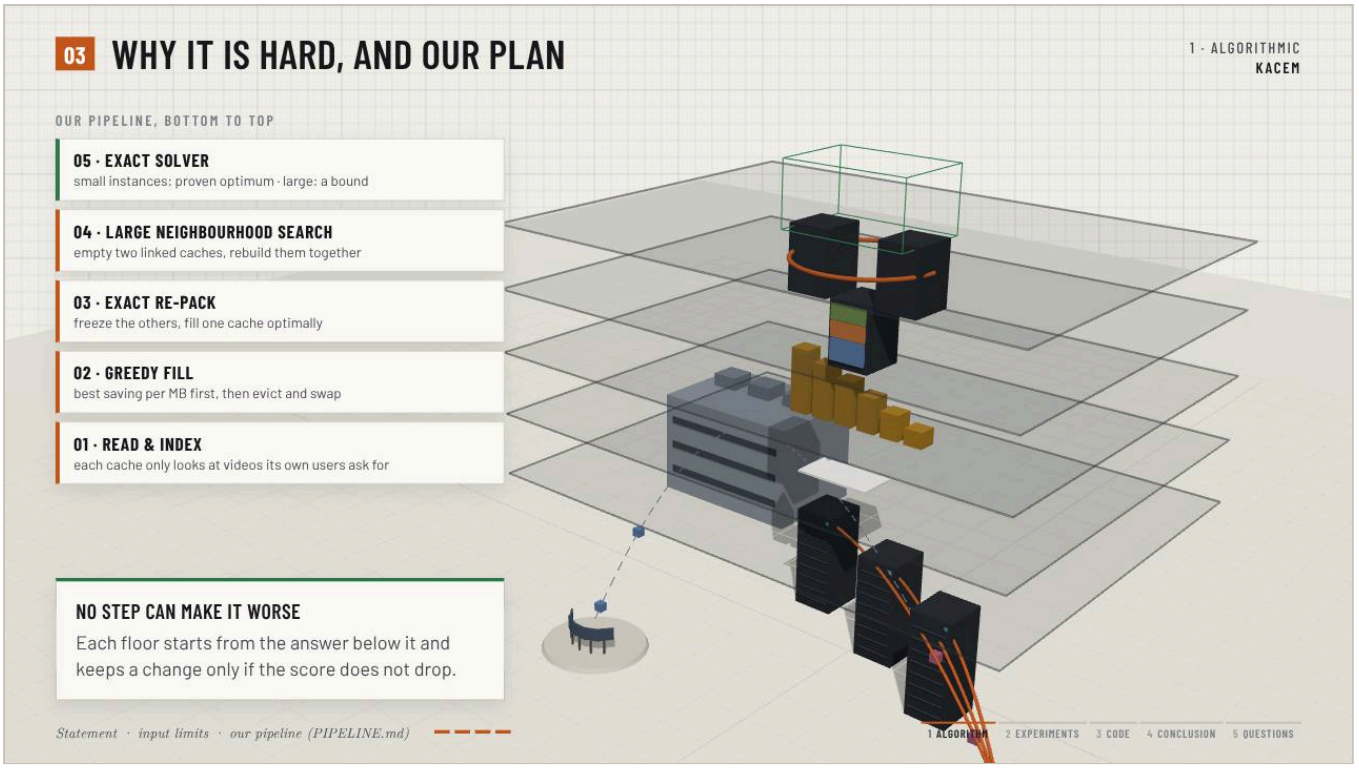


WHAT YOU SEE

The pipeline appears: floors 1 to 3 rise over the 3D network.

IN PLAIN WORDS

> So we stack methods. Read and index the file. Build a fast answer with a greedy. Then improve each cache exactly.



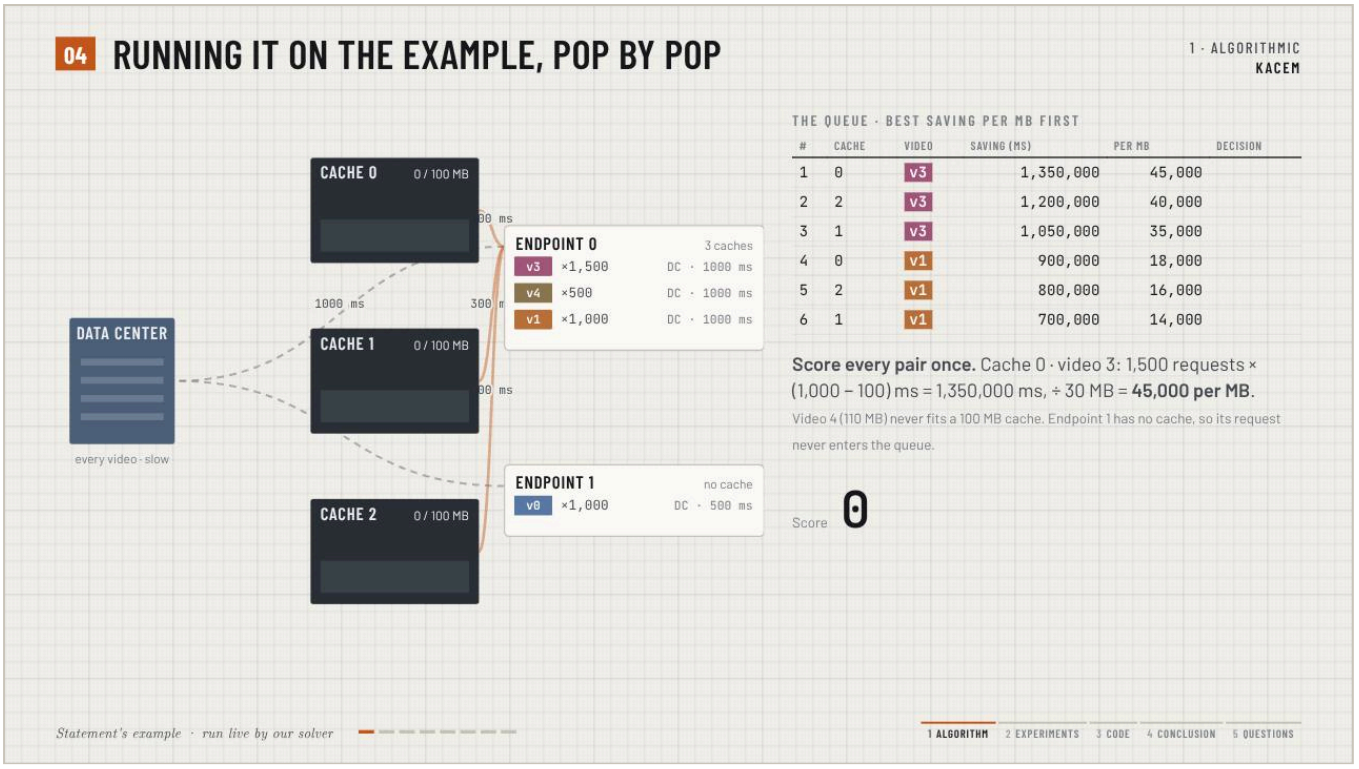
WHAT YOU SEE

Floors 4 and 5, and the rule that ties them together.

IN PLAIN WORDS

> Then change two caches together, and for small data sets solve the whole problem exactly.

> Each floor starts from the answer below it and only keeps a change if the score does not drop.



WHAT YOU SEE

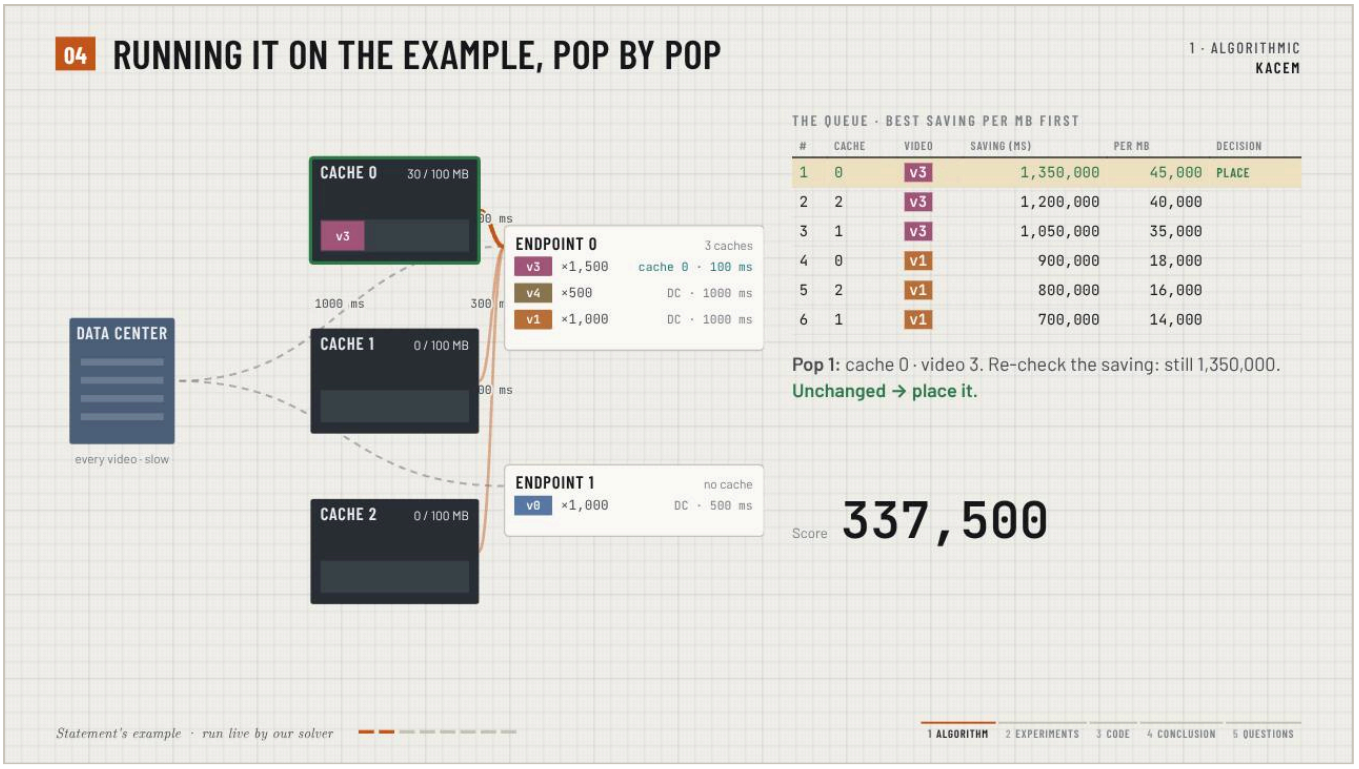
The queue: every (cache, video) pair with its saving and saving per MB.

IN PLAIN WORDS

> Here is the greedy, step by step, on the statement's example. First we score every pair: requests times the time saved. For cache 0 and video 3, that is 1,500 times 900 milliseconds, divided by its 30 MB: 45,000 per MB.

> Video 4 is 110 MB, it never fits. Endpoint 1 has no cache, so it never enters the queue.

Word to know · Lazy heap We only re-check the top item. Values only go down, so the top is safe if unchanged.

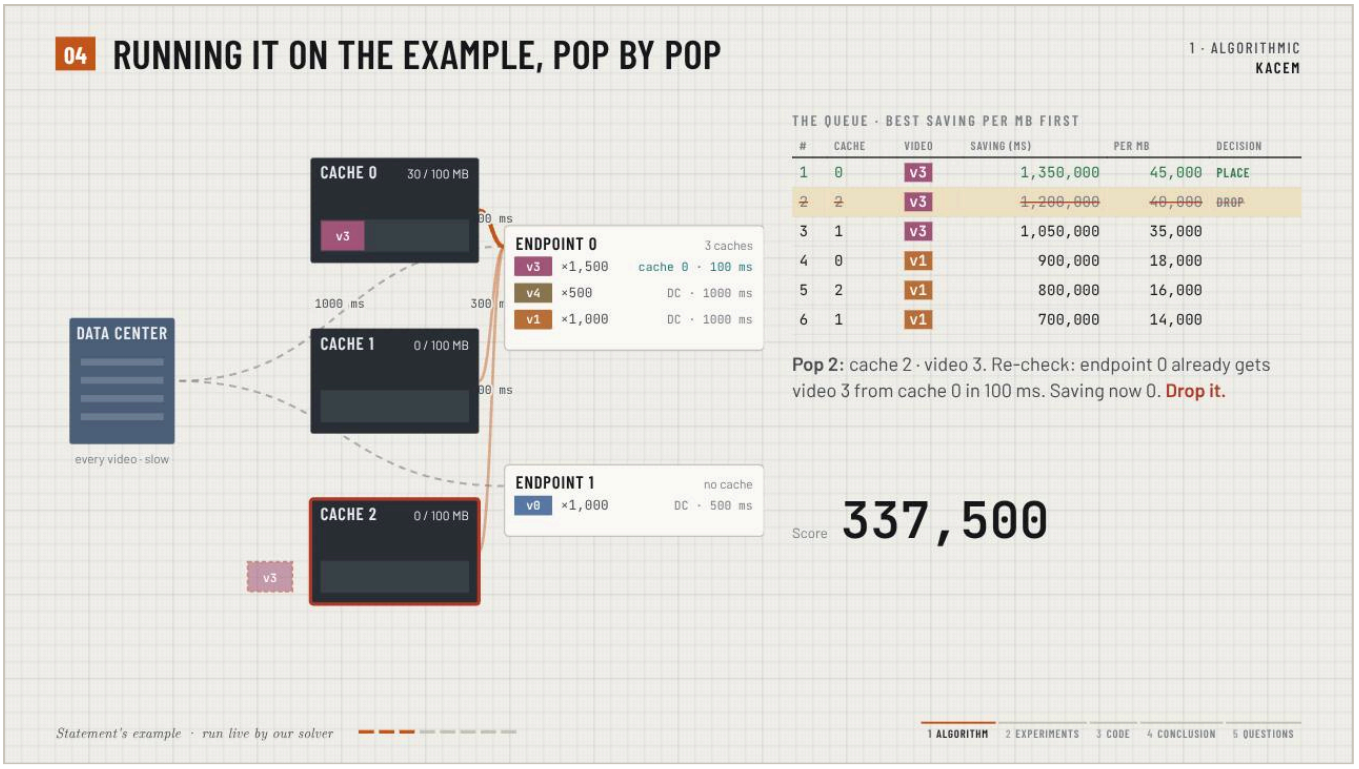


WHAT YOU SEE

Pop 1: cache 0 · video 3 → place.

IN PLAIN WORDS

> We always take the best pair, re-check its saving, and if it has not changed we place it. Video 3 goes into cache 0.

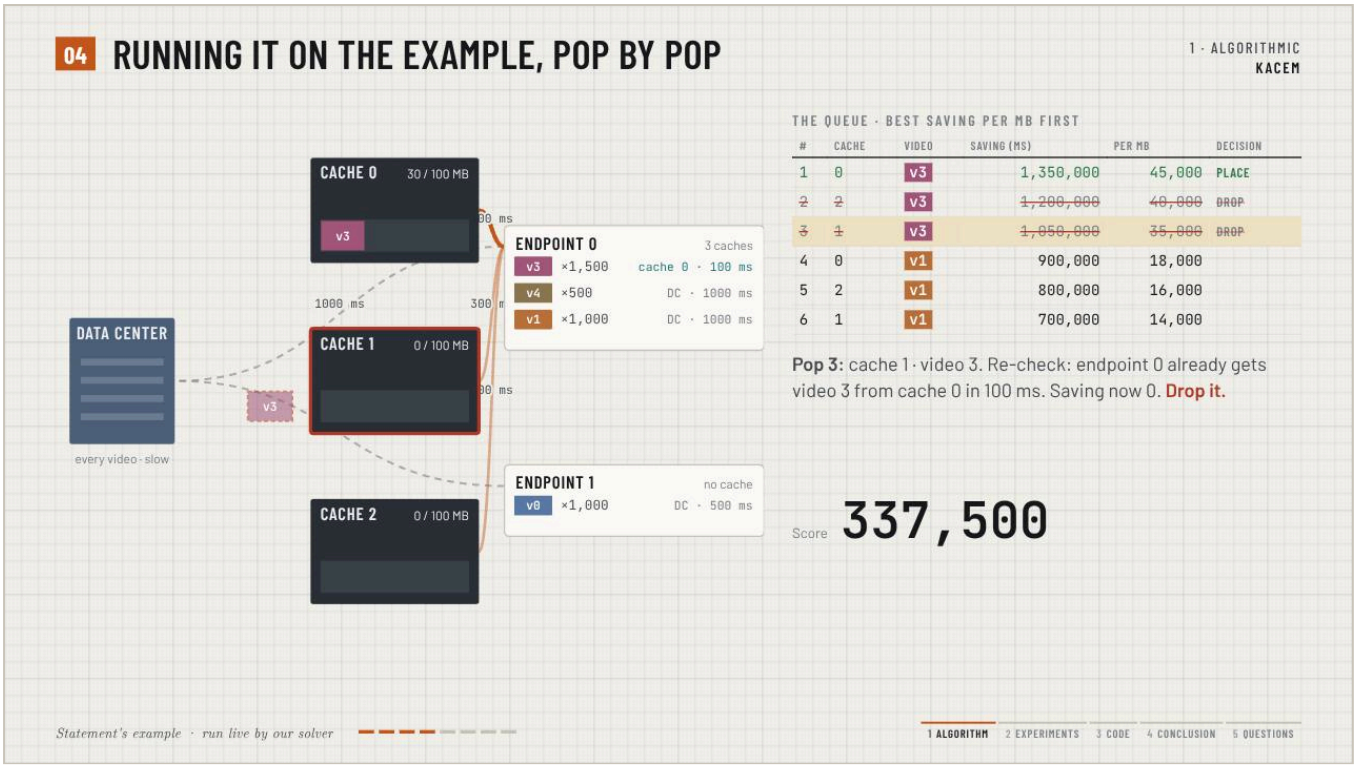


WHAT YOU SEE

Pop 2: cache 2 · video 3 → drop (the key moment).

IN PLAIN WORDS

> Next is video 3 in cache 2. It looked worth 40,000 per MB. But endpoint 0 now gets video 3 from cache 0 in 100 ms. Re-checked, it saves nothing. We drop it. This re-check is what stops us storing useless copies.

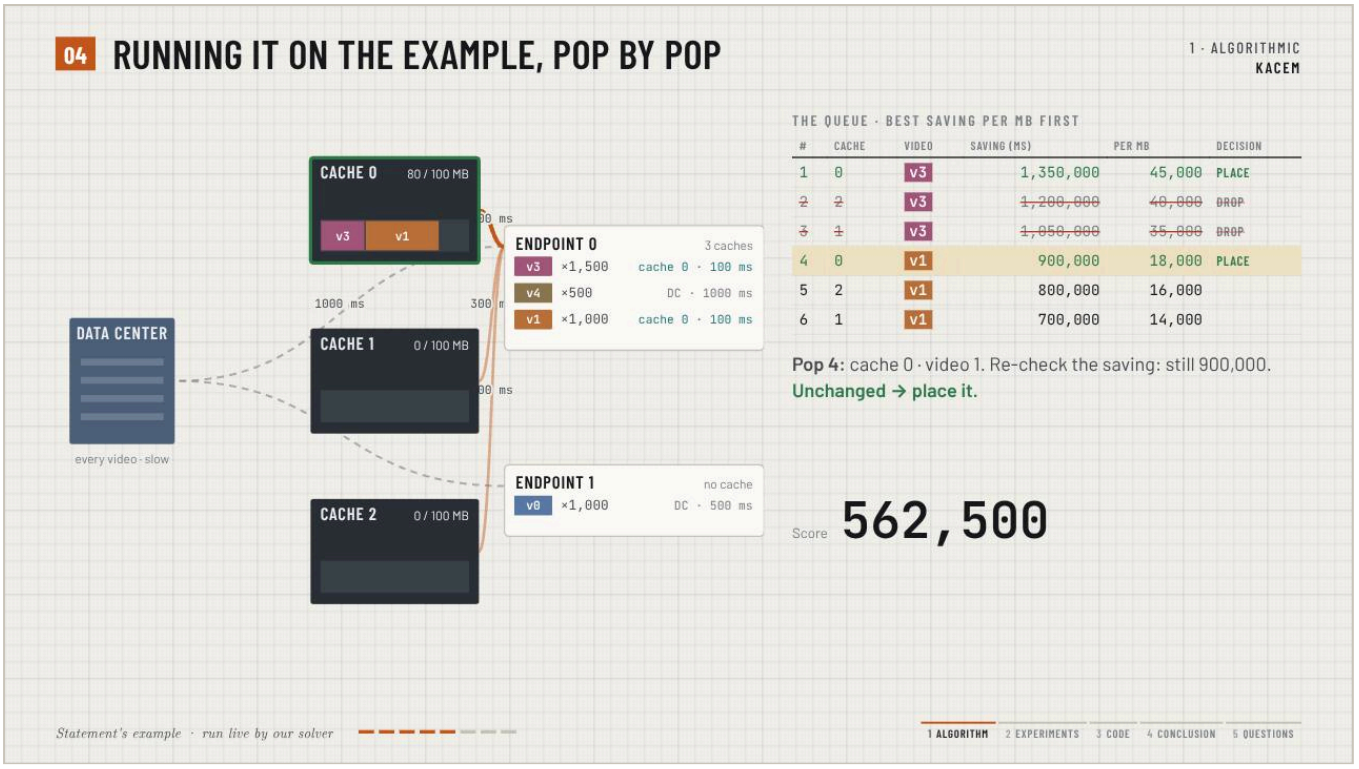


WHAT YOU SEE

Pop 3: cache 1 · video 3 → drop.

IN PLAIN WORDS

> Same for cache 1.

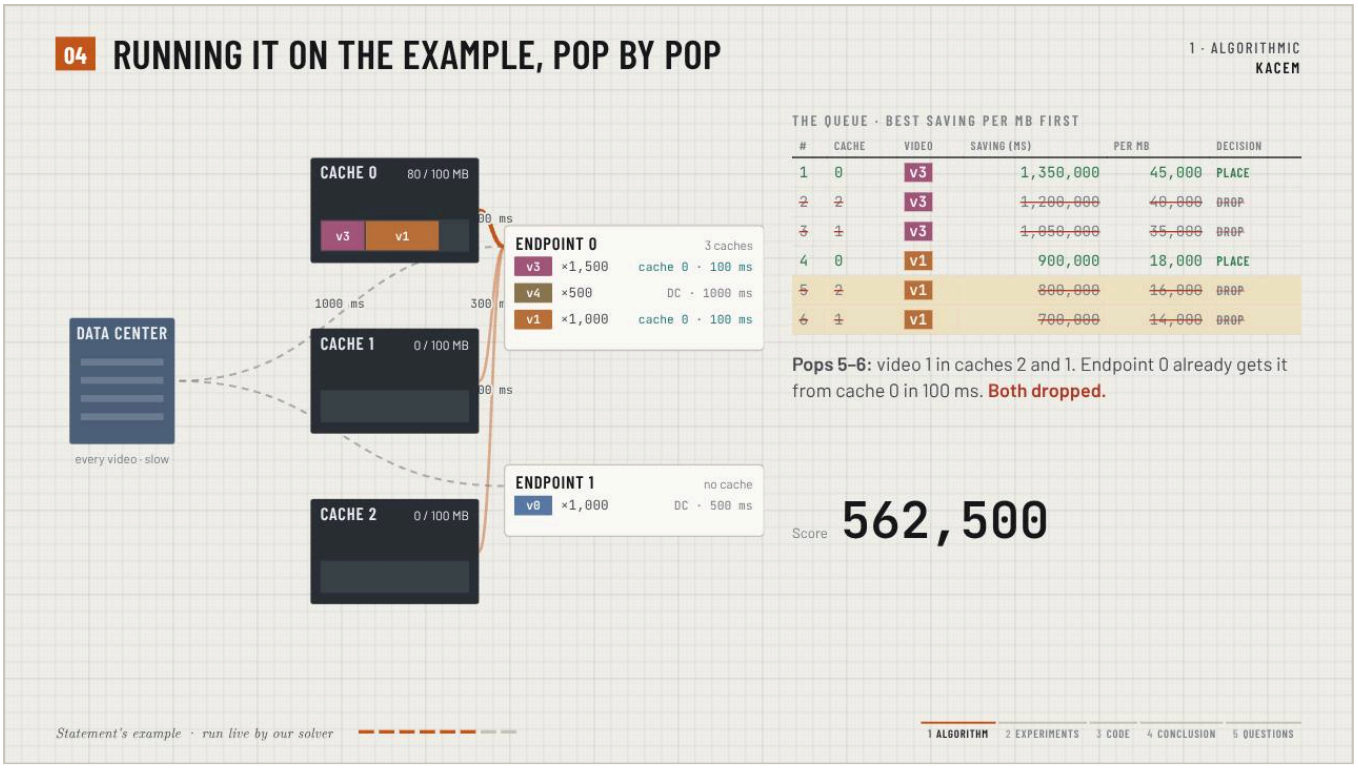


WHAT YOU SEE

Pop 4: cache 0 · video 1 → place. Score 562,500.

IN PLAIN WORDS

> Video 1 into cache 0: unchanged, placed. Cache 0 is now 80 out of 100 MB.

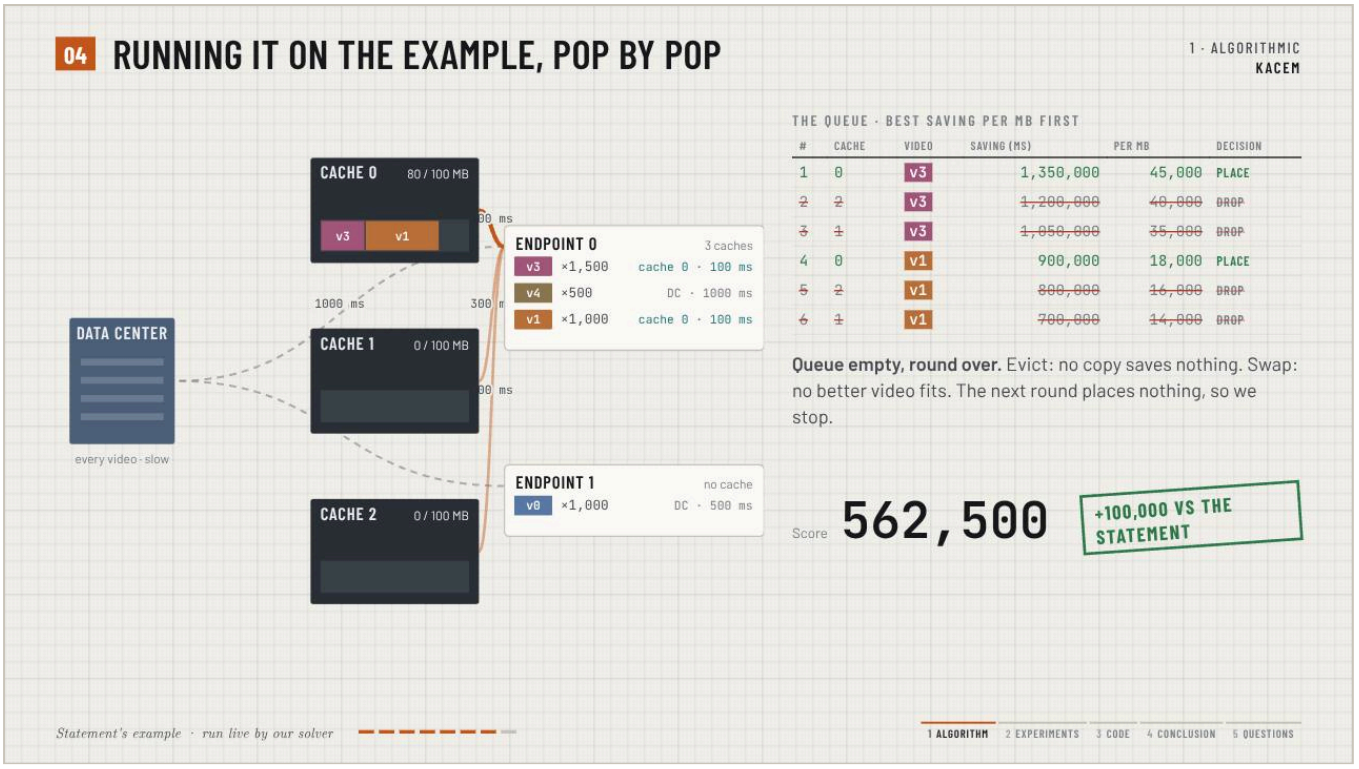


WHAT YOU SEE

Pops 5 and 6 dropped.

IN PLAIN WORDS

> Video 1 in the other caches: already served by cache 0. Dropped.

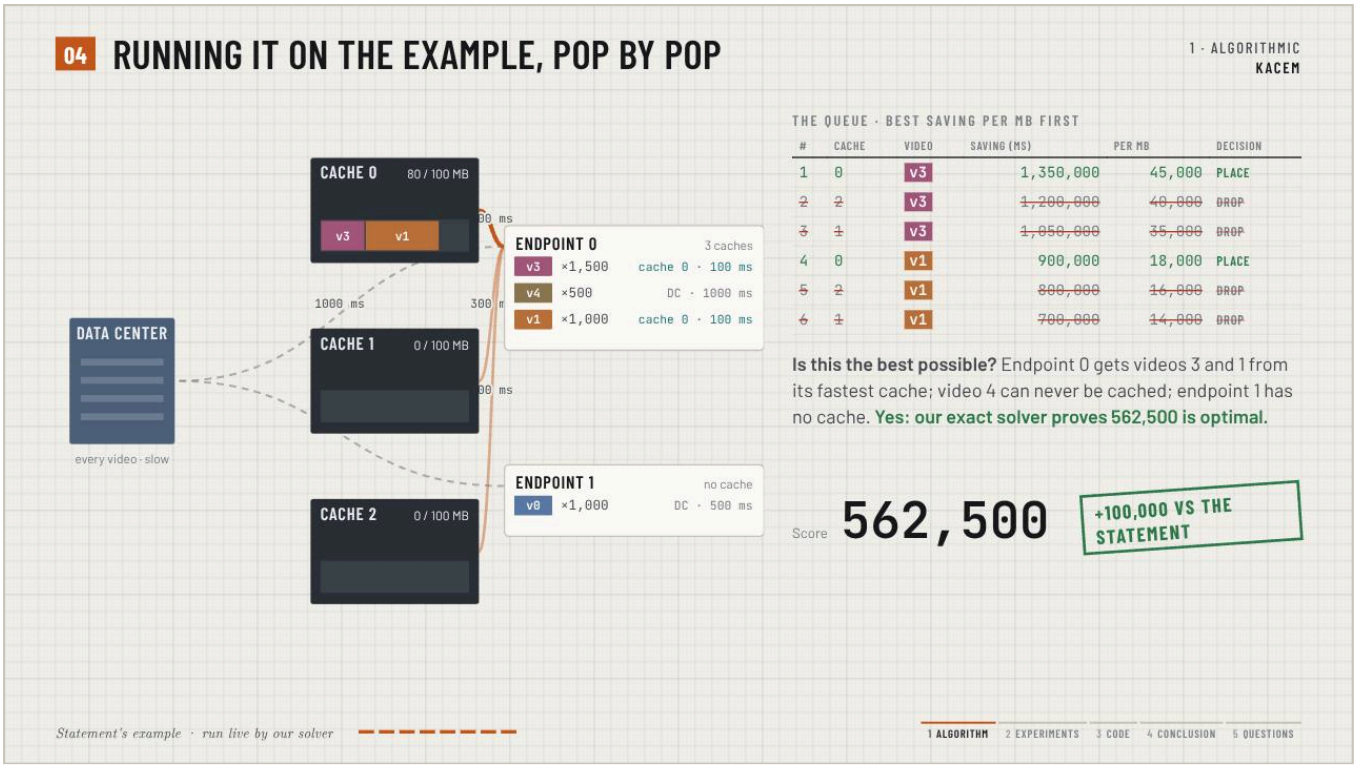


WHAT YOU SEE

Round over: evict and swap find nothing. +100,000 vs the statement.

IN PLAIN WORDS

> After the round we evict useless copies and try swaps. Nothing to do here. We stop with 562,500: 100,000 more than the statement's answer.

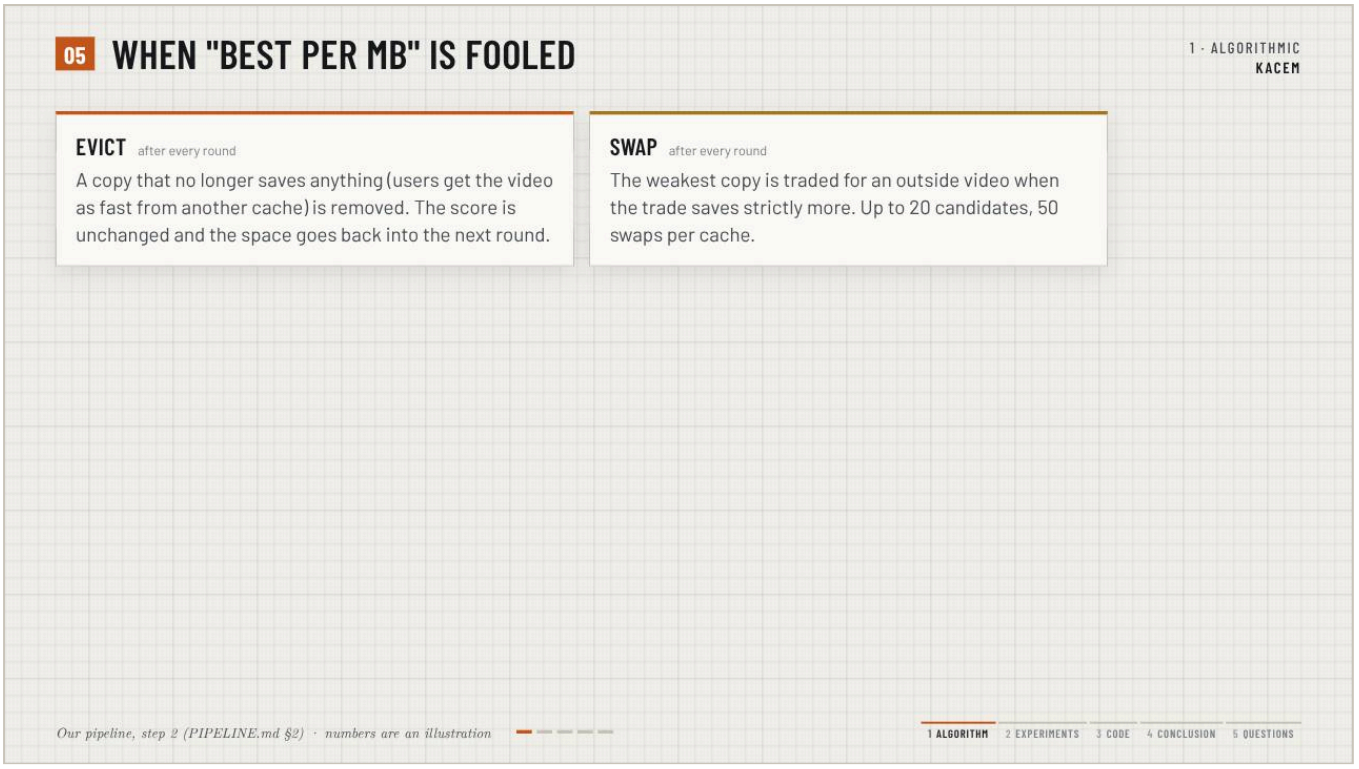


WHAT YOU SEE

Is it optimal? Yes, and the exact solver agrees.

IN PLAIN WORDS

> And here it is the best possible: endpoint 0 gets both videos from its fastest cache, and nothing else can be improved. Our exact solver proves it.



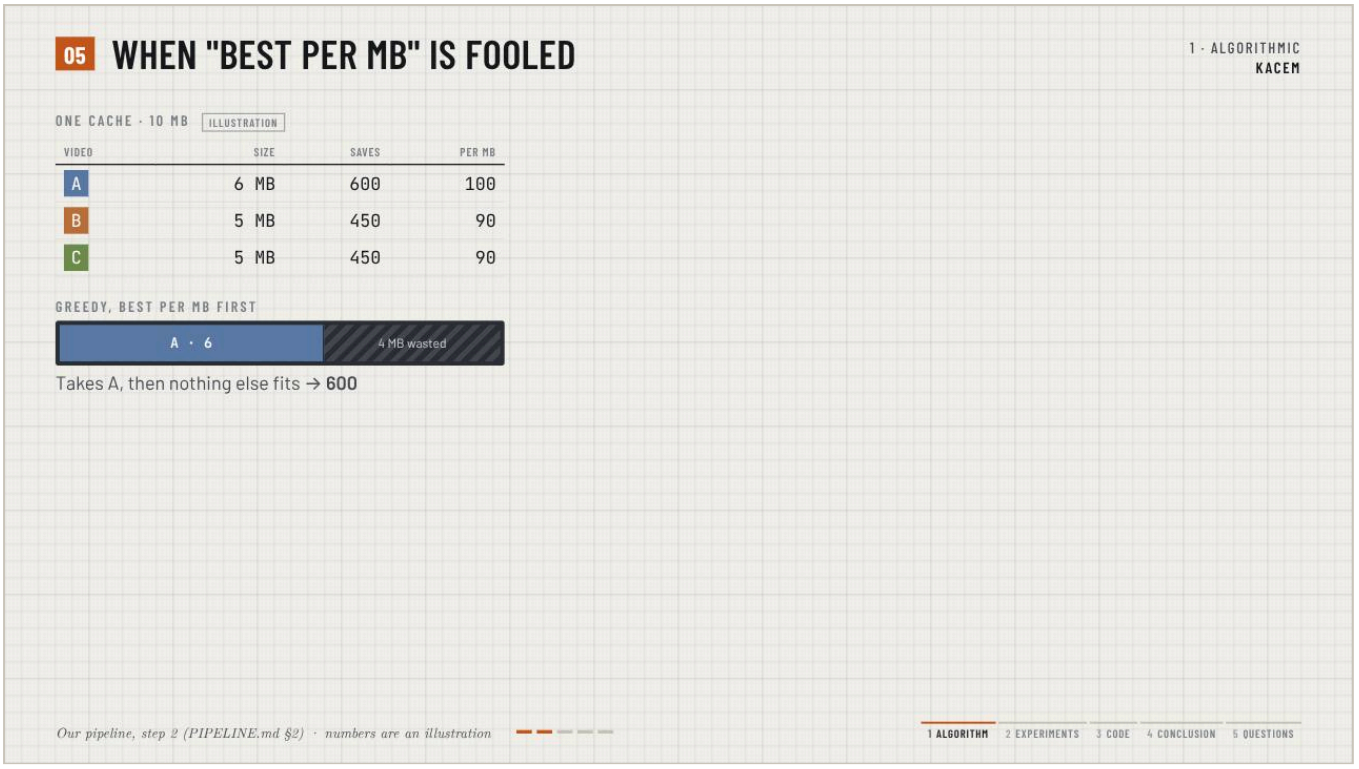
WHAT YOU SEE

Evict and swap, after every round.

IN PLAIN WORDS

> Two clean-ups run after every round. Evict removes copies that no longer save anything. Swap trades the weakest copy for a better video.

Word to know · Dynamic programming (DP) Solve a big problem by filling a table of small answers. Gives the exact knapsack answer.

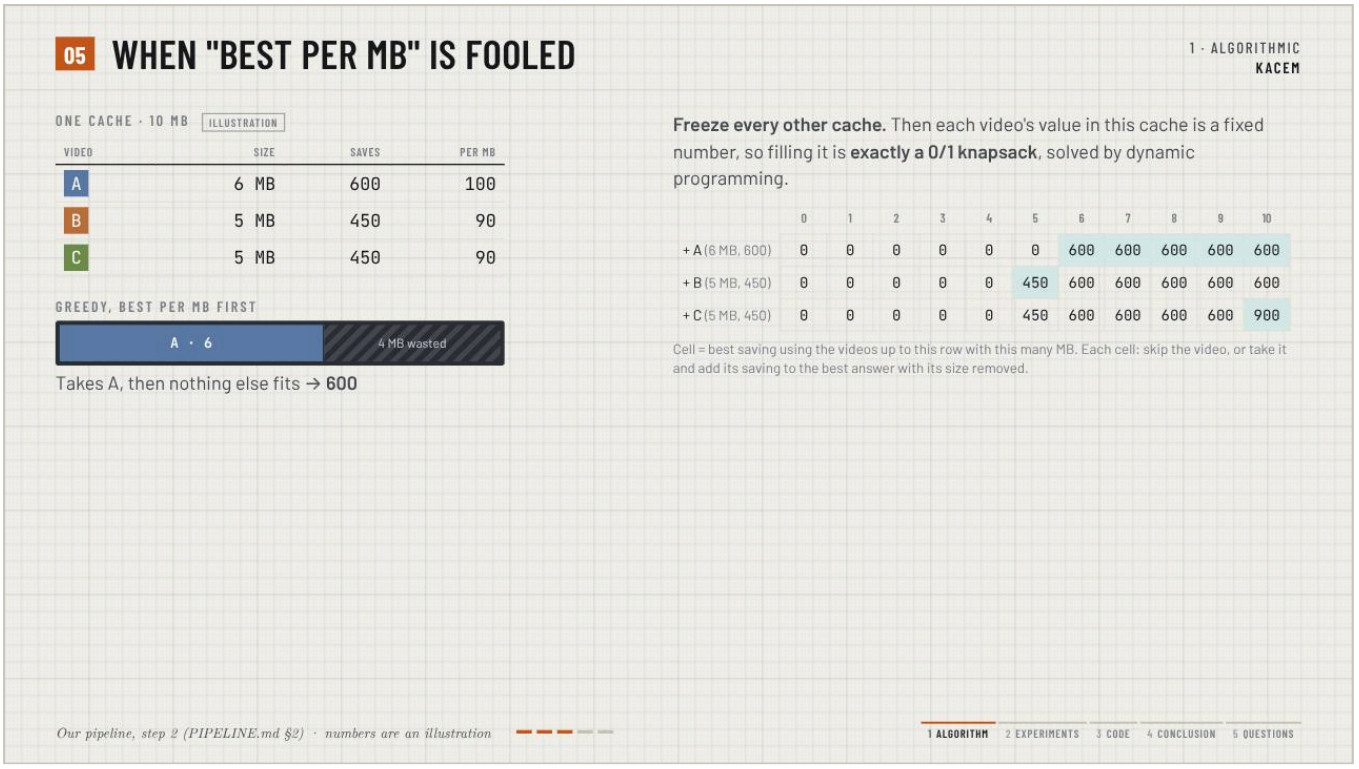


WHAT YOU SEE

The ratio trap: one cache of 10 MB.

IN PLAIN WORDS

> But choosing by saving per MB can be fooled. One cache of 10 MB. Video A has the best ratio, so the greedy takes it. Then nothing else fits: 600, and 4 MB wasted.

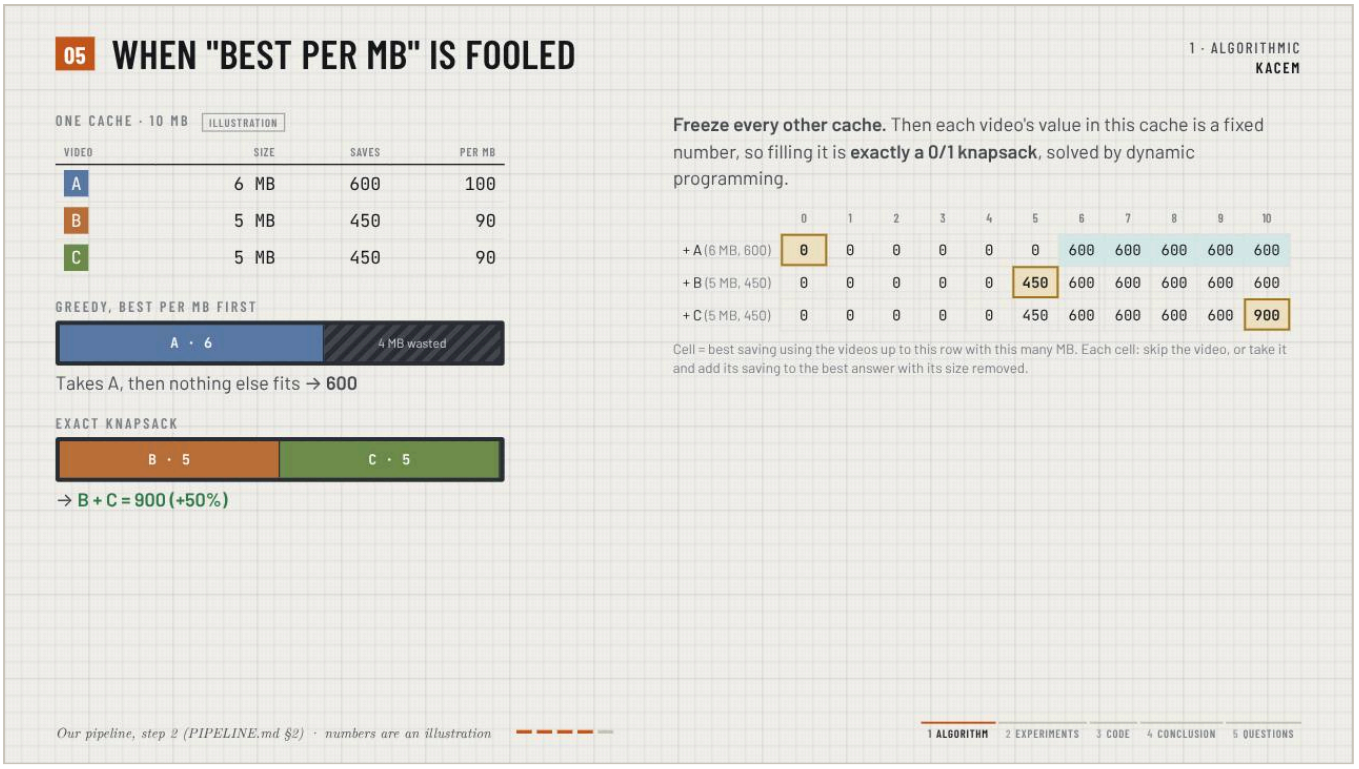


WHAT YOU SEE

The DP table fills.

IN PLAIN WORDS

> The fix: freeze all the other caches. Then each video is worth a fixed amount in this cache, and filling it is exactly a knapsack. We solve it optimally with dynamic programming.

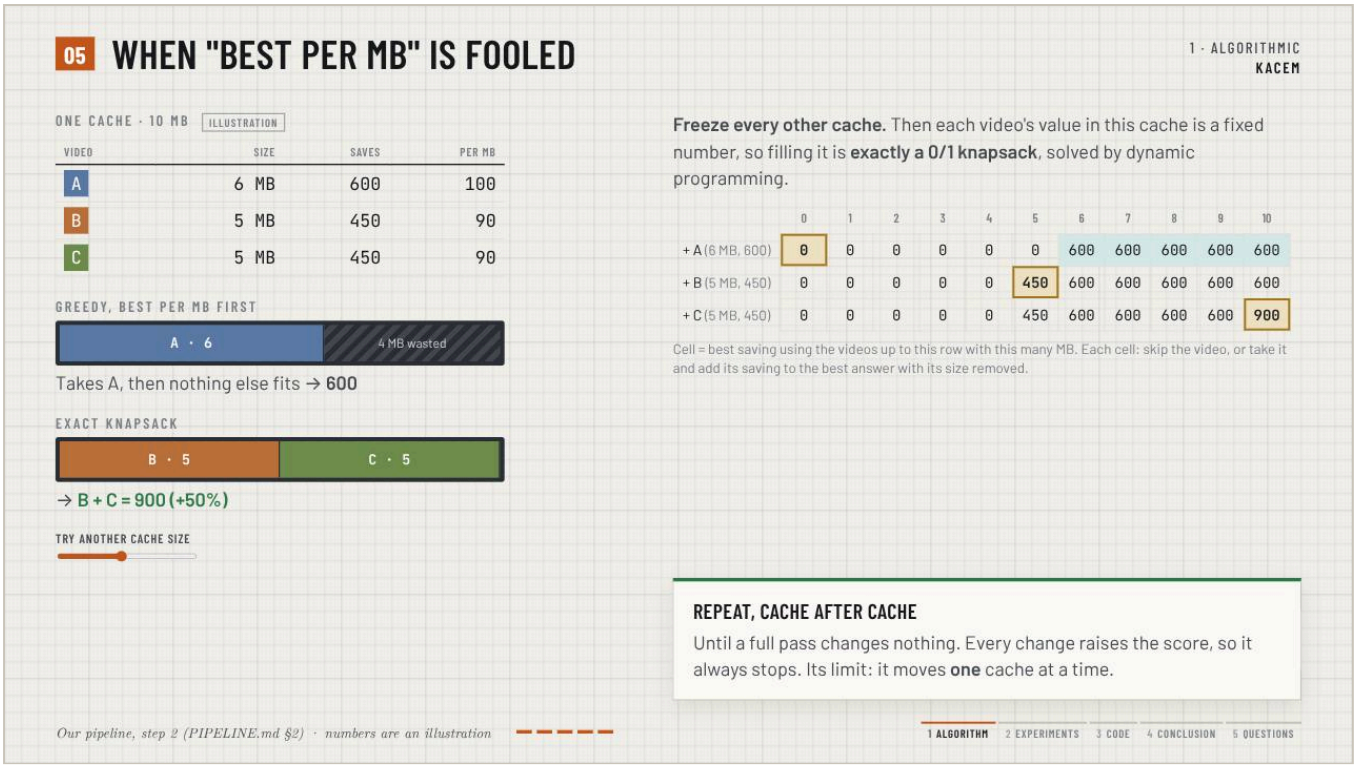


WHAT YOU SEE

The path back through the table: B + C = 900.

IN PLAIN WORDS

> The table says: take B and C. 900 instead of 600.



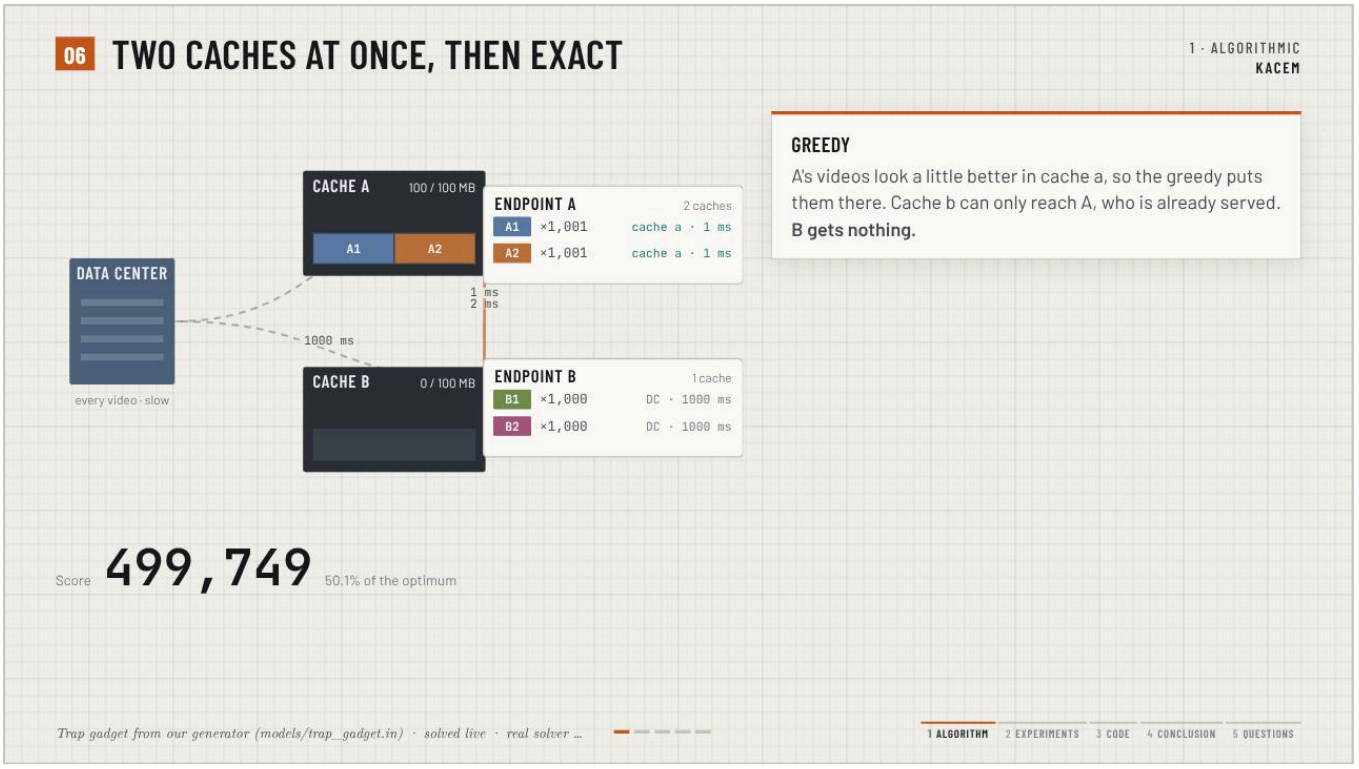
WHAT YOU SEE

Repeat cache after cache. The slider lets you try other sizes.

IN PLAIN WORDS

> We repeat this for every cache until nothing changes. It can only go up, so it always stops.

If asked: move the slider. At 11 MB both methods agree.



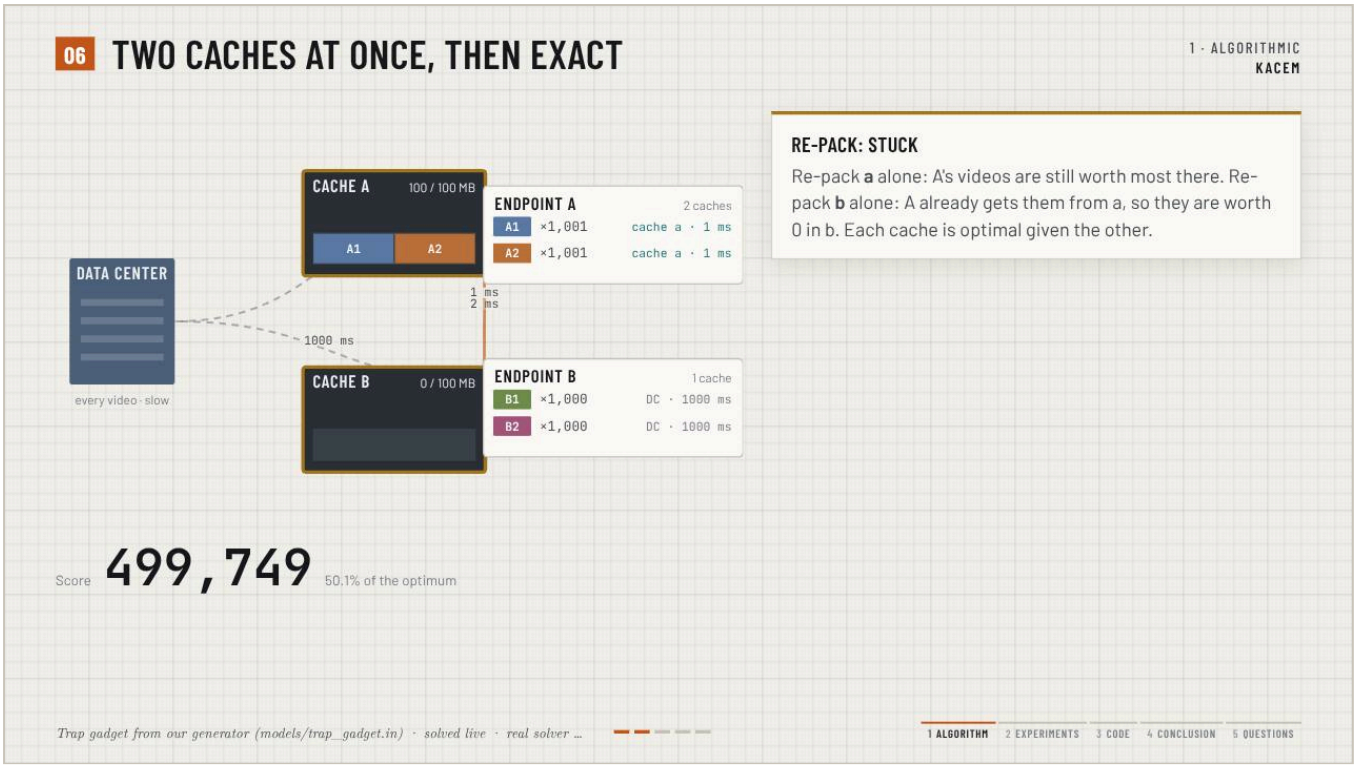
WHAT YOU SEE

The trap instance after the greedy: B gets nothing, ~50% of the optimum.

IN PLAIN WORDS

> But re-pack changes one cache at a time. Here is a trap from our generator. Endpoint A's videos look slightly better in cache a, so the greedy puts them there. Endpoint B gets nothing: half the optimum.

Word to know · LNS Large neighbourhood search: empty a few related caches, rebuild them, keep it if not worse.

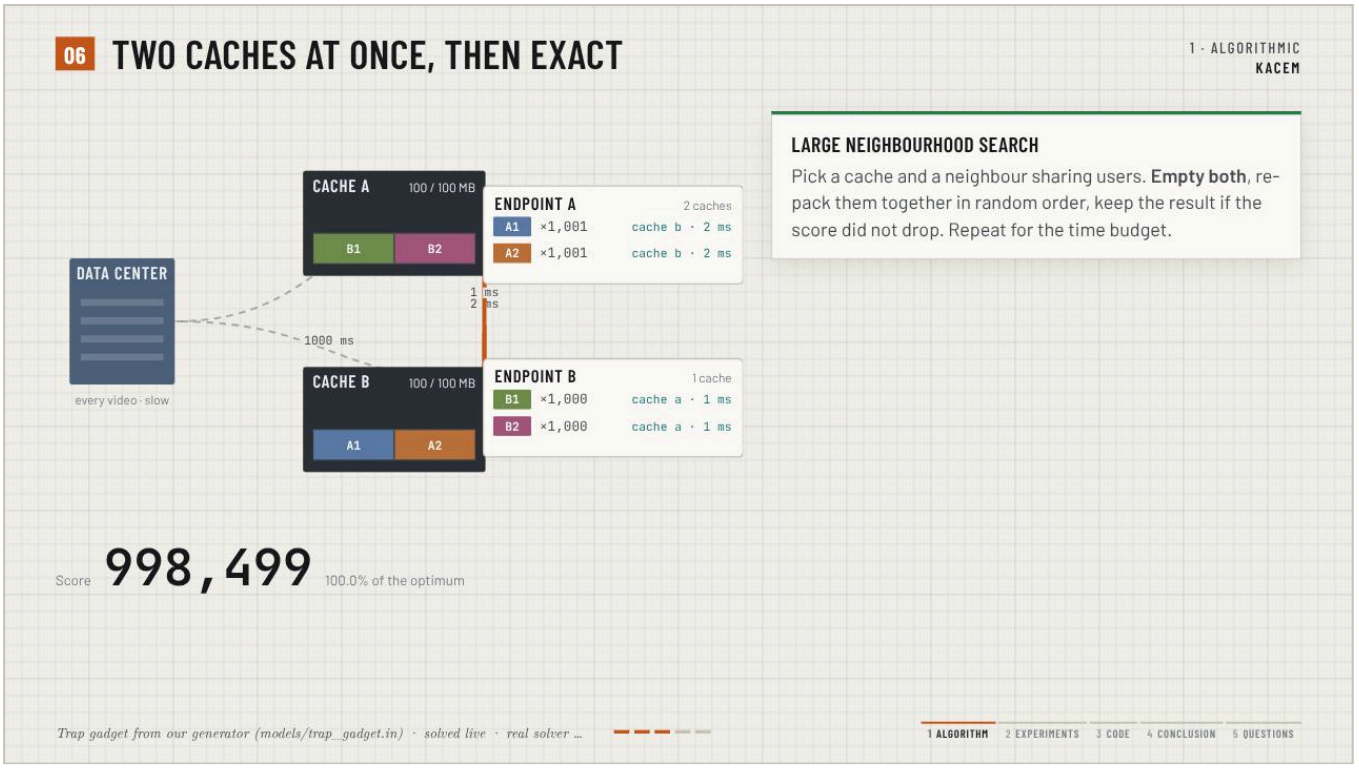


WHAT YOU SEE

Re-pack is stuck.

IN PLAIN WORDS

> Re-pack cache a alone: A's videos are still the best there. Re-pack cache b alone: A already gets them from a, so they are worth zero in b. Each cache is optimal given the other. Stuck.

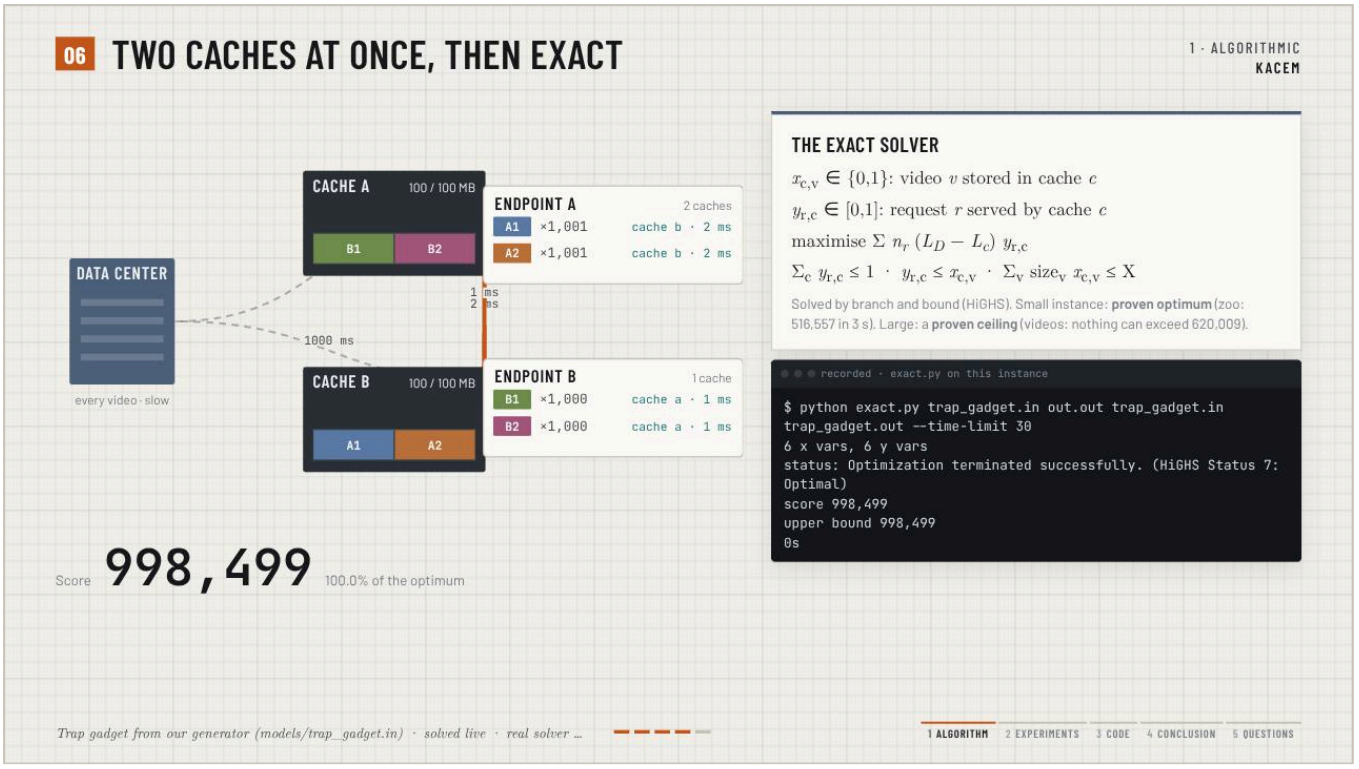


WHAT YOU SEE

LNS: both caches emptied and rebuilt together. Score jumps to 998,499.

IN PLAIN WORDS

> Large neighbourhood search empties two caches that share users and rebuilds them together. Now A goes to b and B goes to a. We keep a change only if the score does not drop.

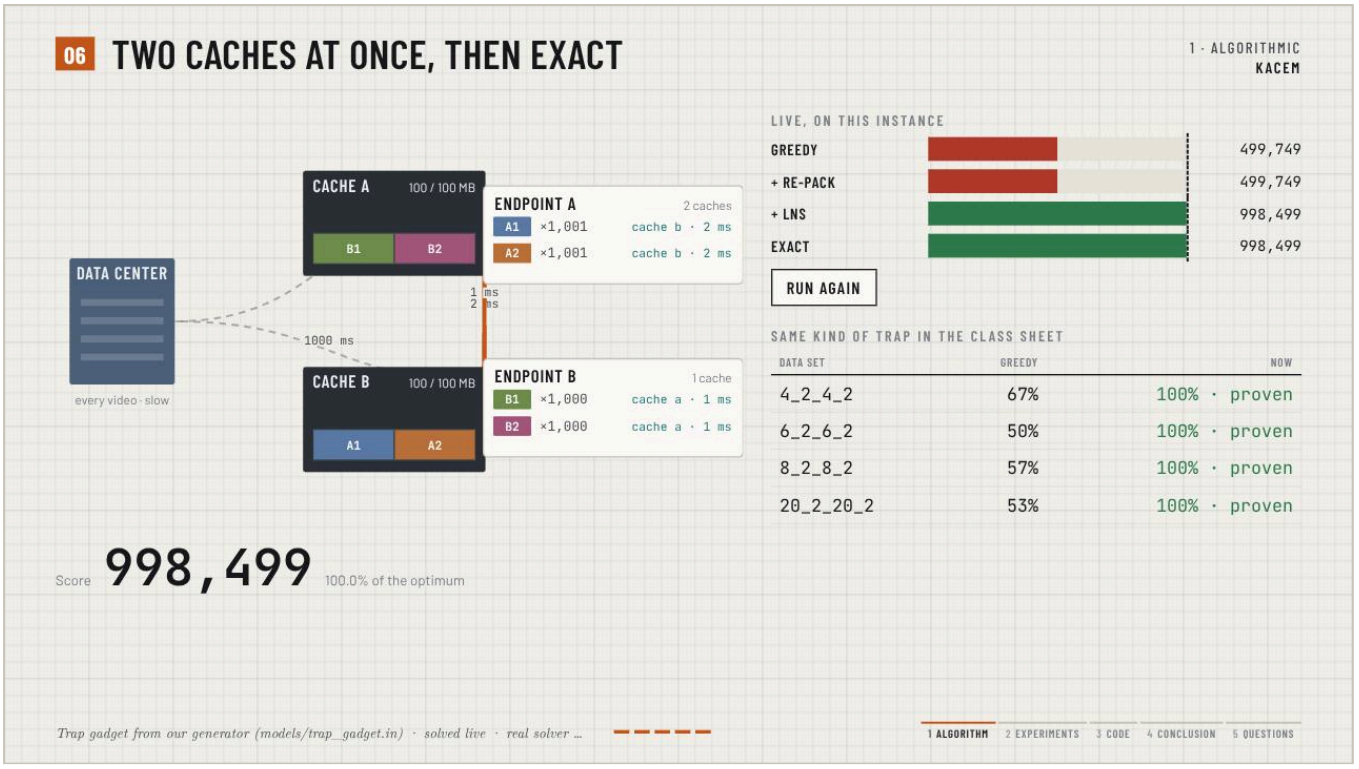


WHAT YOU SEE

The exact solver: the integer program and the real output on this instance.

IN PLAIN WORDS

> For small data sets we also write the whole problem as an integer program and solve it exactly. It proves 998,499 is optimal here, and the zoo data set in 3 seconds. On big ones it still gives a ceiling: on videos, no answer can pass 620,009.



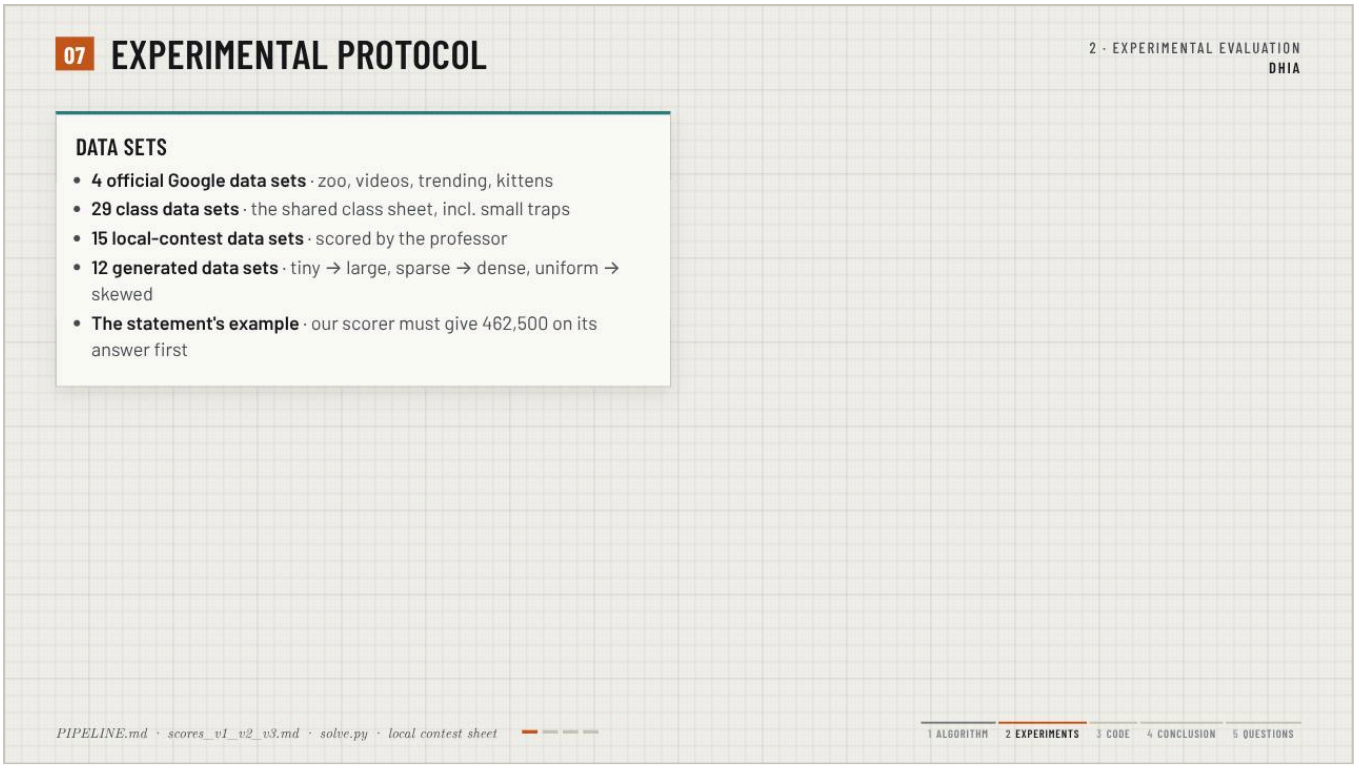
WHAT YOU SEE

The ladder, computed live. The class trap data sets went from 50–67% to 100%.

IN PLAIN WORDS

> Live, on this instance: greedy, re-pack, LNS, exact. And the same kind of trap in the class sheet: from about half the optimum to proven optimal.

HANDOVER to Dhia.



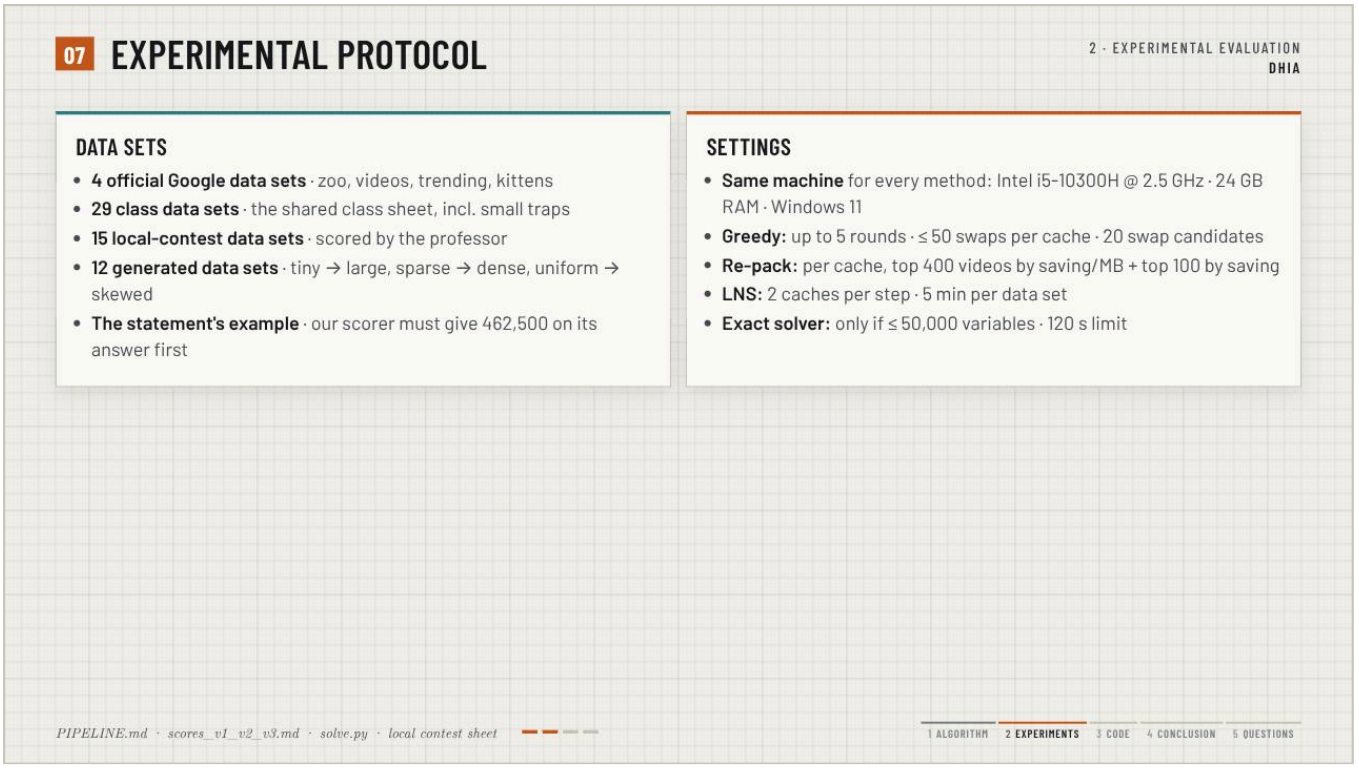
WHAT YOU SEE

Data sets.

IN PLAIN WORDS

> How we measured. Four Google data sets, the 29 class data sets, the 15 of the local contest, and 12 we generated ourselves. Before anything, our scorer had to reproduce the statement's 462,500.

Word to know · VBS / best in class The best score any group reached on a data set.

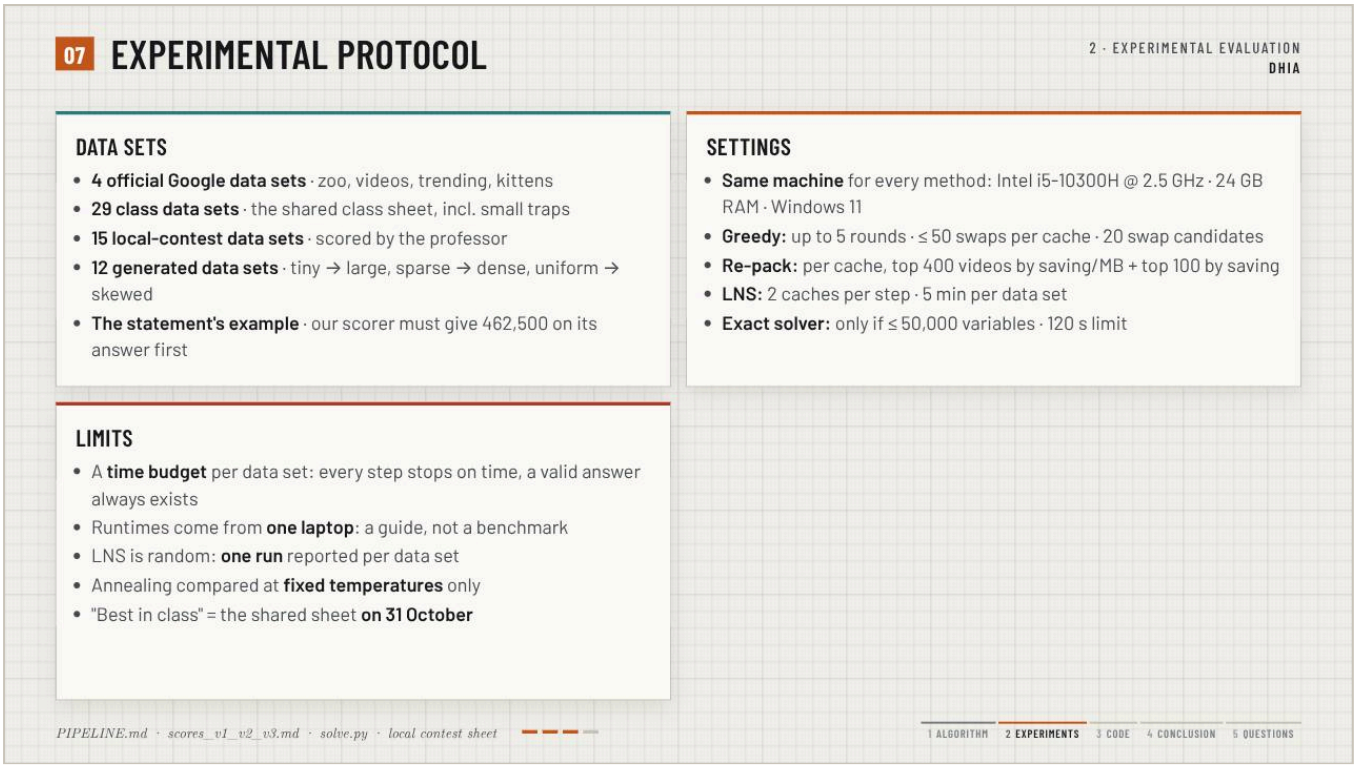


WHAT YOU SEE

Settings.

IN PLAIN WORDS

> Same laptop for every method. Greedy up to five rounds; re-pack on the best 500 candidates per cache; LNS on two caches, five minutes per data set; the exact solver only on small models, 120 seconds.

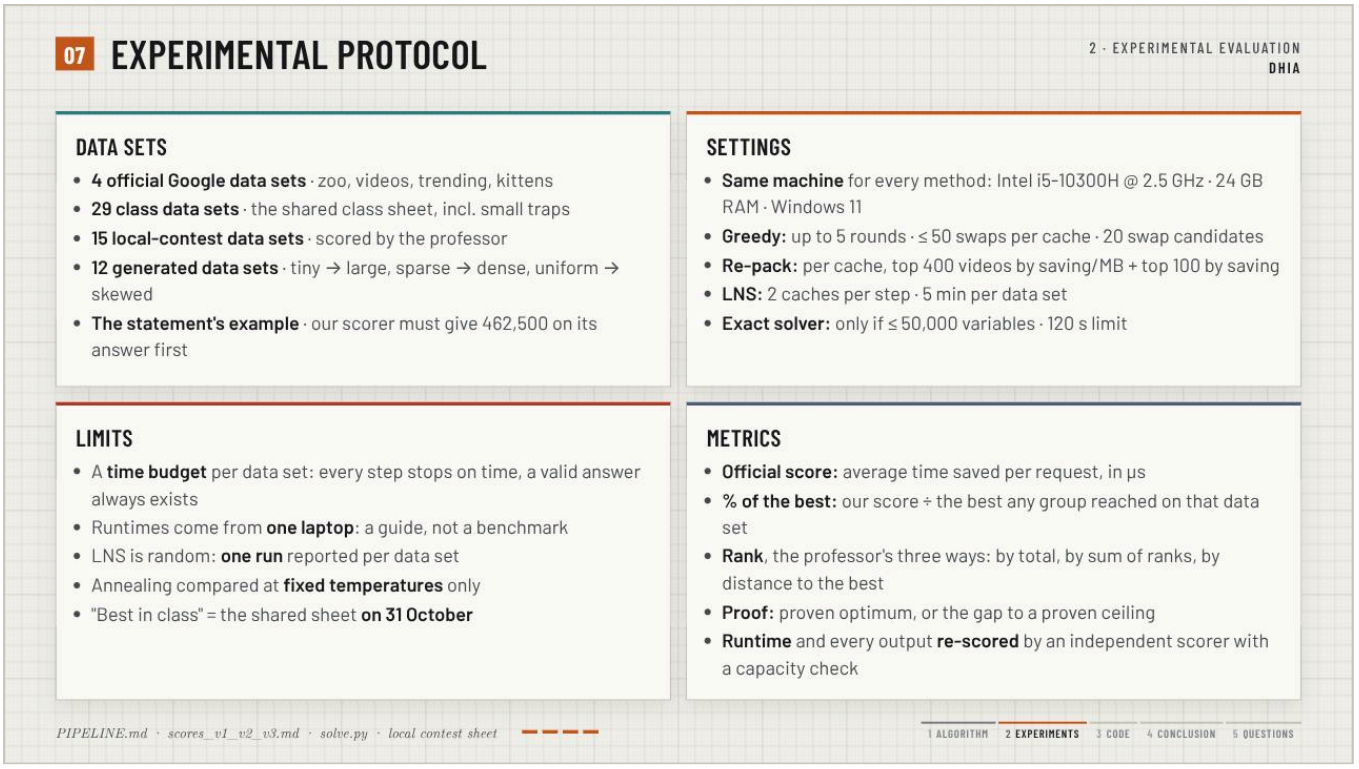


WHAT YOU SEE

Limits.

IN PLAIN WORDS

> The limits: a time budget per data set, runtimes from one laptop only, one LNS run per data set, and the class best is the sheet on 31 October.

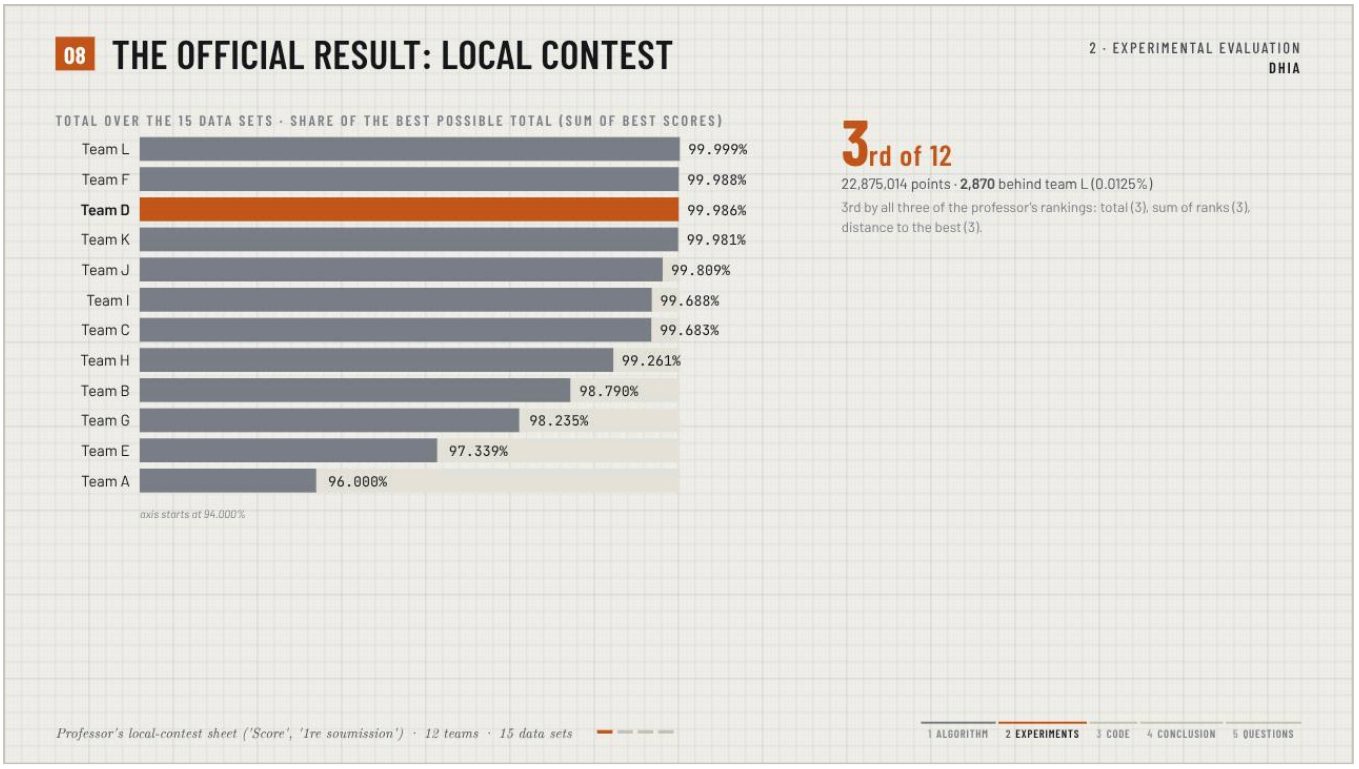


WHAT YOU SEE

Metrics.

IN PLAIN WORDS

> The official score; our score as a share of the best any group reached; the professor's three rankings; and proof: a proven optimum or the gap to a proven ceiling. Every output is re-scored independently.



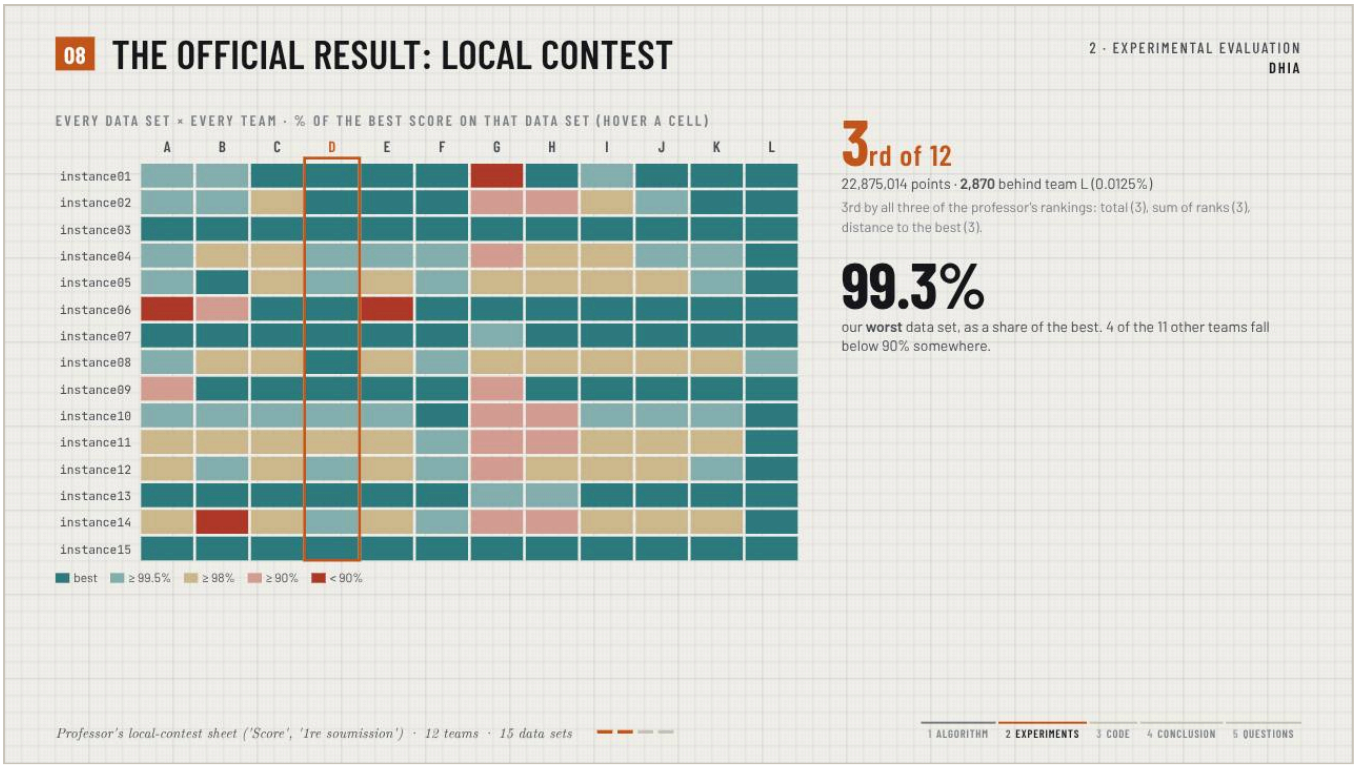
WHAT YOU SEE

Totals as a share of the best possible: L, F, then us.

IN PLAIN WORDS

> The official result: the local contest. We finished third of twelve, by all three rankings. 2,870 points behind first, out of almost 23 million.

Word to know · VBS / best in class The best score any group reached on a data set.

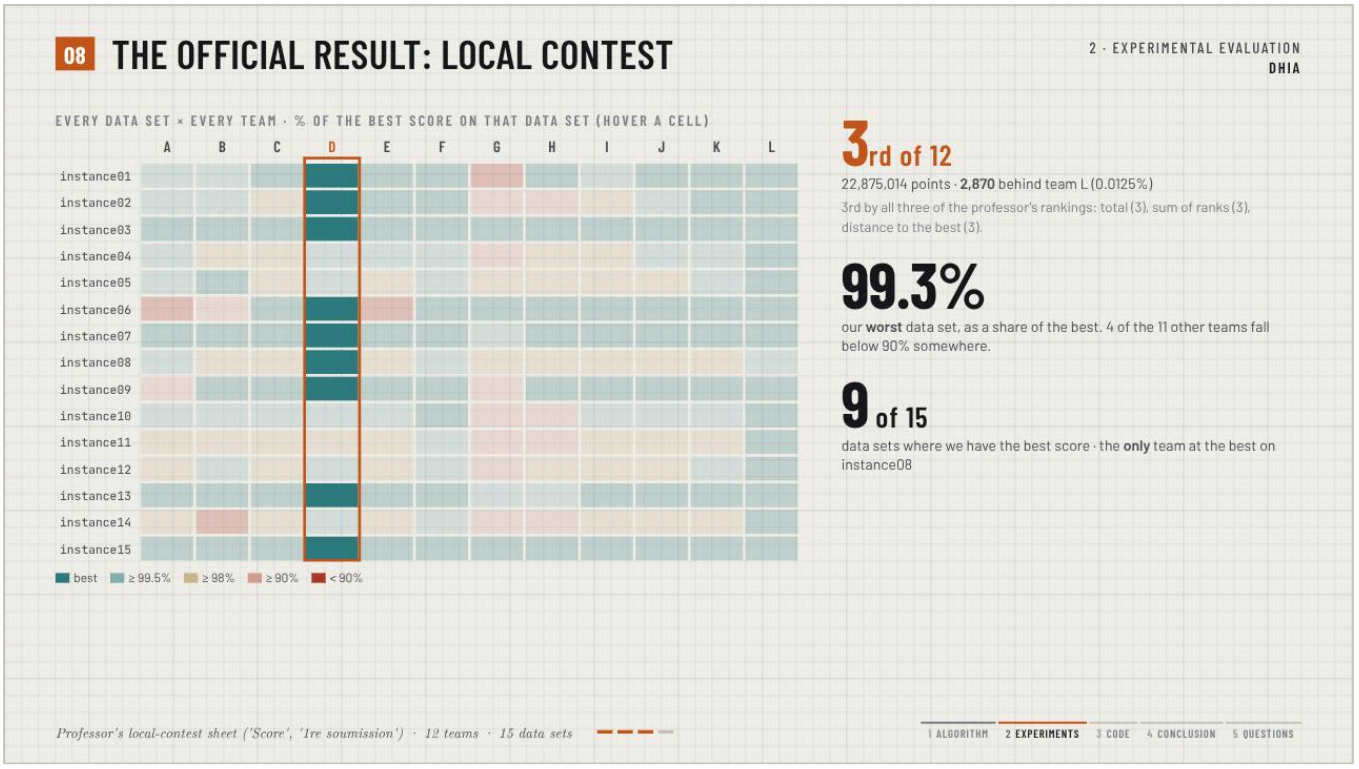


WHAT YOU SEE

Heat map: data set × team. Our column is framed.

IN PLAIN WORDS

> Every cell is one team on one data set, as a share of the best. Our worst data set is still above 99.3% of the best. Most teams fall below 90% somewhere.

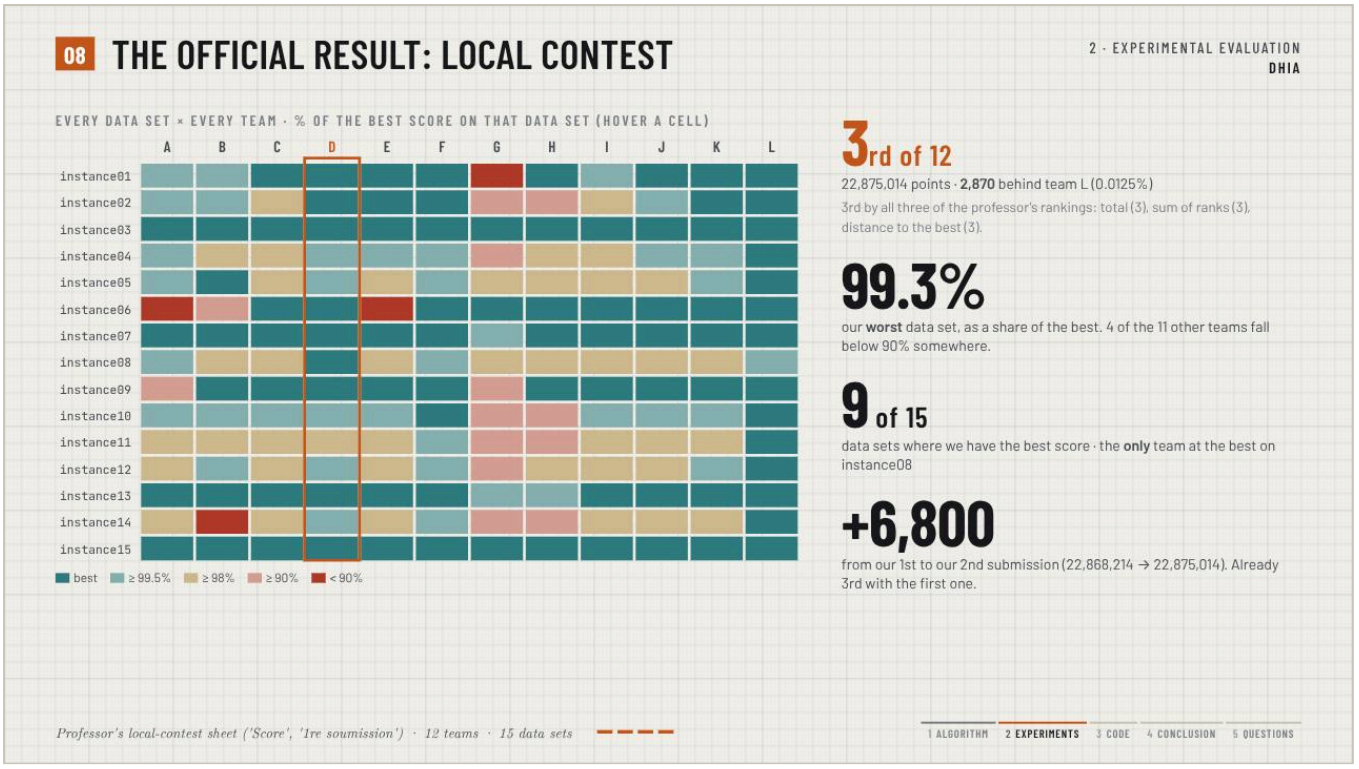


WHAT YOU SEE

Our best cells only.

IN PLAIN WORDS

> We have the best score on 9 of the 15 data sets, and we are the only team at the best on one of them.

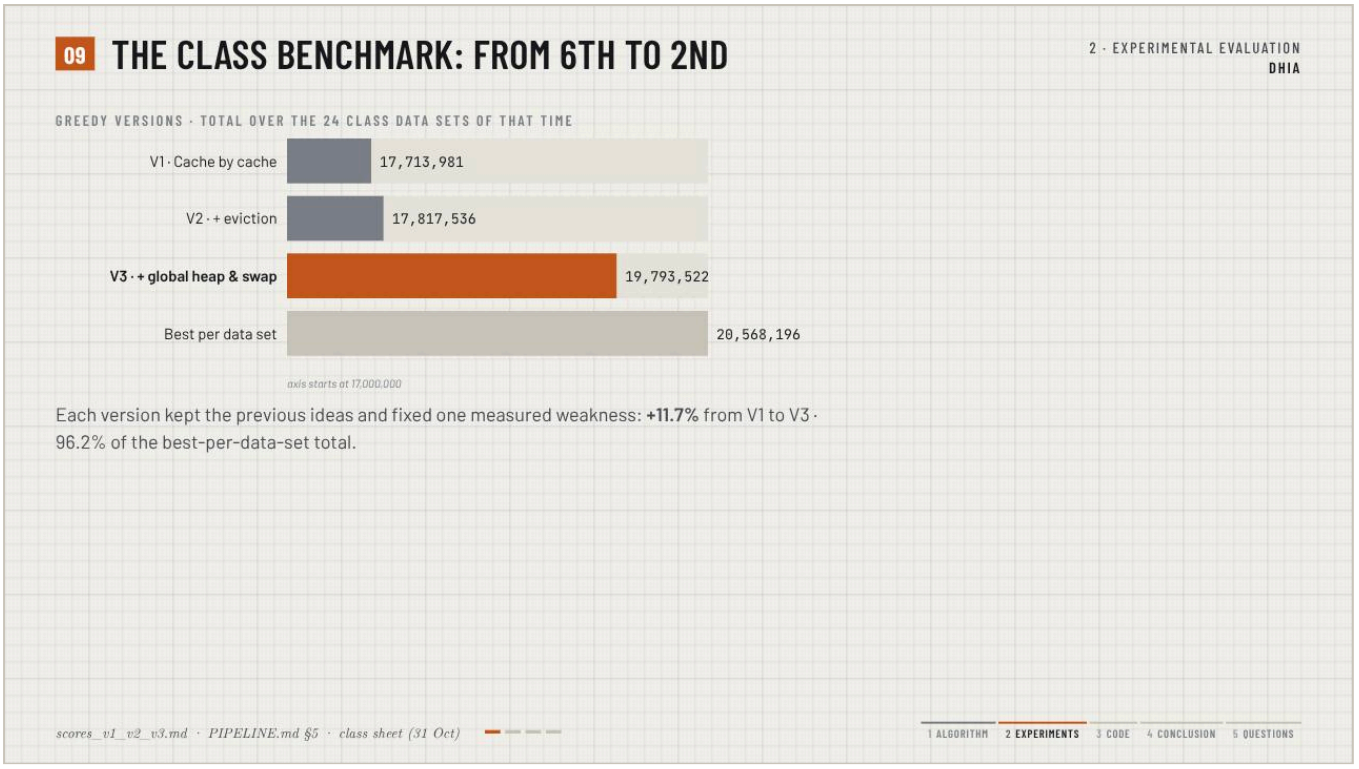


WHAT YOU SEE

First to second submission.

IN PLAIN WORDS

> Between our two submissions we gained 6,800 points. We were already third with the first.



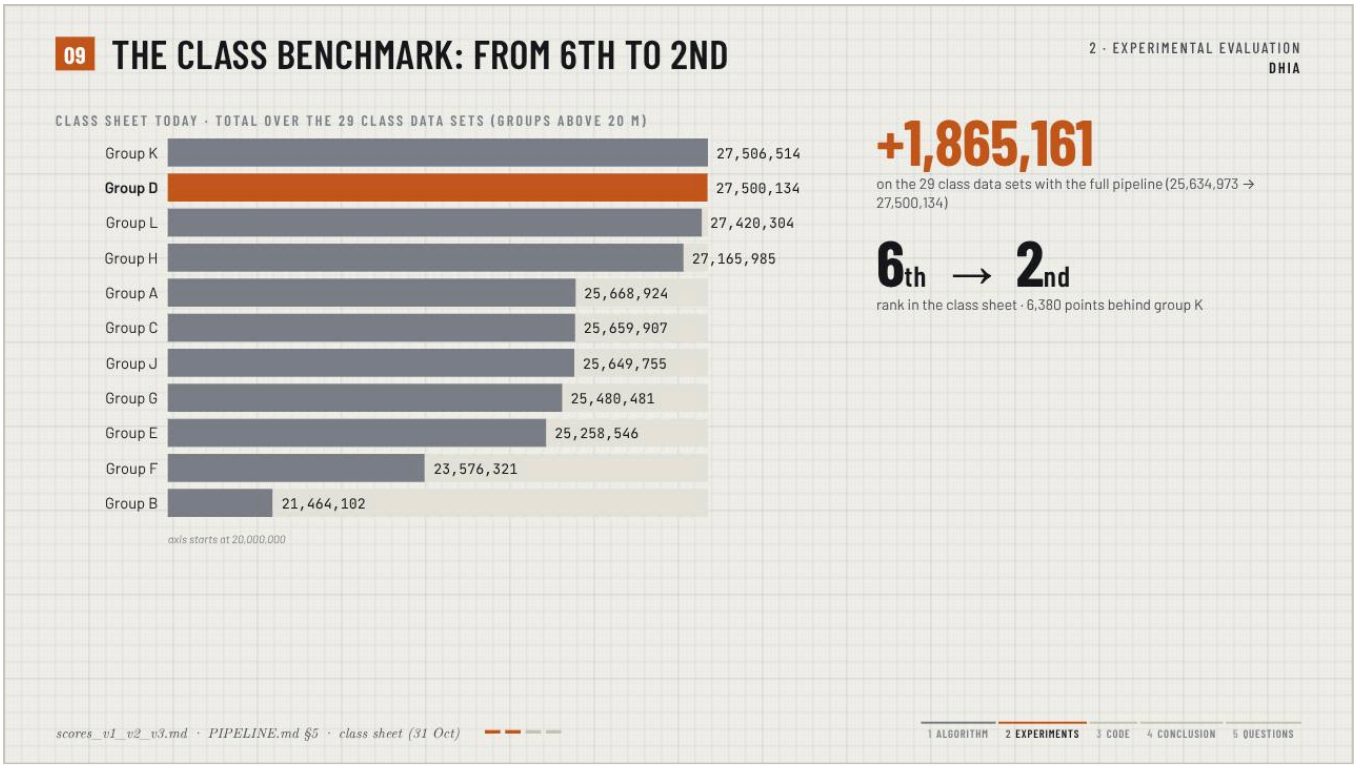
WHAT YOU SEE

V1 → V3 on the class data sets.

IN PLAIN WORDS

> On the class benchmark, the greedy versions first: cache by cache, then eviction, then one global heap with swaps. Plus 11.7 percent.

Word to know · Proven optimum An answer the exact solver proves nobody can beat.

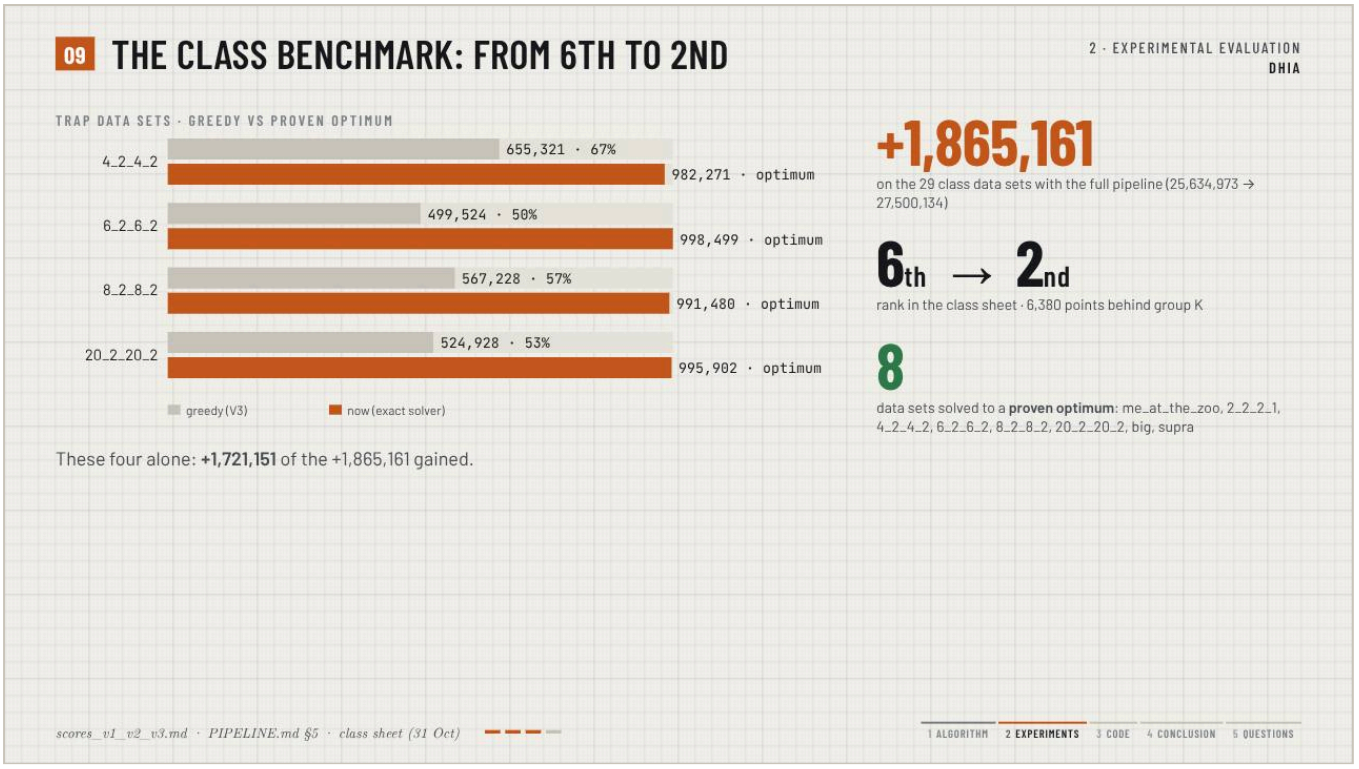


WHAT YOU SEE

The full pipeline: 6th → 2nd.

IN PLAIN WORDS

> Then the full pipeline: plus 1.87 million points on the 29 class data sets, from sixth to second in the class sheet.

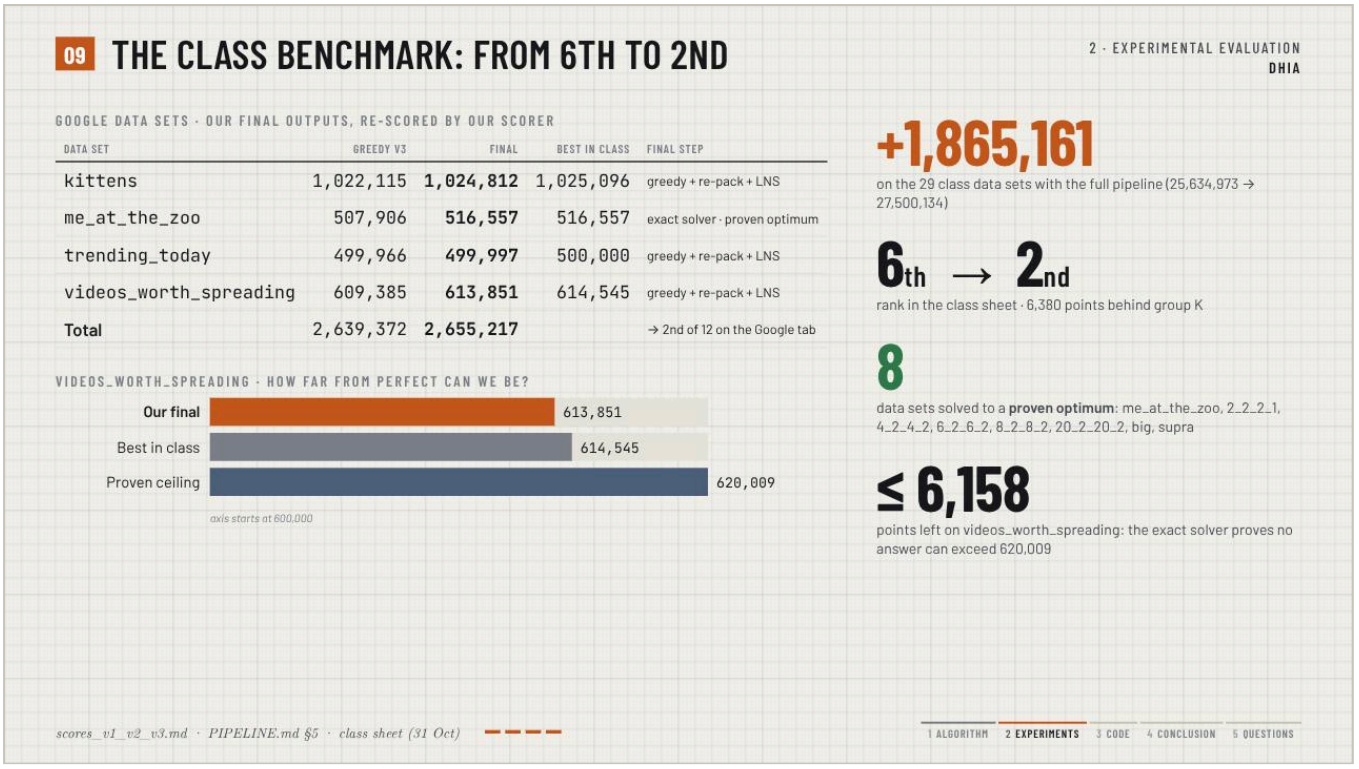


WHAT YOU SEE

Where the points came from: the traps.

IN PLAIN WORDS

> Most of it comes from the trap data sets: the greedy was at 50 to 67 percent, now they are proven optimal. Eight data sets are proven optimal in total.



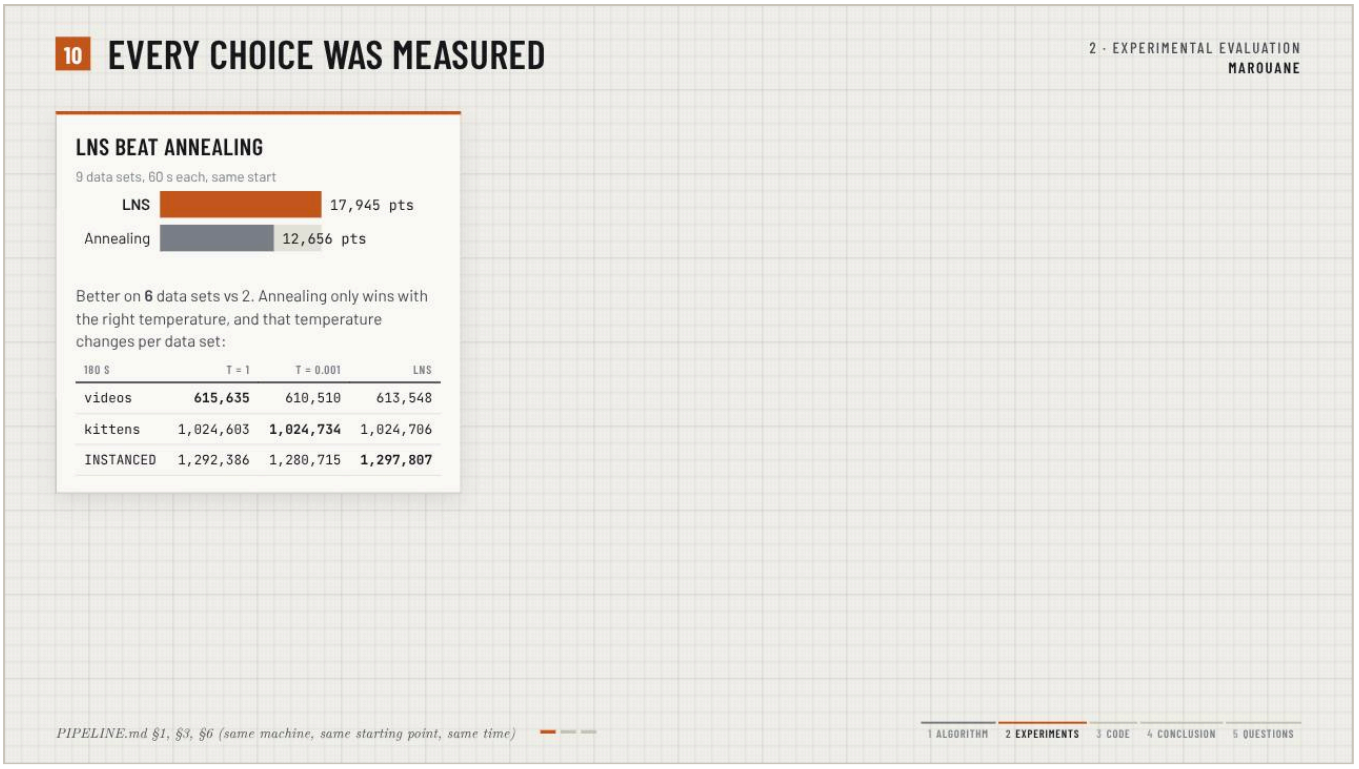
WHAT YOU SEE

Google data sets, and the ceiling on videos.

IN PLAIN WORDS

> On the four Google data sets: zoo is proven optimal, and on videos we know we are at most 6,158 points from perfect.

HANDOVER to Marouane.



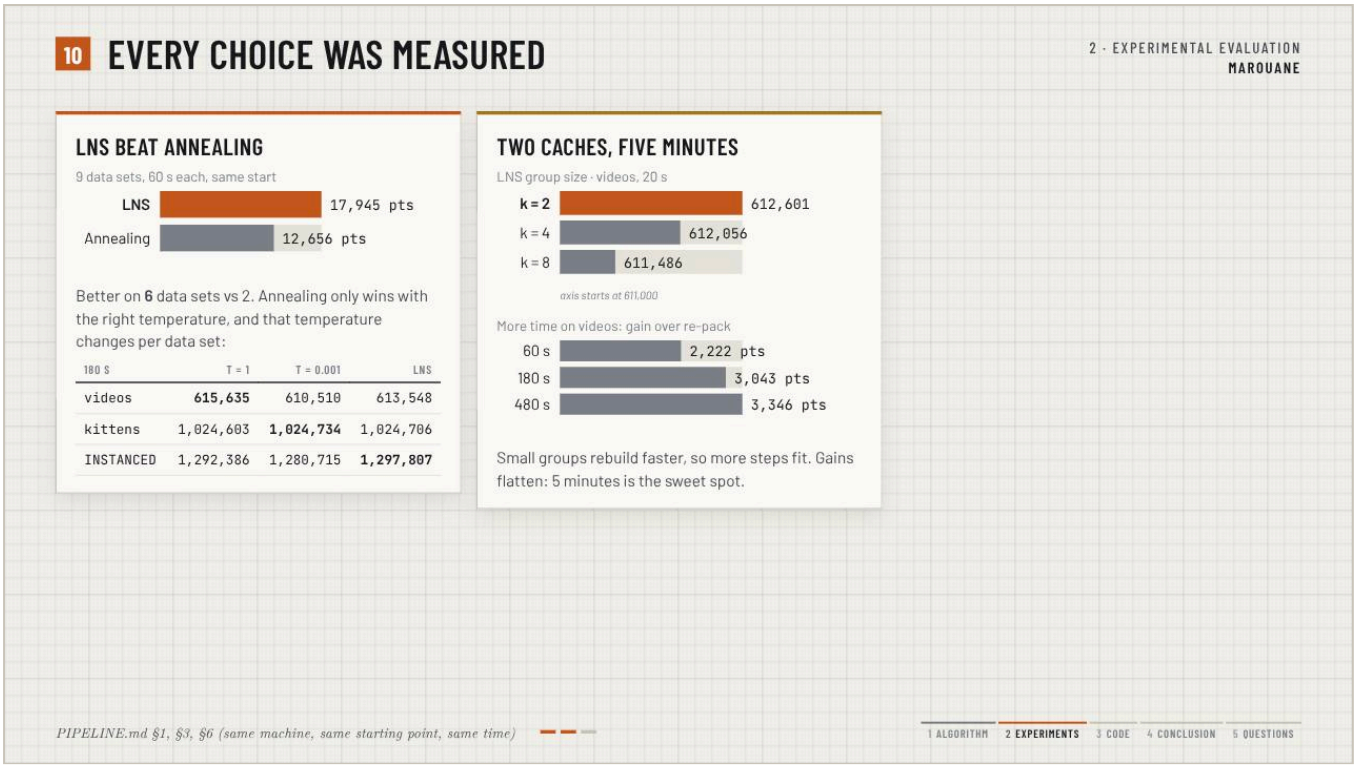
WHAT YOU SEE

LNS vs annealing.

IN PLAIN WORDS

> Every parameter was chosen by a test. We implemented simulated annealing and LNS and compared them from the same start, same time. LNS gained more and won on six of nine. Annealing needs the right temperature, and it changes per data set.

Word to know · Simulated annealing A search that sometimes accepts a worse answer to escape a dead end. We tested it; LNS won.

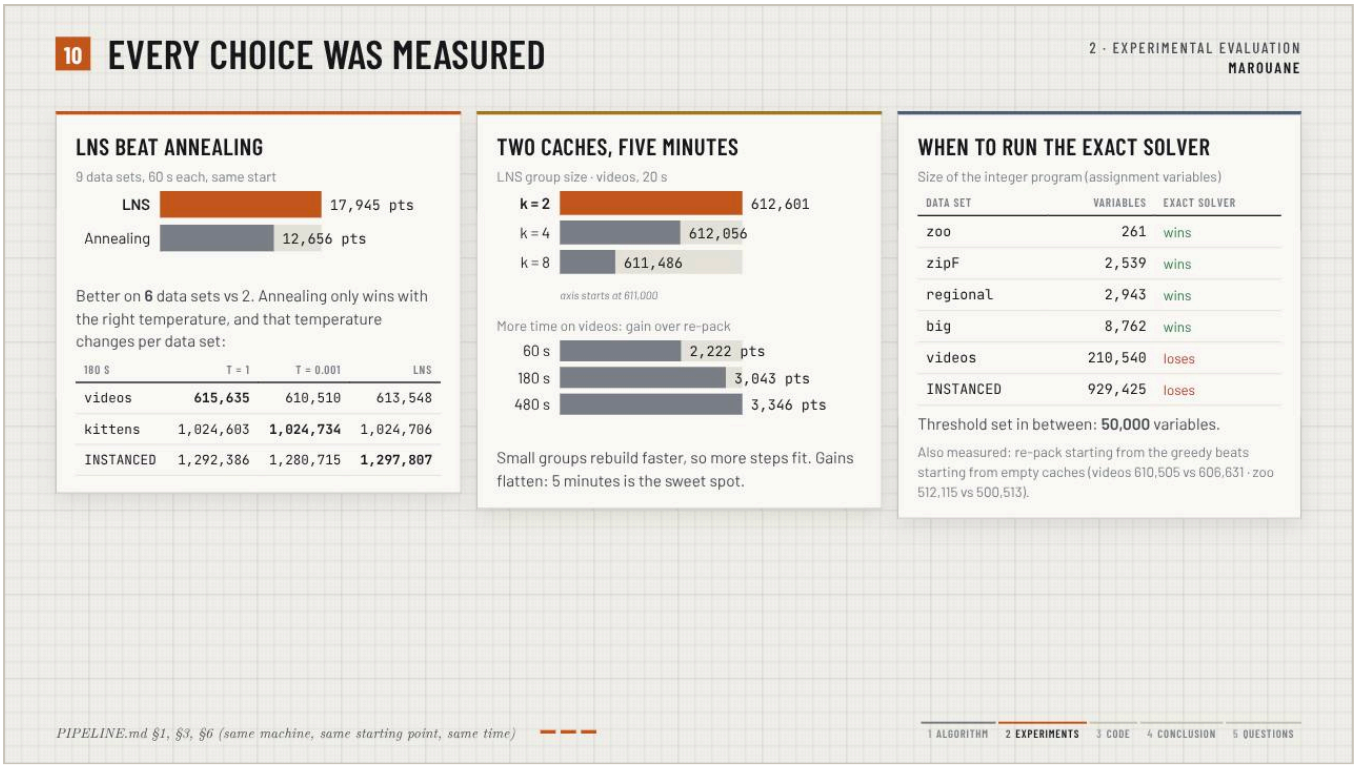


WHAT YOU SEE

Group size and time.

IN PLAIN WORDS

> Two caches per LNS step beats four or eight: smaller groups rebuild faster. And gains flatten after a few minutes, so five minutes per data set.



WHAT YOU SEE

Exact-solver threshold.

IN PLAIN WORDS

> The exact solver wins below ten thousand variables and loses above two hundred thousand, so we set the limit at fifty thousand.



WHAT YOU SEE

Strengths.

IN PLAIN WORDS

> Strengths: no step can lower the score, eight proven optima, and consistency: never below 99.3% of the best in the contest.

Word to know · Upper bound (ceiling) A number no answer can pass. On videos_worth_spreading: 620,009.

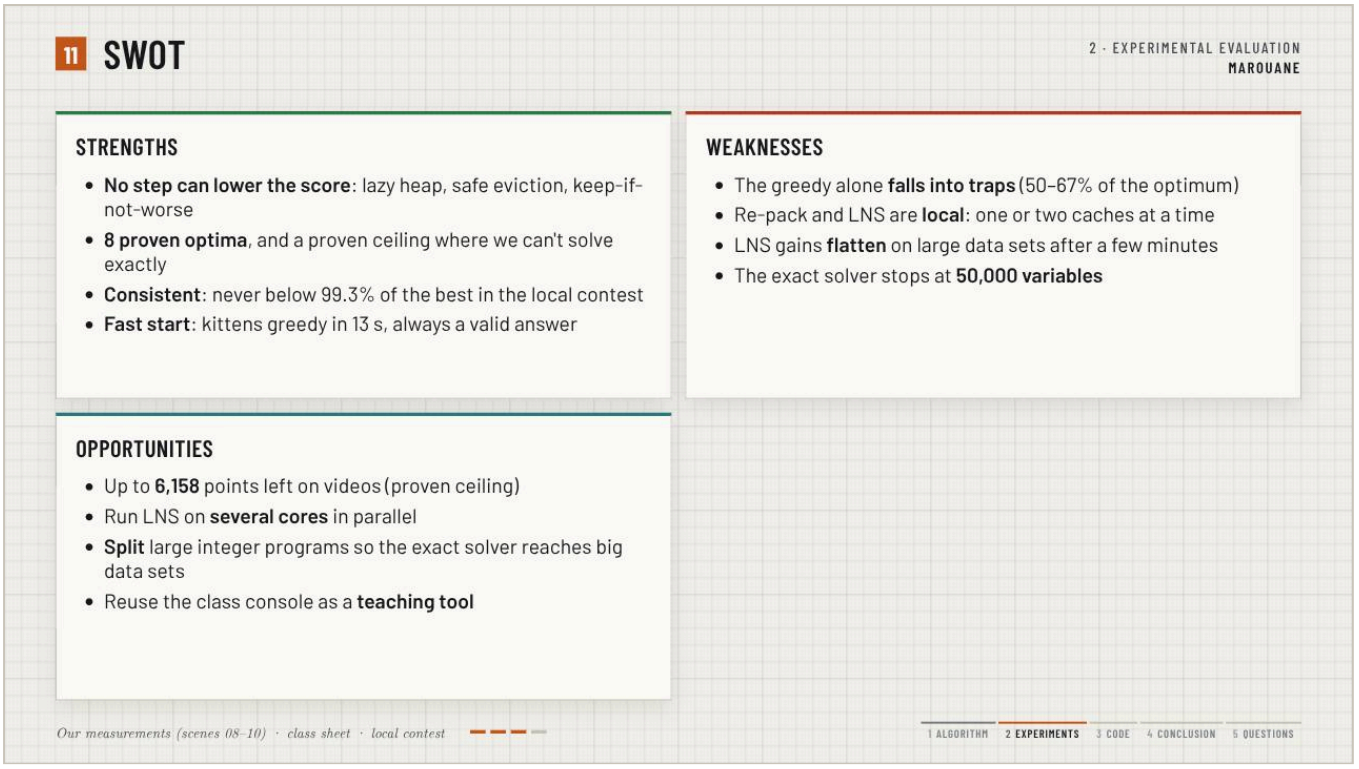


WHAT YOU SEE

Weaknesses.

IN PLAIN WORDS

> Weaknesses: the greedy alone falls into traps, our improvements are local, and the exact solver stops at fifty thousand variables.

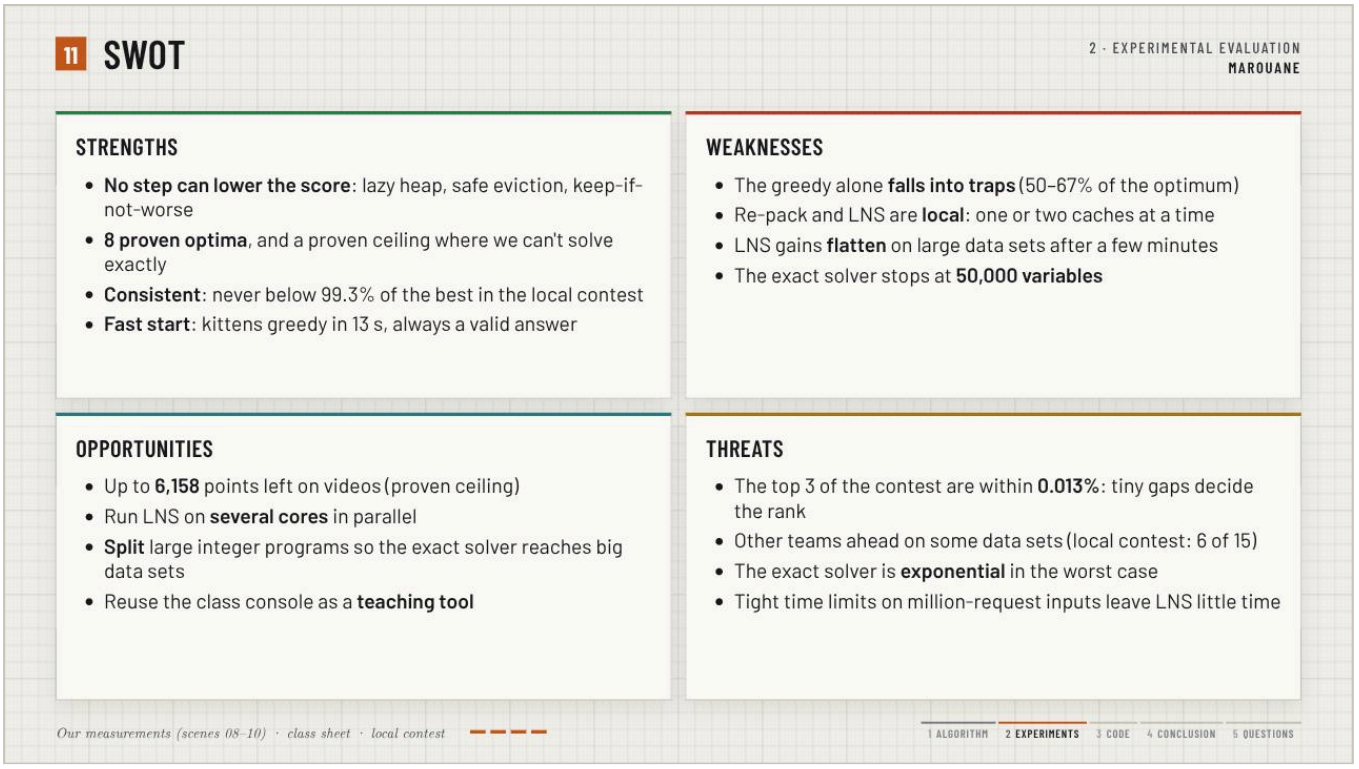


WHAT YOU SEE

Opportunities.

IN PLAIN WORDS

> Opportunities: about six thousand points left on videos, parallel LNS, and splitting big integer programs.

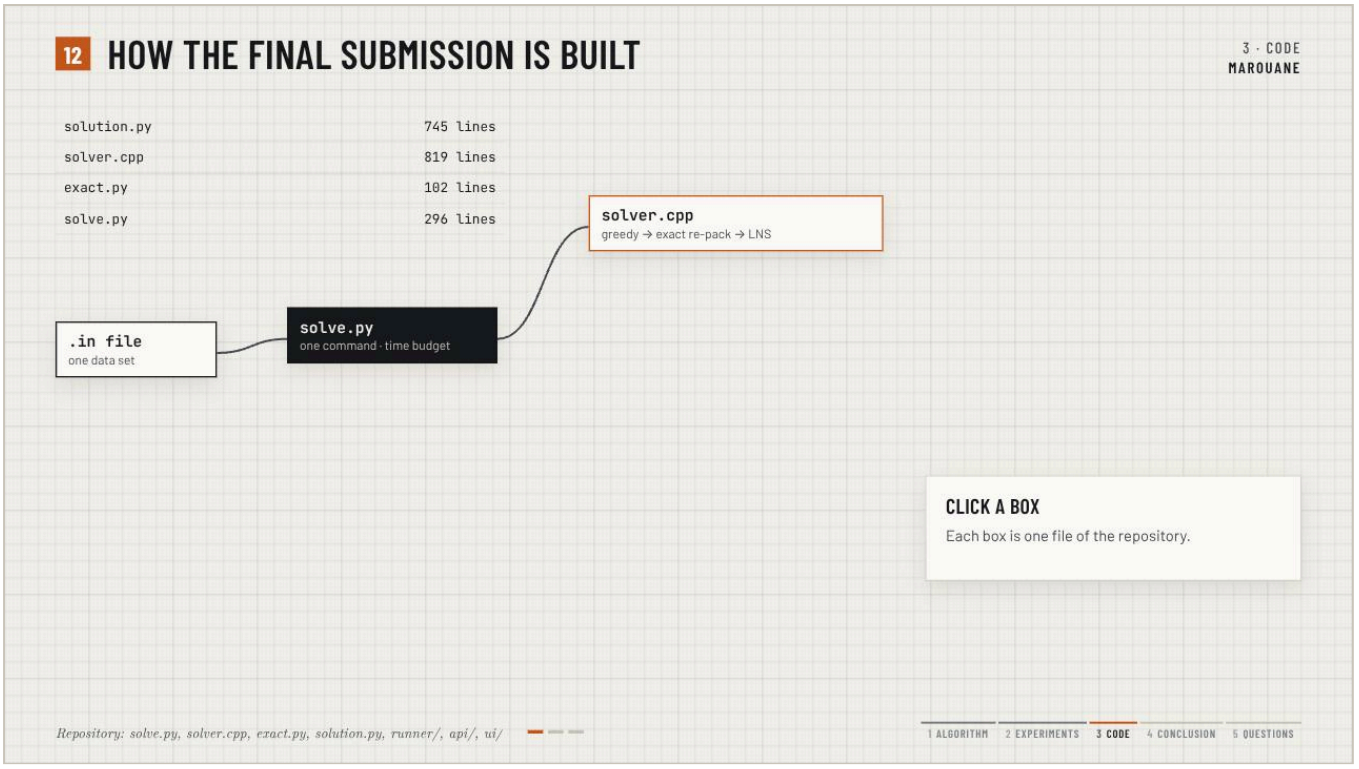


WHAT YOU SEE

Threats.

IN PLAIN WORDS

> Threats: the top three of the contest are within a hundredth of a percent, so small details decide the rank.



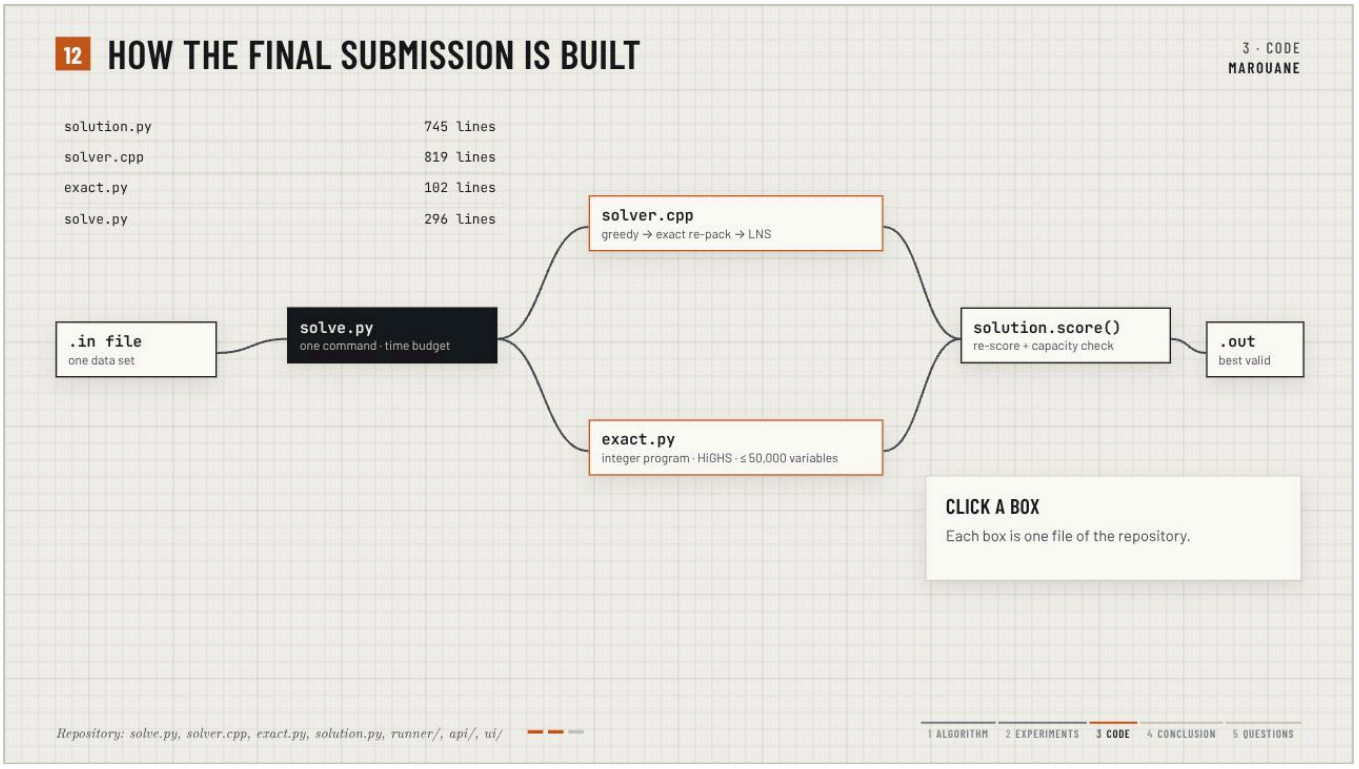
WHAT YOU SEE

Core flow: .in → solve.py → solver.cpp.

IN PLAIN WORDS

> Now the code of our final submission. One command, solve.py, takes a data set and a time budget, and runs our C++ solver: greedy, re-pack, LNS.

Word to know · Integer program (MILP) The whole problem written as equations with 0/1 variables, given to a solver (HiGHS).

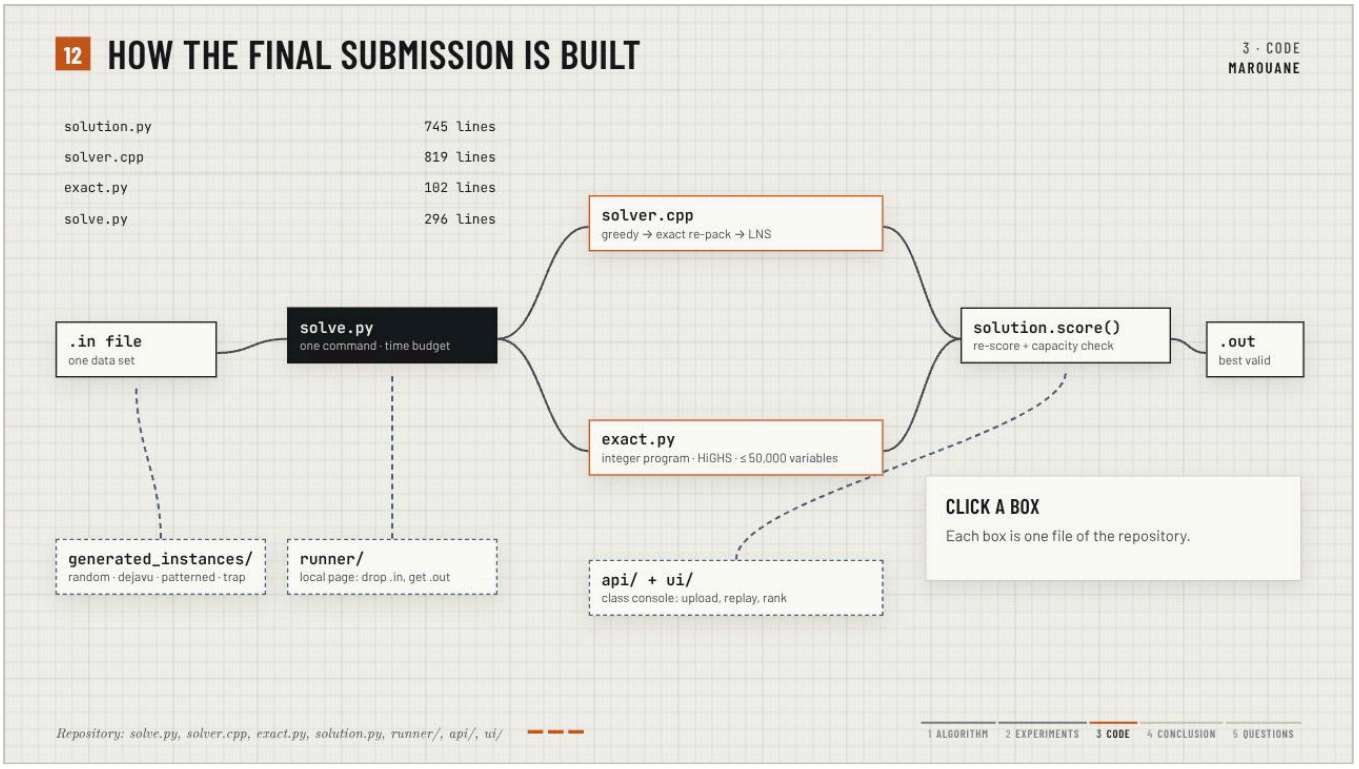


WHAT YOU SEE

Exact solver in parallel, re-score, keep the best.

IN PLAIN WORDS

> If the model is small, it runs the exact solver at the same time. Every candidate is re-scored by our independent Python scorer and checked for capacity. The best valid one is the .out.

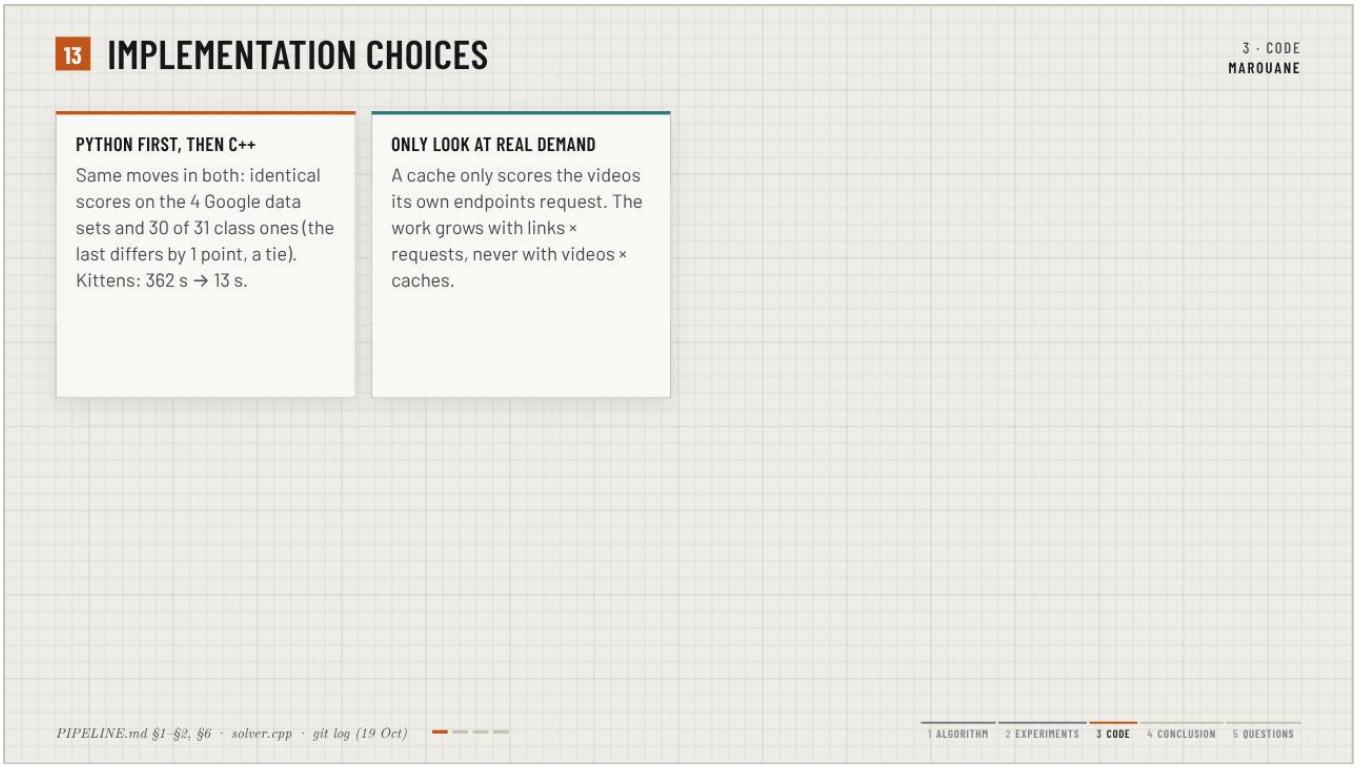


WHAT YOU SEE

The tools around it. Click any box to show what it does.

IN PLAIN WORDS

> Around it: the instance generator, a local runner page, and the class console you used, which reuses our parser and scorer.



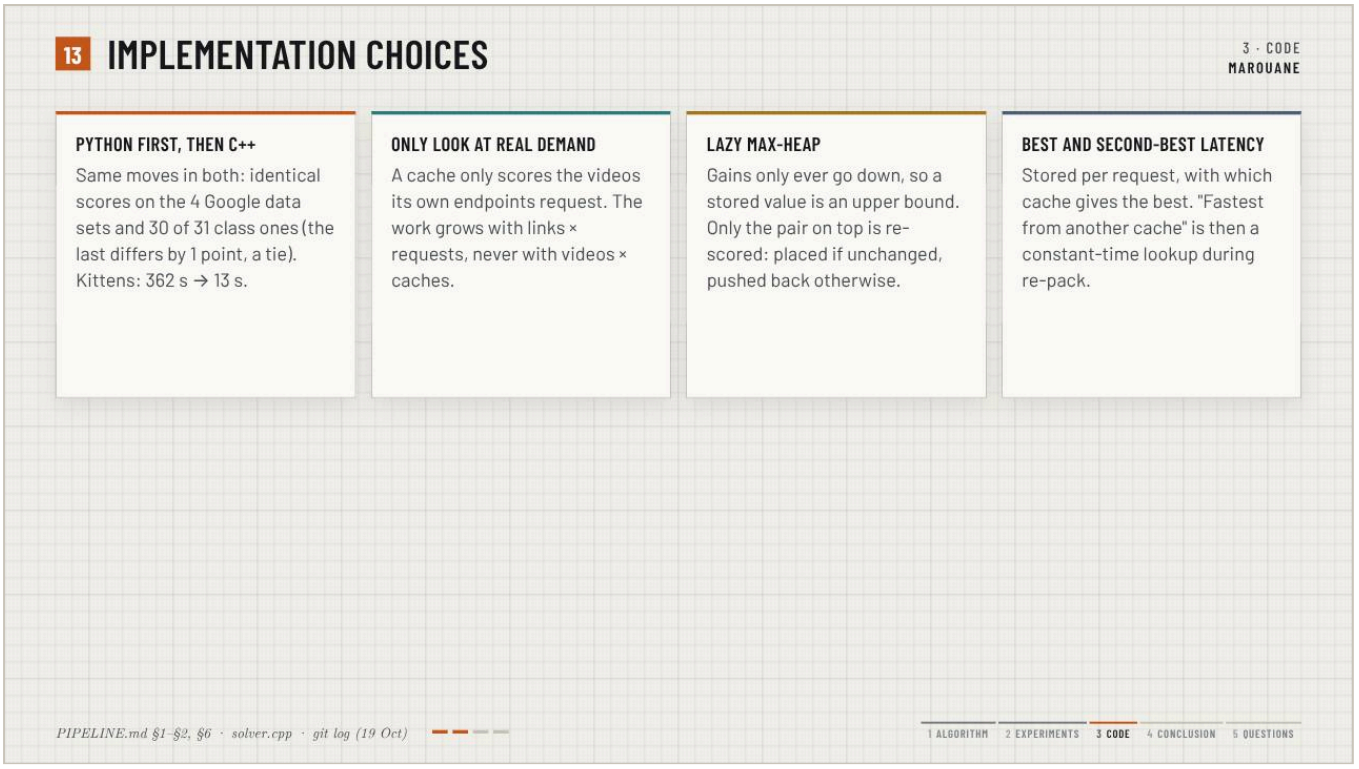
WHAT YOU SEE

Python first, then C++ · real demand only.

IN PLAIN WORDS

> Two choices for speed. We wrote the Python version first and ported it move by move to C++: same scores, kittens from six minutes to 13 seconds. And a cache only ever looks at the videos its own users request.

Word to know · gcd Greatest common divisor. Dividing all sizes by it keeps the same answer with a smaller table.

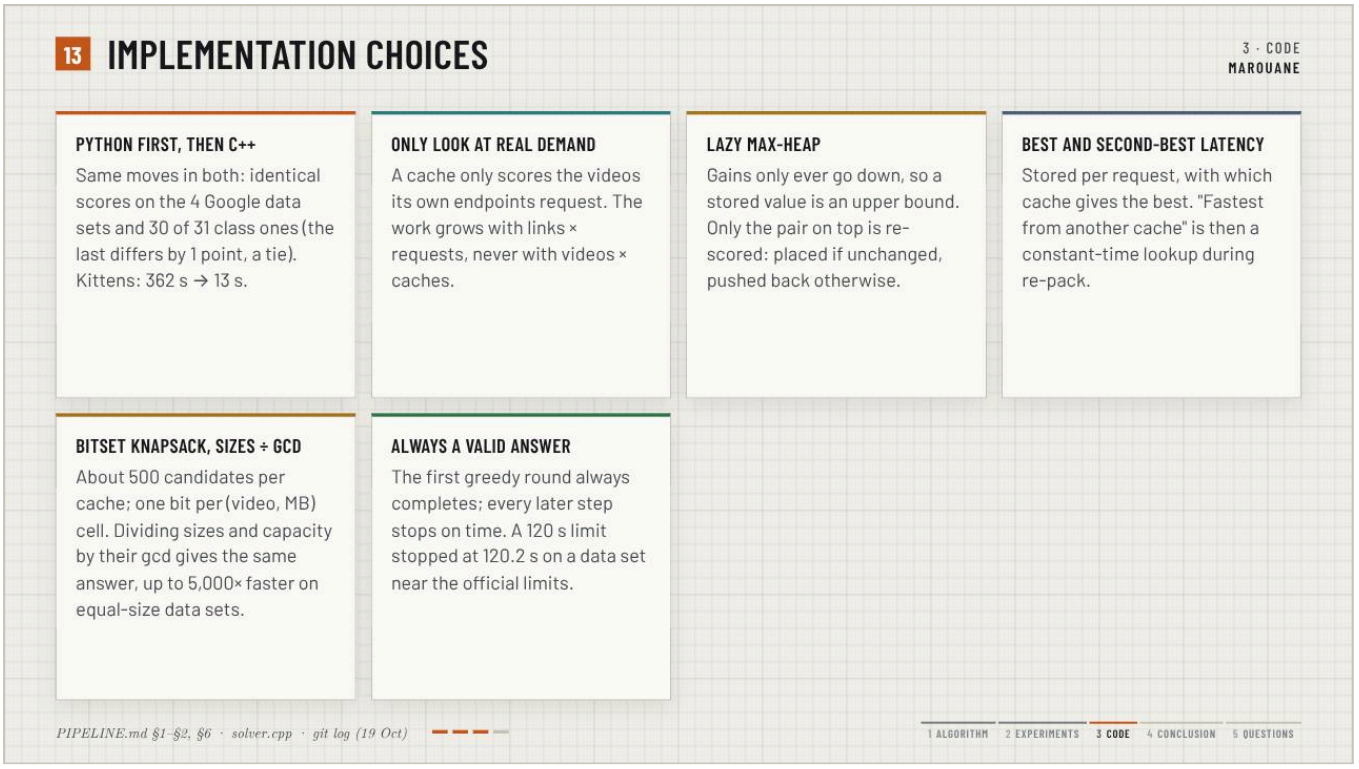


WHAT YOU SEE

Lazy heap · best and second-best latency.

IN PLAIN WORDS

> The heap is lazy: gains only go down, so we only re-check the top. And we store the best and second-best latency per request, so re-pack gets "fastest from another cache" instantly.

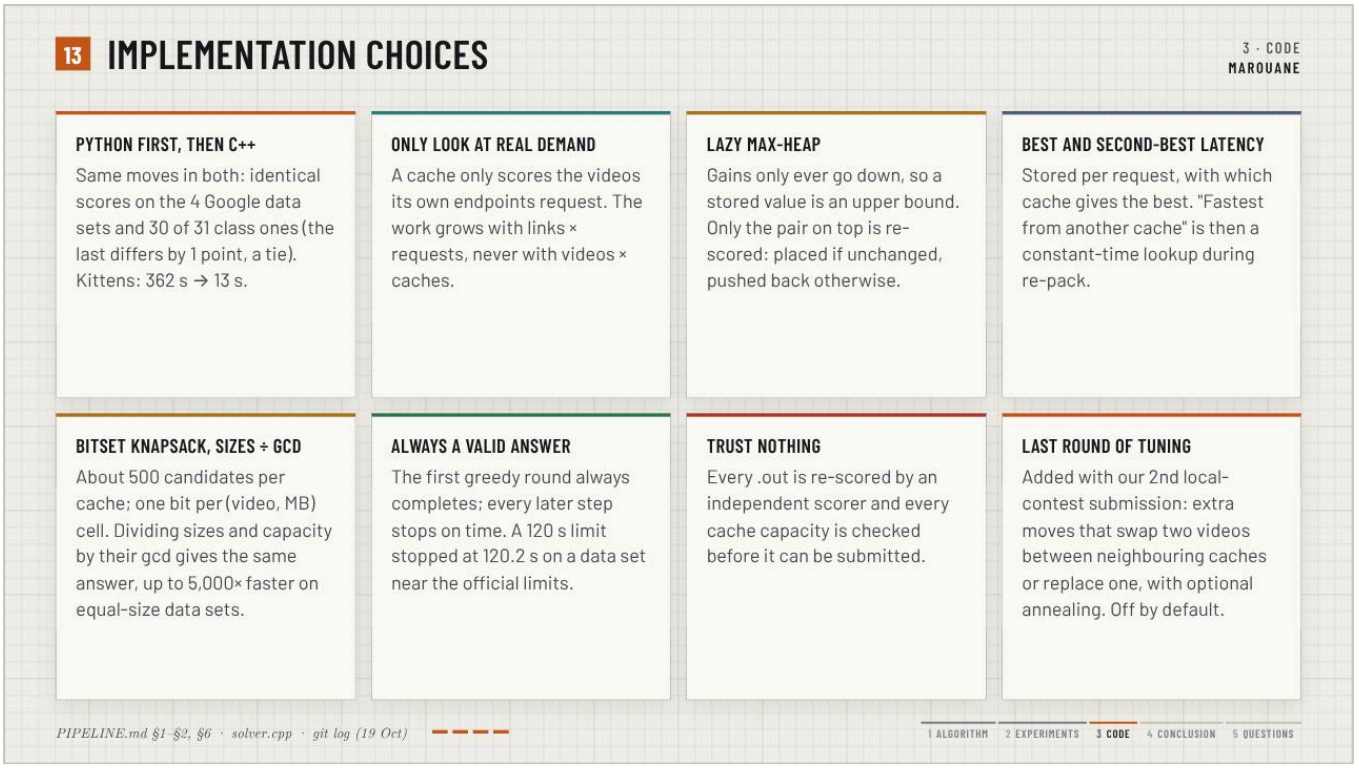


WHAT YOU SEE

Bitset knapsack · always a valid answer.

IN PLAIN WORDS

> The knapsack uses one bit per cell, and dividing sizes by their gcd makes it up to 5,000 times faster on equal sizes. And the first greedy round always finishes, so there is always a valid answer on time.



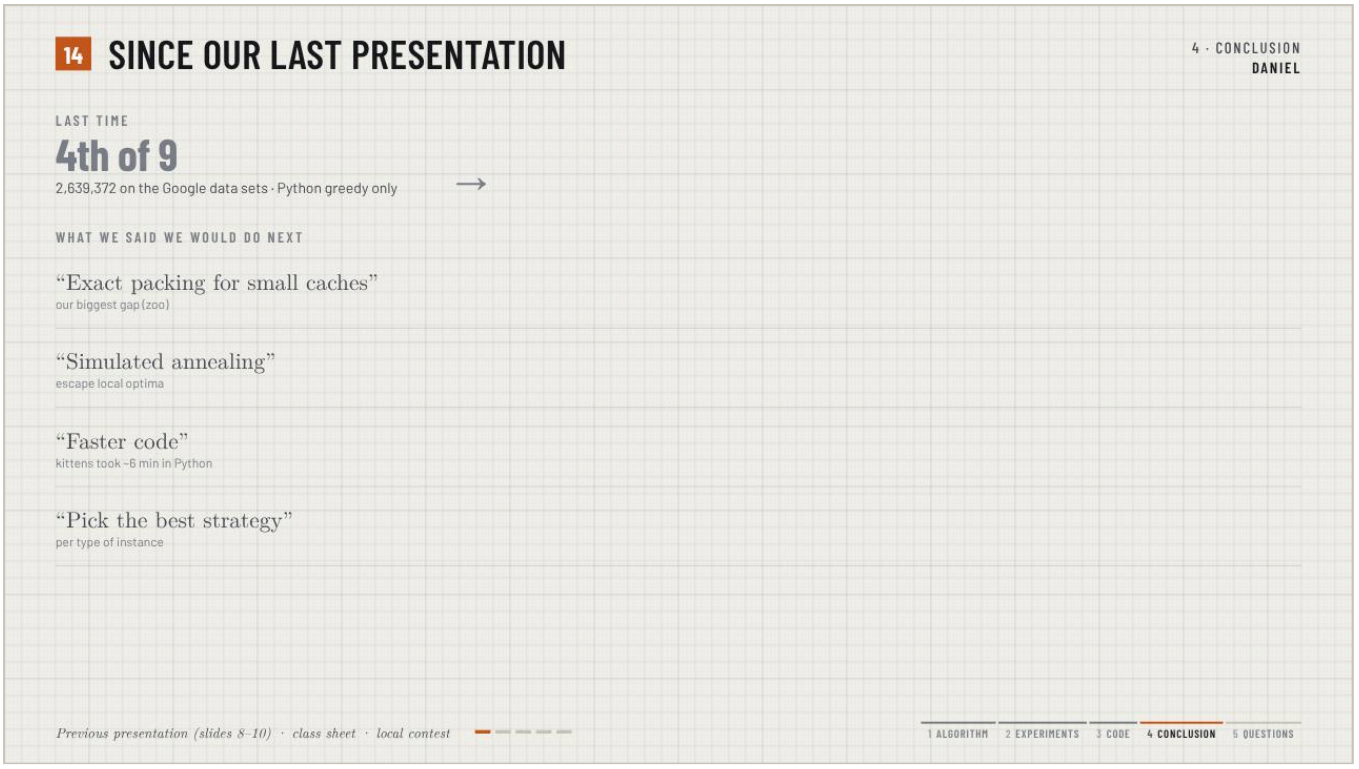
WHAT YOU SEE

Trust nothing · last round of tuning.

IN PLAIN WORDS

> Nothing is submitted before an independent re-score. And with the second contest submission we added extra swap and replace moves between neighbouring caches.

HANDOVER to Daniel.



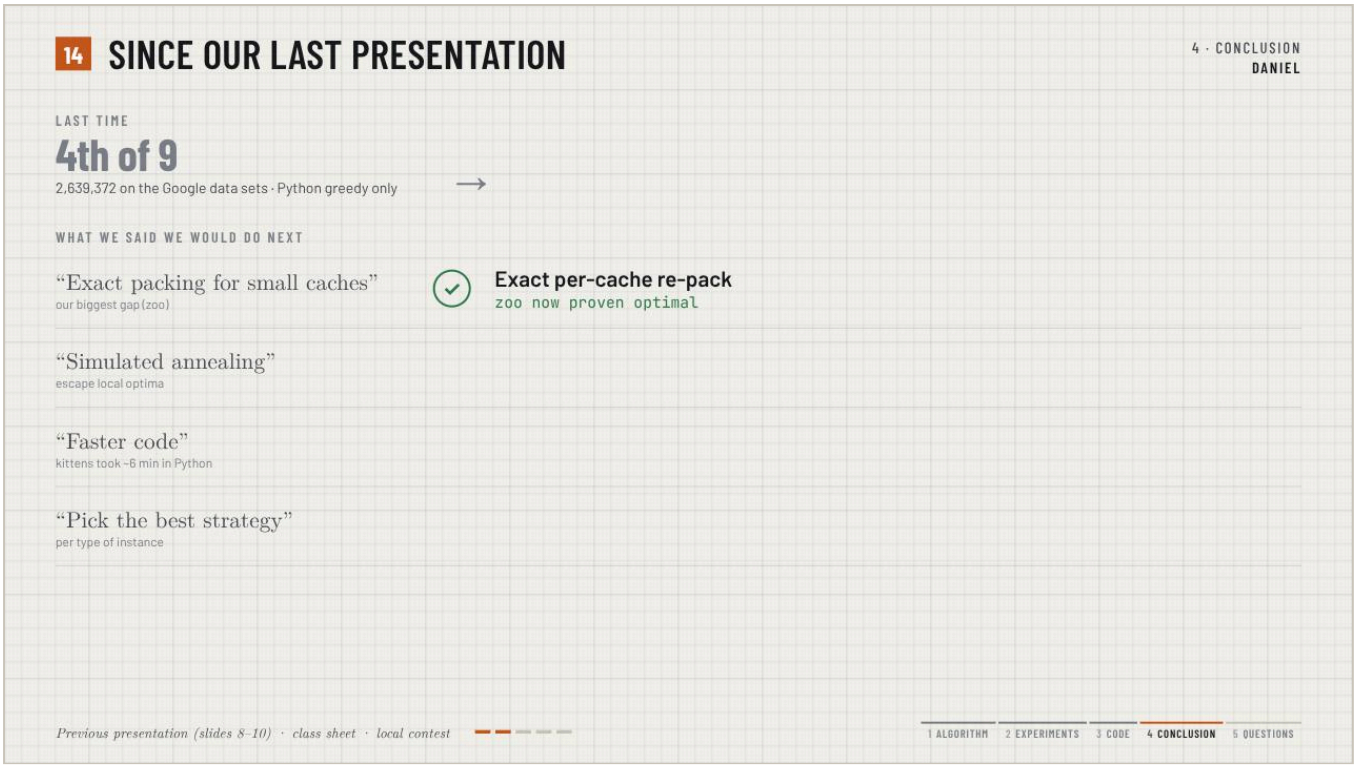
WHAT YOU SEE

Last time: 4th of 9, Python only. The list of next steps we promised.

IN PLAIN WORDS

> To conclude, let's go back to our last presentation. We were fourth of nine, with a Python greedy, and we listed what we would do next.

Word to know · Re-pack Freeze every other cache and fill one cache exactly with a knapsack.

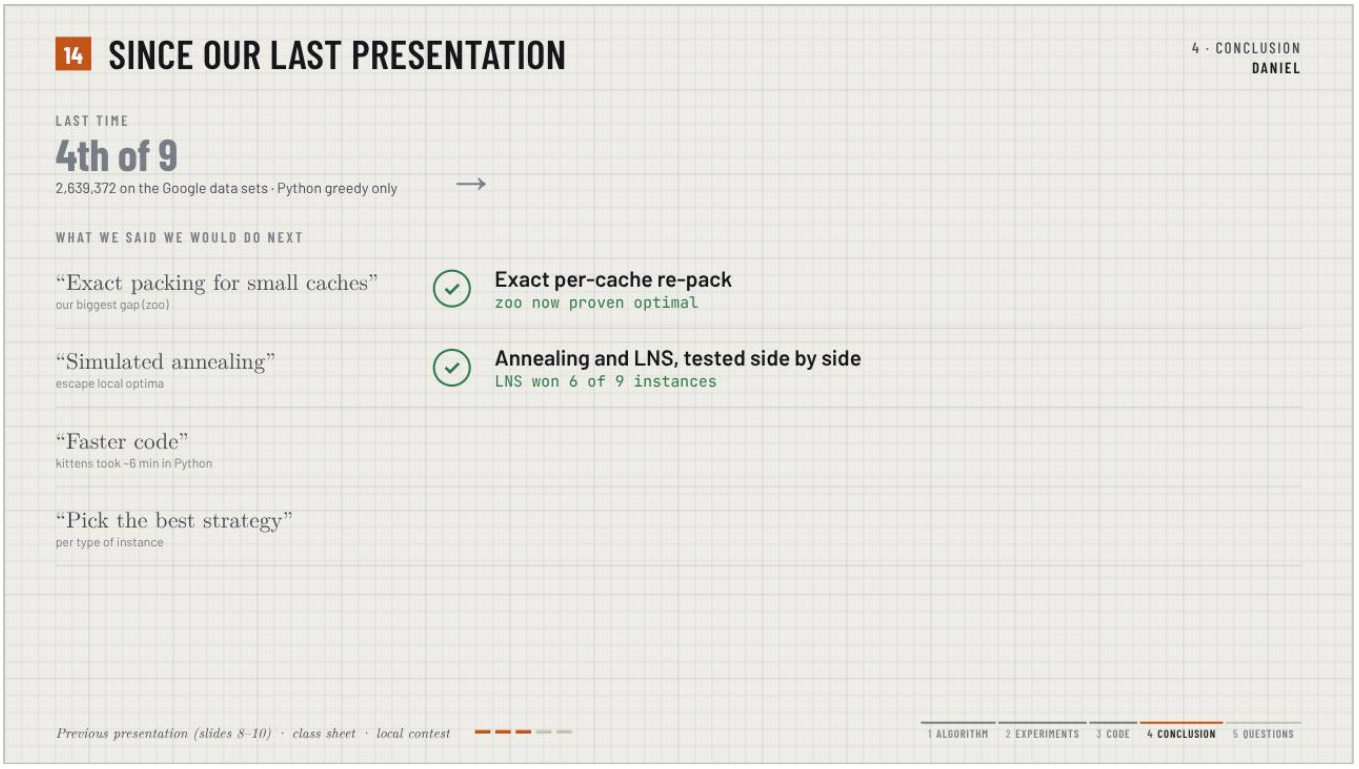


WHAT YOU SEE

✓ Exact packing.

IN PLAIN WORDS

> Exact packing for small caches: done, it is the re-pack. Zoo is now proven optimal.

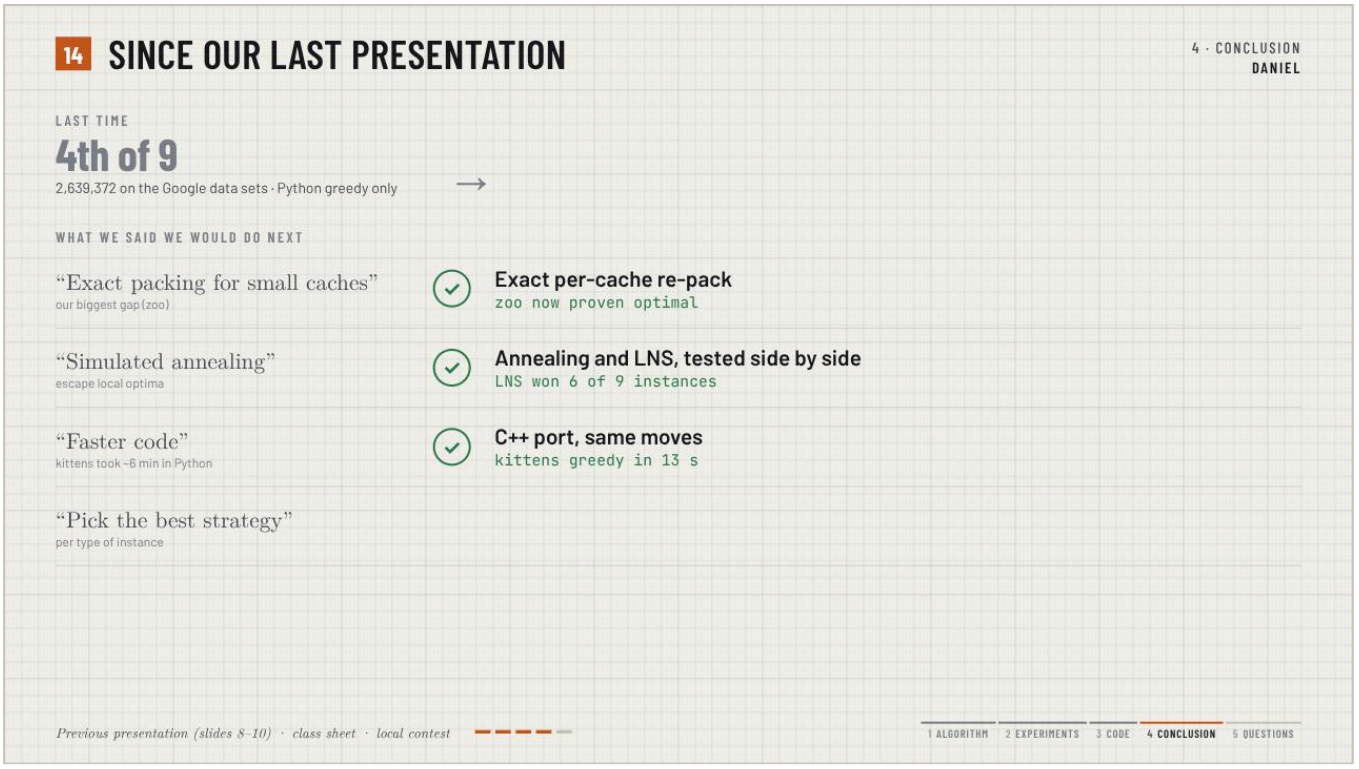


WHAT YOU SEE

✓ Annealing.

IN PLAIN WORDS

> Simulated annealing: done, and tested against LNS. LNS won, so we kept LNS.

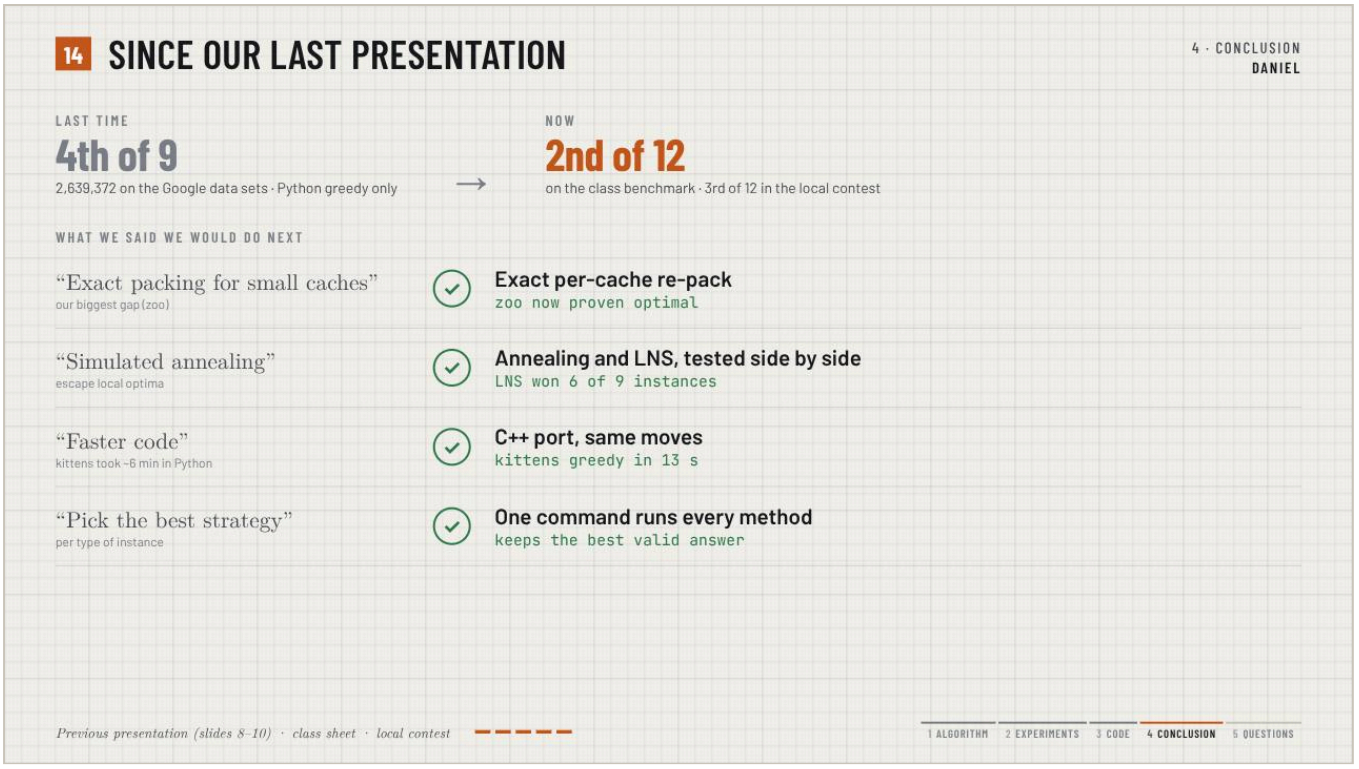


WHAT YOU SEE

✓ Faster code.

IN PLAIN WORDS

> Faster code: the C++ port, kittens in 13 seconds.



WHAT YOU SEE

✓ Best strategy + now.

IN PLAIN WORDS

> Pick the best strategy: one command runs everything and keeps the best. Today we are second of twelve on the class benchmark, and third in the local contest.



15 CONCLUSION

4 · CONCLUSION
DANIEL

LAYER	EXACT?	CHANGES PER STEP	FIXES
Greedy (heap, evict, swap)	no	one (cache, video) pair	a good start, fast
Exact re-pack	yes, one cache	a whole cache	the ratio rule misses combinations
Large neighbourhood search	yes, two caches	two linked caches	re-pack is stuck one cache at a time
Exact solver	yes, globally	the whole solution	traps · gives a proven ceiling

Each layer fixes one **measured** weakness of the one below, and none can make the answer worse.

Scenes 03–14

1 ALGORITHM 2 EXPERIMENTS 3 CODE 4 CONCLUSION 5 QUESTIONS

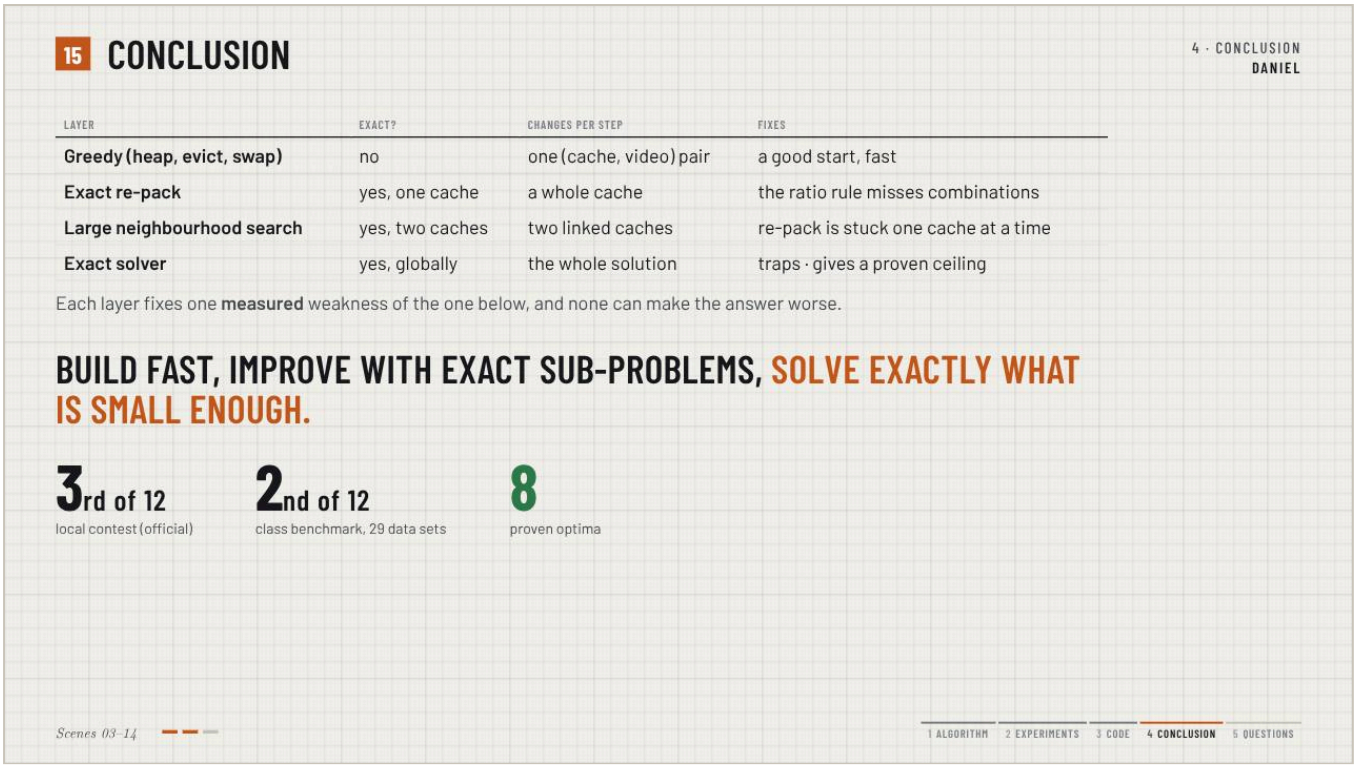
WHAT YOU SEE

The four layers and what each fixes.

IN PLAIN WORDS

> So our solution is four layers. Each one fixes a weakness we measured in the one before, and none can make the answer worse.

Word to know · Branch and bound How the solver searches: split the problem, and skip parts that cannot beat the best so far.



15 CONCLUSION

4 · CONCLUSION
DANIEL

LAYER	EXACT?	CHANGES PER STEP	FIXES
Greedy (heap, evict, swap)	no	one (cache, video) pair	a good start, fast
Exact re-pack	yes, one cache	a whole cache	the ratio rule misses combinations
Large neighbourhood search	yes, two caches	two linked caches	re-pack is stuck one cache at a time
Exact solver	yes, globally	the whole solution	traps · gives a proven ceiling

Each layer fixes one **measured** weakness of the one below, and none can make the answer worse.

BUILD FAST, IMPROVE WITH EXACT SUB-PROBLEMS, SOLVE EXACTLY WHAT IS SMALL ENOUGH.

3rd of 12
local contest (official)

2nd of 12
class benchmark, 29 data sets

8
proven optima

Scenes 03–14

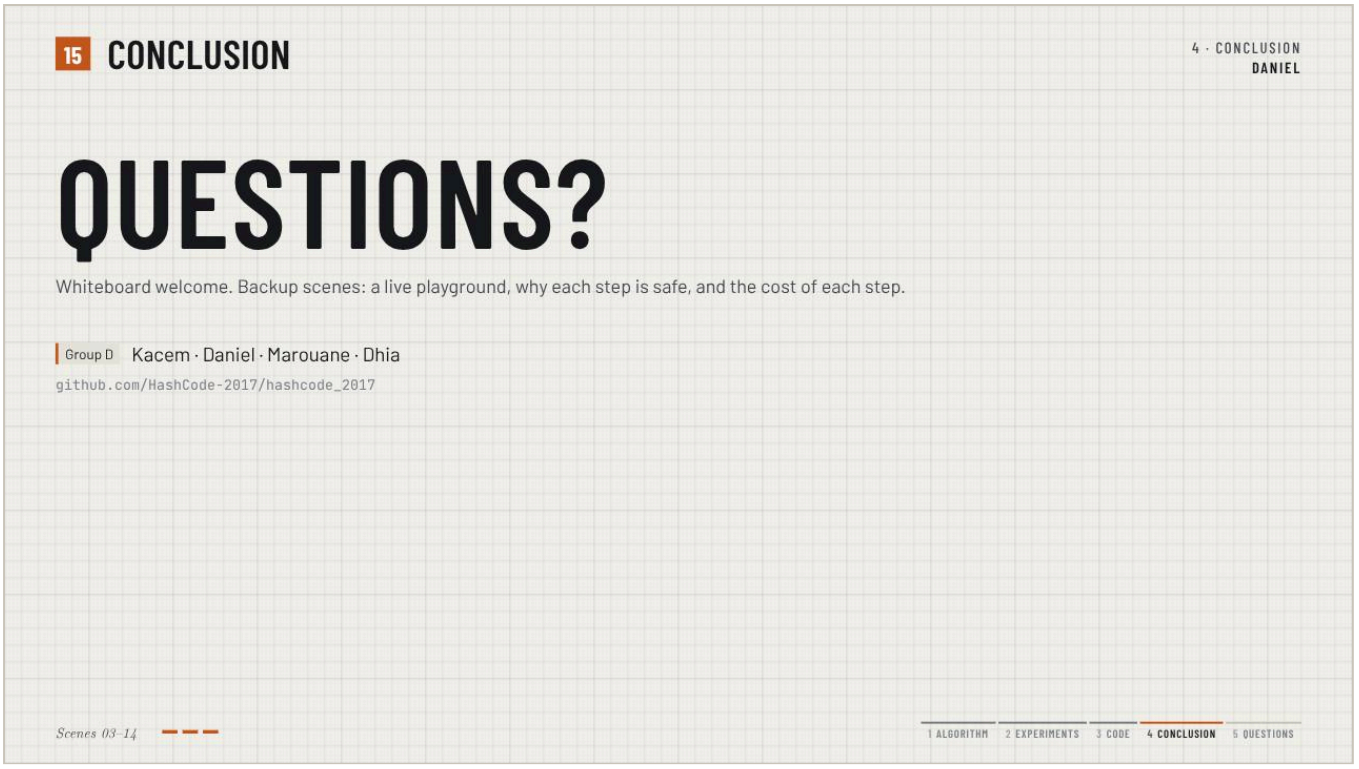
1 ALGORITHM 2 EXPERIMENTS 3 CODE 4 CONCLUSION 5 QUESTIONS

WHAT YOU SEE

The one-sentence summary and three numbers.

IN PLAIN WORDS

> Build fast, improve with exact sub-problems, and solve exactly what is small enough. Third of twelve in the contest, second on the class benchmark, eight proven optima.

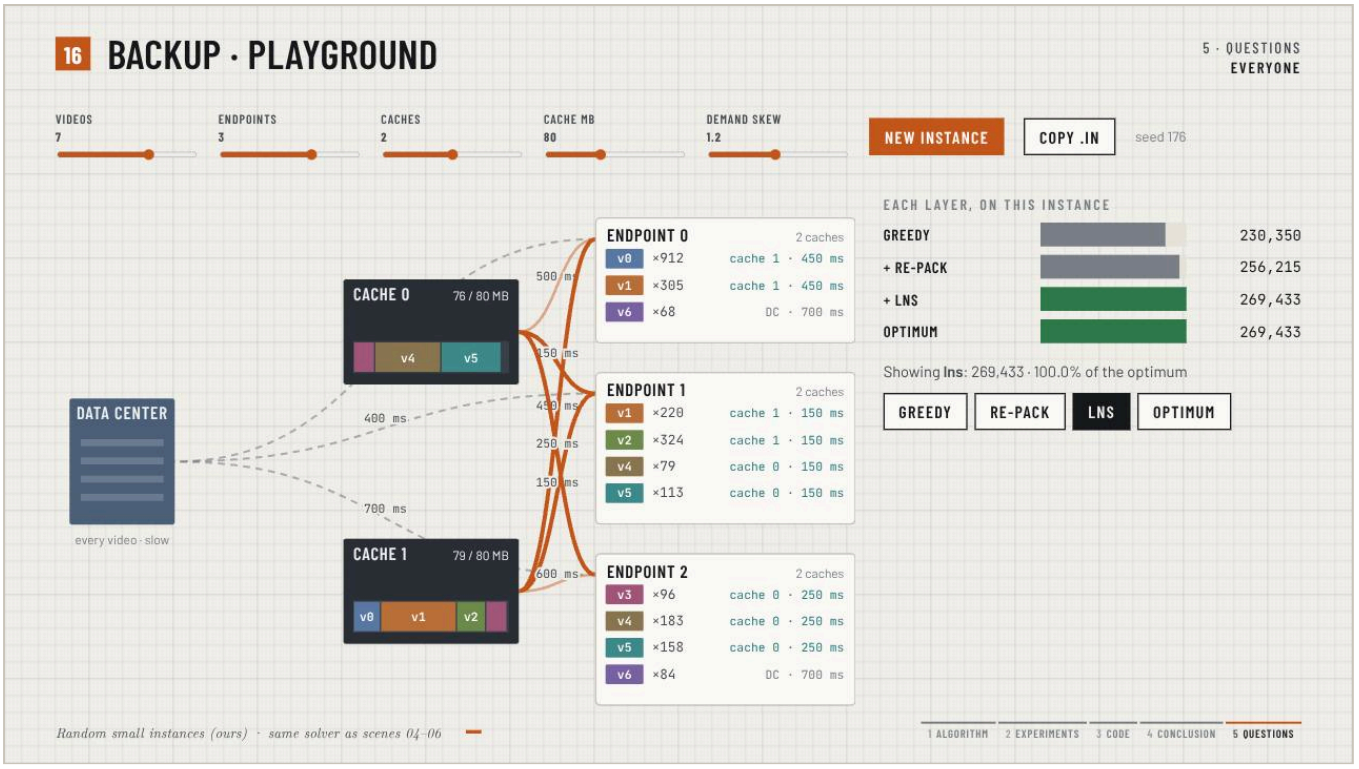


WHAT YOU SEE

Questions. Backup scenes: 16 playground, 17 proofs, 18 complexity (type 16, 17, 18).

IN PLAIN WORDS

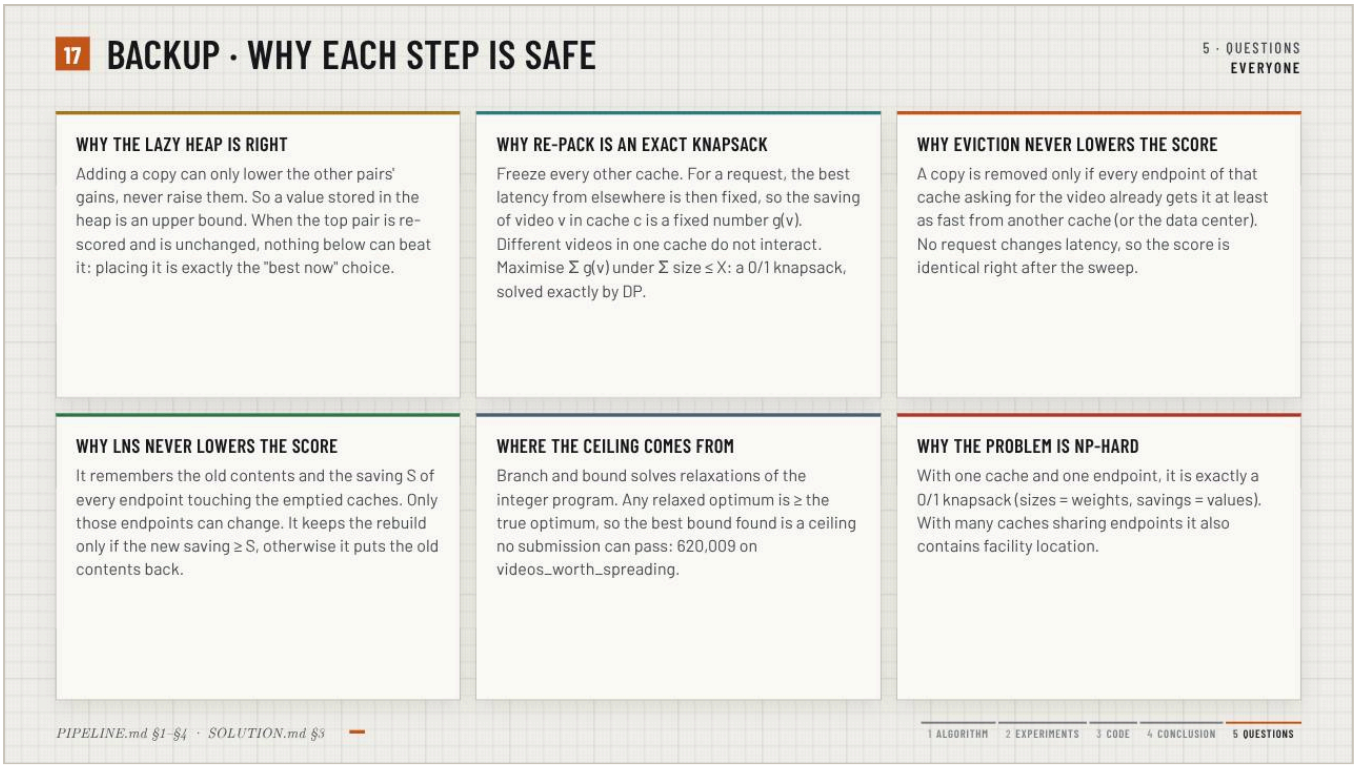
> Thank you. We are happy to take your questions.



WHAT YOU SEE

Playground: sliders change the instance; buttons show each layer's placement. Use it to answer "what if" questions.

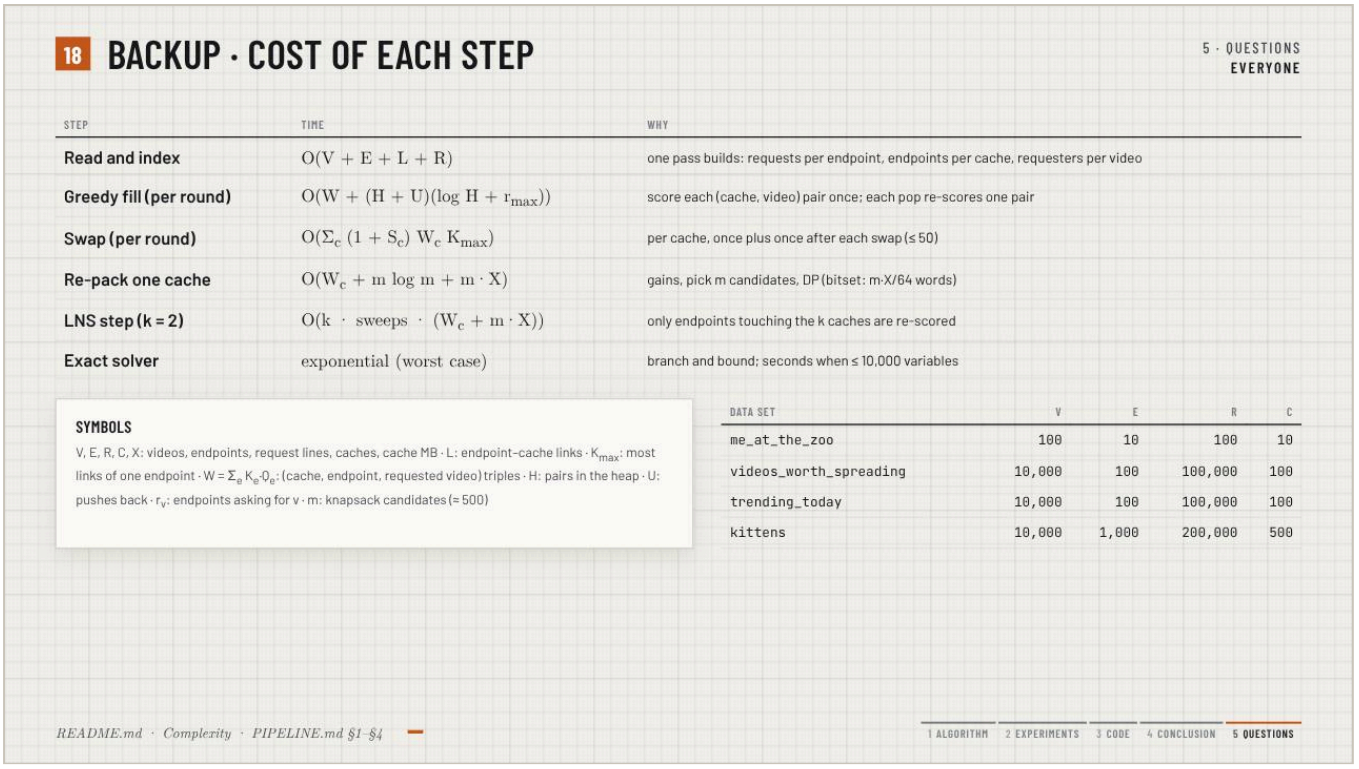
Word to know · Greedy Build the answer one best-looking choice at a time, without going back.



WHAT YOU SEE

Proof sketches for the board: lazy heap, re-pack = knapsack, eviction, LNS acceptance, the ceiling, NP-hardness.

Word to know · Lazy heap We only re-check the top item. Values only go down, so the top is safe if unchanged.



18 BACKUP · COST OF EACH STEP

5 · QUESTIONS
EVERYONE

STEP	TIME	WHY
Read and index	$O(V + E + L + R)$	one pass builds: requests per endpoint, endpoints per cache, requesters per video
Greedy fill (per round)	$O(W + (H + U)(\log H + r_{max}))$	score each (cache, video) pair once; each pop re-scores one pair
Swap (per round)	$O(\Sigma_c (1 + S_c)\, W_c\, K_{max})$	per cache, once plus once after each swap (≤ 50)
Re-pack one cache	$O(W_c + m \log m + m \cdot X)$	gains, pick m candidates, DP (bitset: m·X/64 words)
LNS step (k = 2)	$O(k \cdot \text{sweeps} \cdot (W_c + m \cdot X))$	only endpoints touching the k caches are re-scored
Exact solver	exponential (worst case)	branch and bound; seconds when ≤ 10,000 variables

SYMBOLS

V, E, R, C, X: videos, endpoints, request lines, caches, cache MB · L: endpoint–cache links · K_{max}: most links of one endpoint · $W = \Sigma_e K_e \cdot O_e$: (cache, endpoint, requested video) triples · H: pairs in the heap · U: pushes back · r_v: endpoints asking for v · m: knapsack candidates (≈ 500)

DATA SET	V	E	R	C
me_at_the_zoo	100	10	100	10
videos_worth_spreading	10,000	100	100,000	100
trending_today	10,000	100	100,000	100
kittens	10,000	1,000	200,000	500

README.md · Complexity · PIPELINE.md §1–§4

1 ALGORITHM 2 EXPERIMENTS 3 CODE 4 CONCLUSION 5 QUESTIONS

WHAT YOU SEE

Cost of each step and the size of the Google data sets.

Word to know · NP-hard No known method solves every case both exactly and fast.